



第3章 存储系统III

- Cache存储器
- 虚拟存储器
- 鲲鹏920处理器片上系统的存储系统





3.6 Cache存储器



鲲鹏920处理器

- 集成最多64×TaiShan处理器内核

- 8个DDR4控制器 @2933MT/s

Cache架构

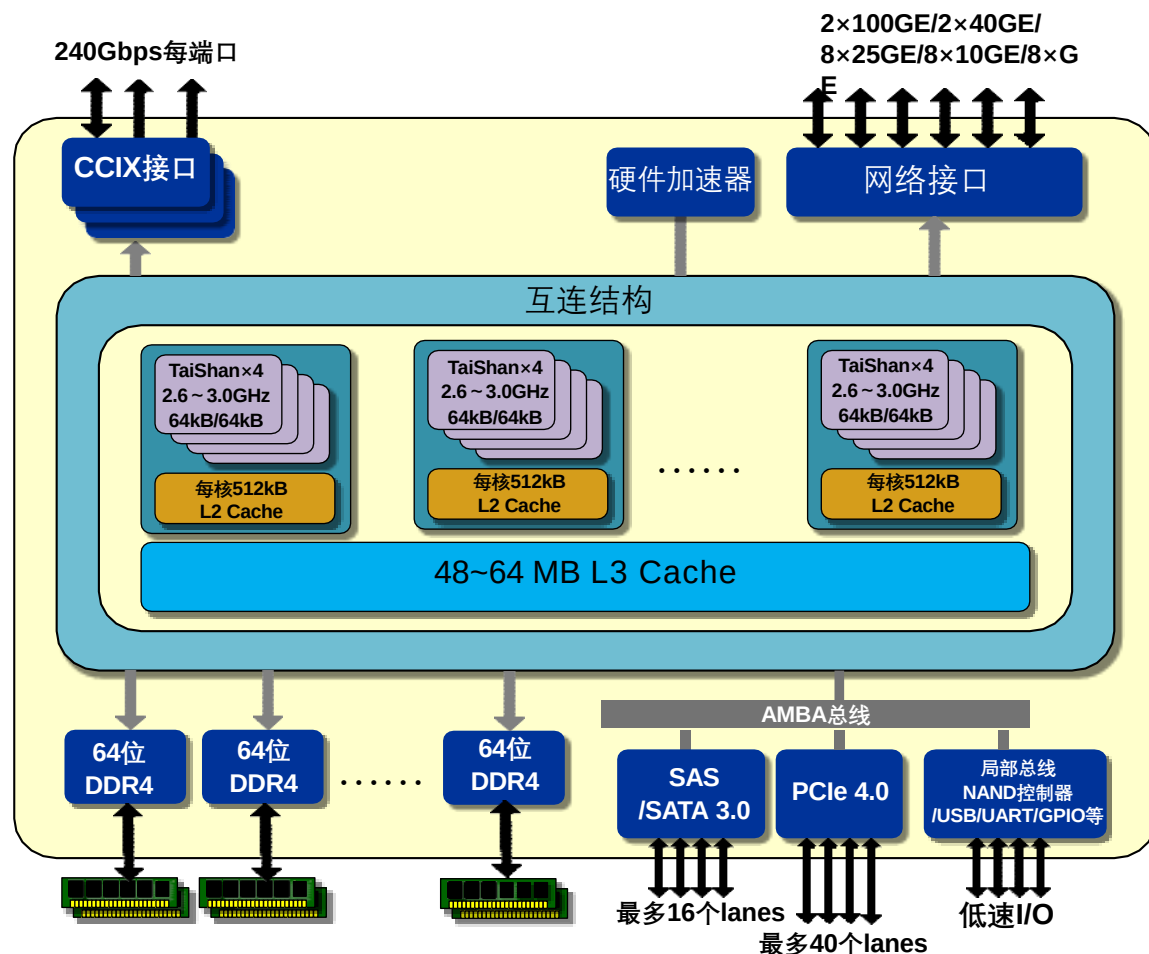
每个内核私有

- ◆ 64KB L1 I-Cache
 - ◆ 64KB L1 D-Cache
 - ◆ 512KB L2 Cache
- 片内共享

- ◆ 48 ~ 64MB L3 Cache

- 集成PCIe/SAS接口

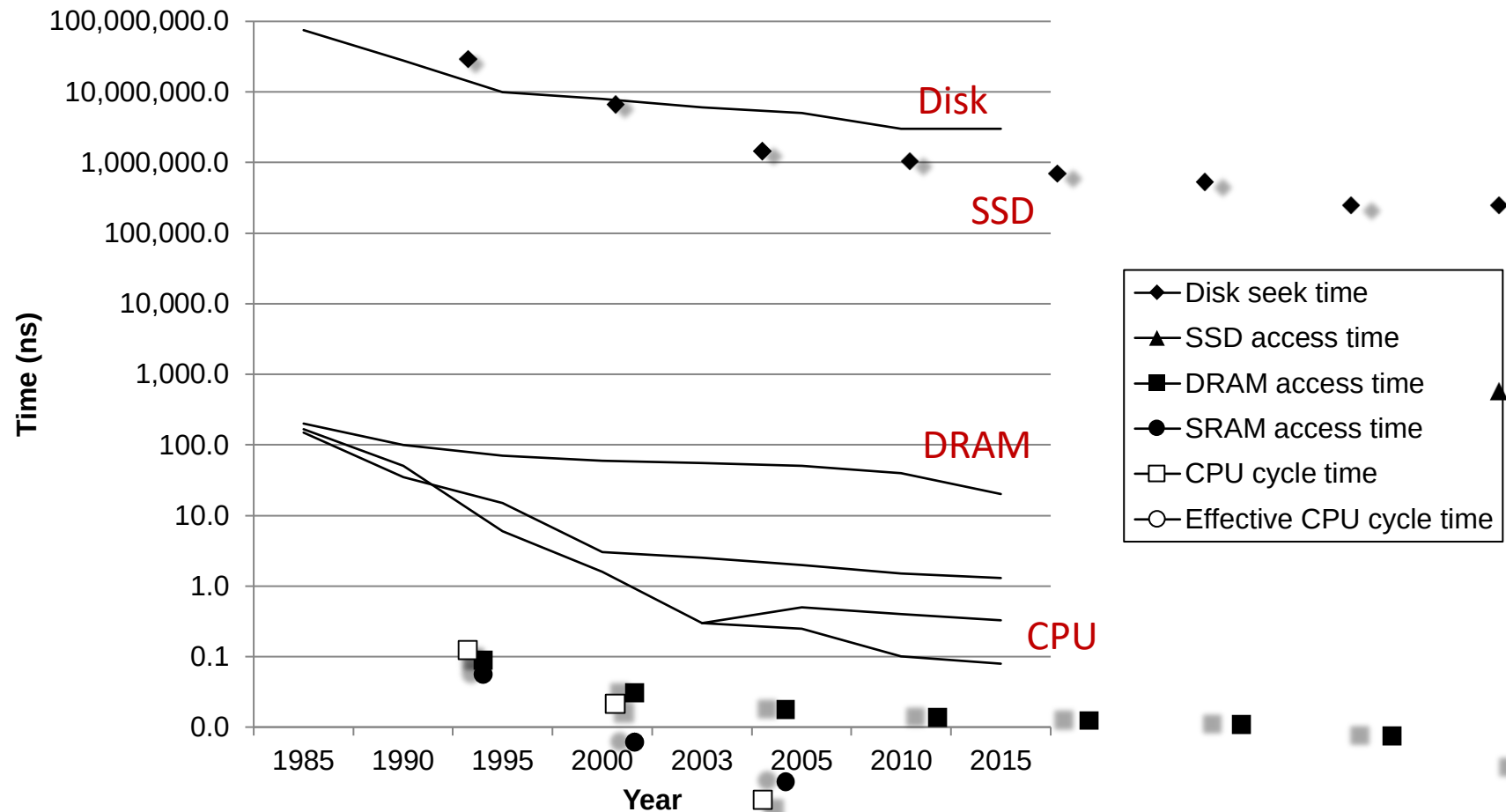
◆





The CPU-Memory Gap

The gap widens between DRAM, disk, and CPU speeds.



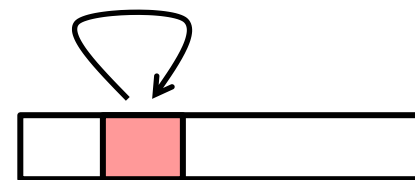


局部性原理

一个编写良好的程序常常具有良好的局部性(locality)。它倾向于一次又一次地访问相同的数据项集合，或是倾向于访问邻近的数据项集合。这种倾向性称之为**局部性原理** (principle of locality)

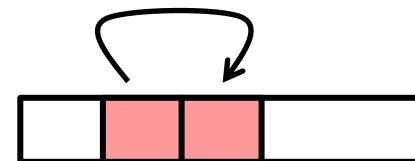
■ 时间局部性 (temporal locality)

在一个具有良好时间局部性的程序中，被引用过一次的内存位置很可能在不远的将来再被多次引用



■ 空间局部性 (spatial locality)

在一个具有良好空间局部性的程序中，如果一个内存位置被引用了一次，那么程序很可能在近期引用附近的一个内存位置





举例

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

是否具有良好的局部性？

■ 引用数据

- ◆ 以递增方式引用数组的元素
- ◆ 每一次循环引用变量sum

■ 引用指令

- ◆ 顺序引用指令
- ◆ 循环重复引用指令

空间局部性

时间局部性





程序局部性的定量观察

一个编写良好的程序

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

4.3ms

一个编写不太良好的程序

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

81.8ms

2.0 GHz Intel Core i7 Haswell

- 层次结构存储器组织(hierarchical memory organization)
 - ◆ 小容量高速缓存cache
- C语言数组按行优先方式存储在存储器中





两级存储体系的组成

■ CPU内部寄存器

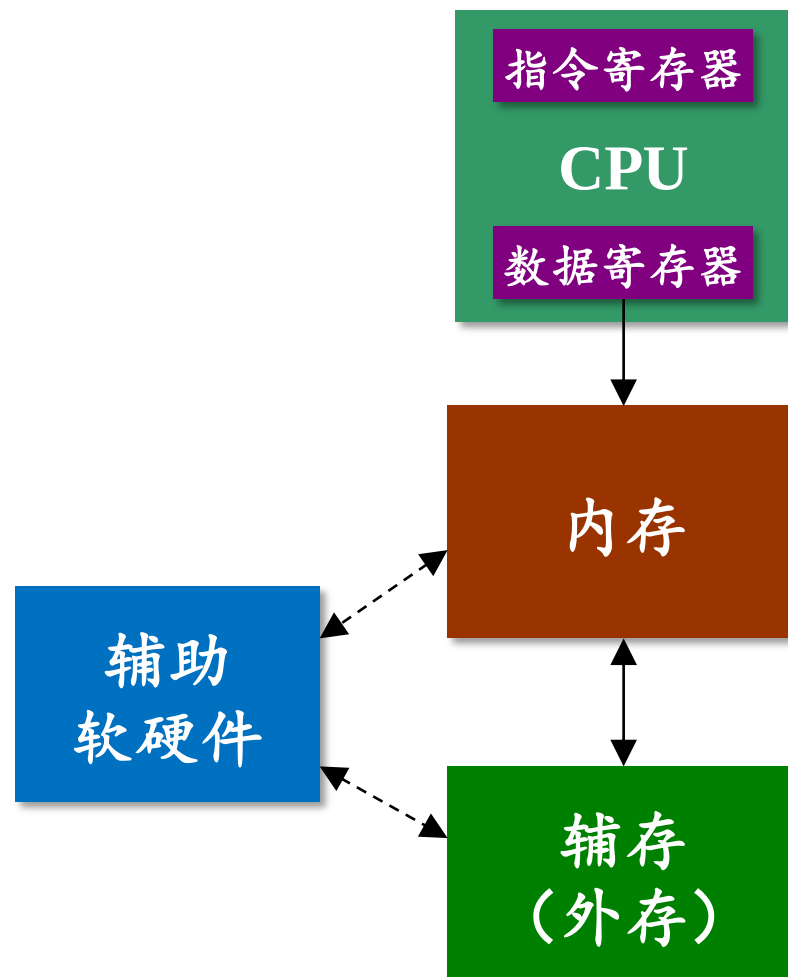
速度高、数量少

■ 内存

速度高、容量小、价格高，
由半导体器件构成

■ 外存（辅存）

速度低、容量大、价格便宜，
由非半导体器件构成



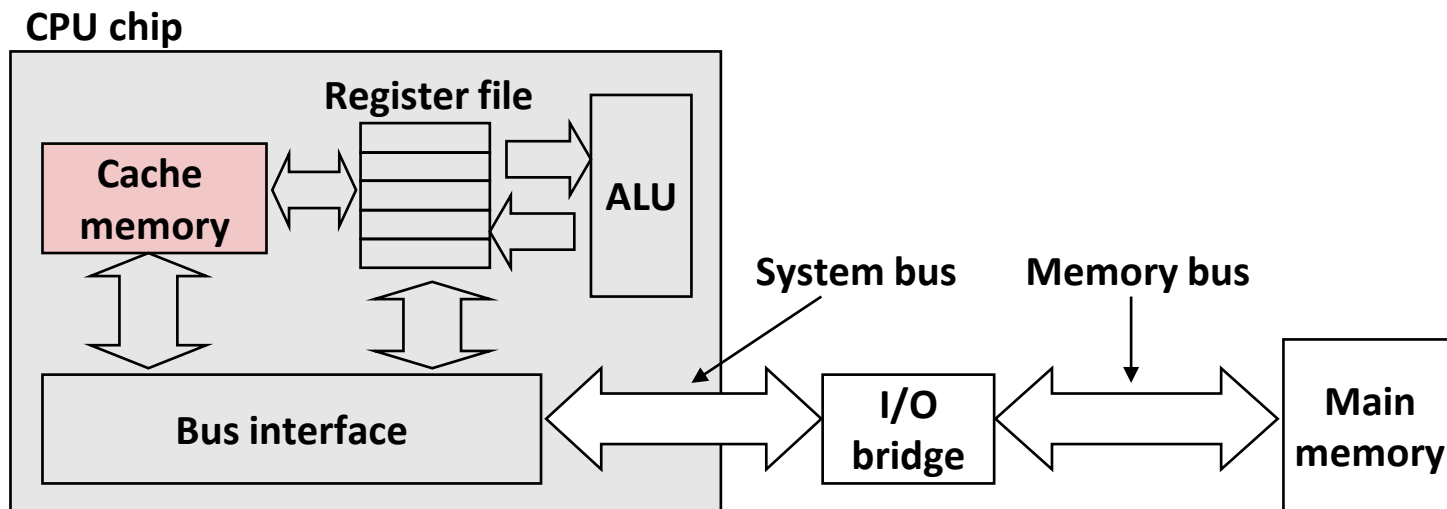
引入外存主要是解决内存容量小的问题





高速缓存Cache

- Cache是一个小而快速的存储部件，它作为存储在更大、也更慢的设备中的数据对象的**缓存区域**
- Cache通常采用高速SRAM来实现
- 利用局部性原理，在Cache中完成大多数访问，从而缩短平均访问时间
- CPU和主存间速度差很大时通常采用两级或多级Cache系统
- **全由硬件调度**，对用户透明



三级存储体系的组成

- CPU内部寄存器
速度高、数量少

- 内存

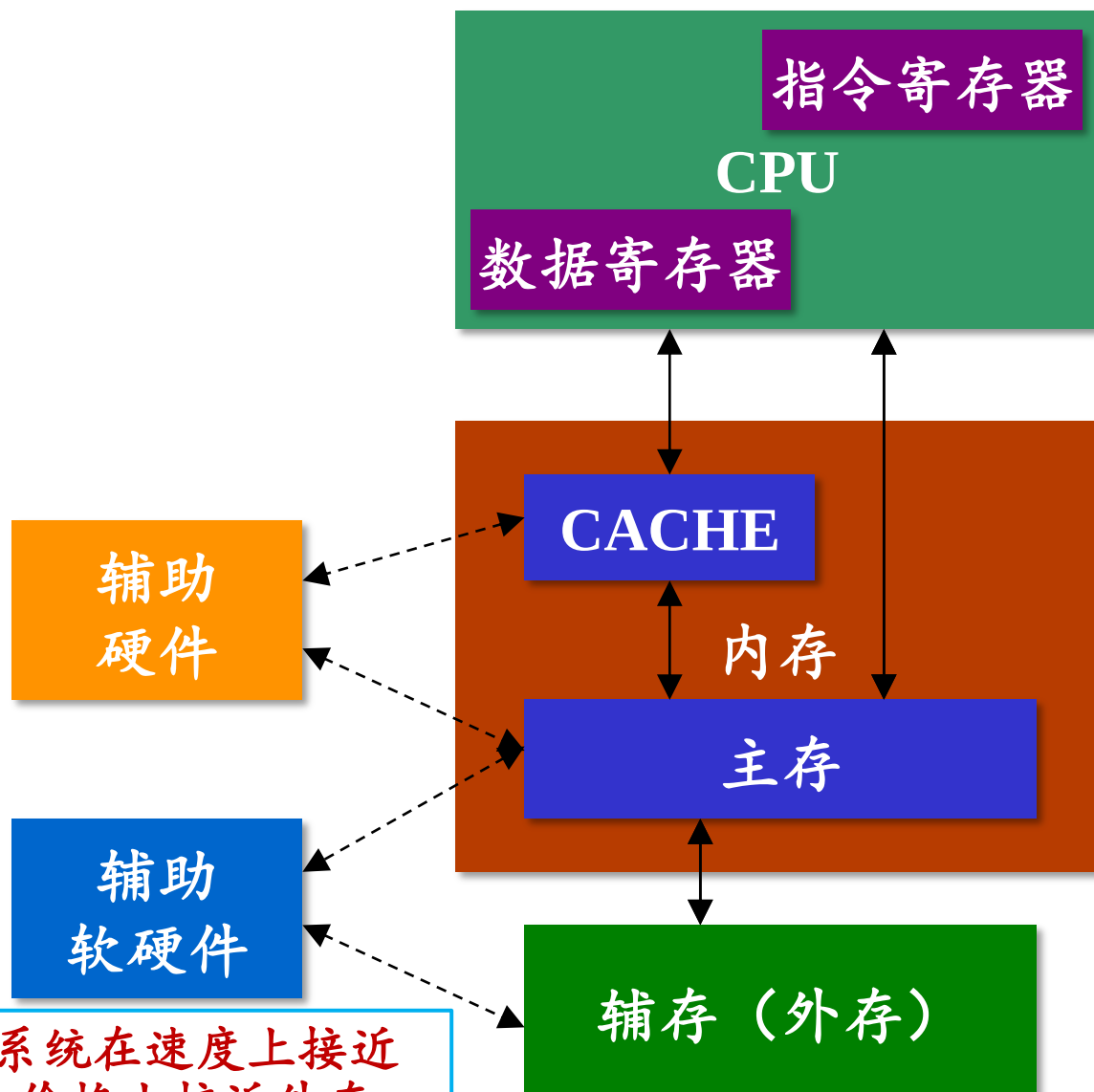
- ◆ 高速缓存cache

引入cache是要解决
主存速度慢的问题

- ◆ 主存

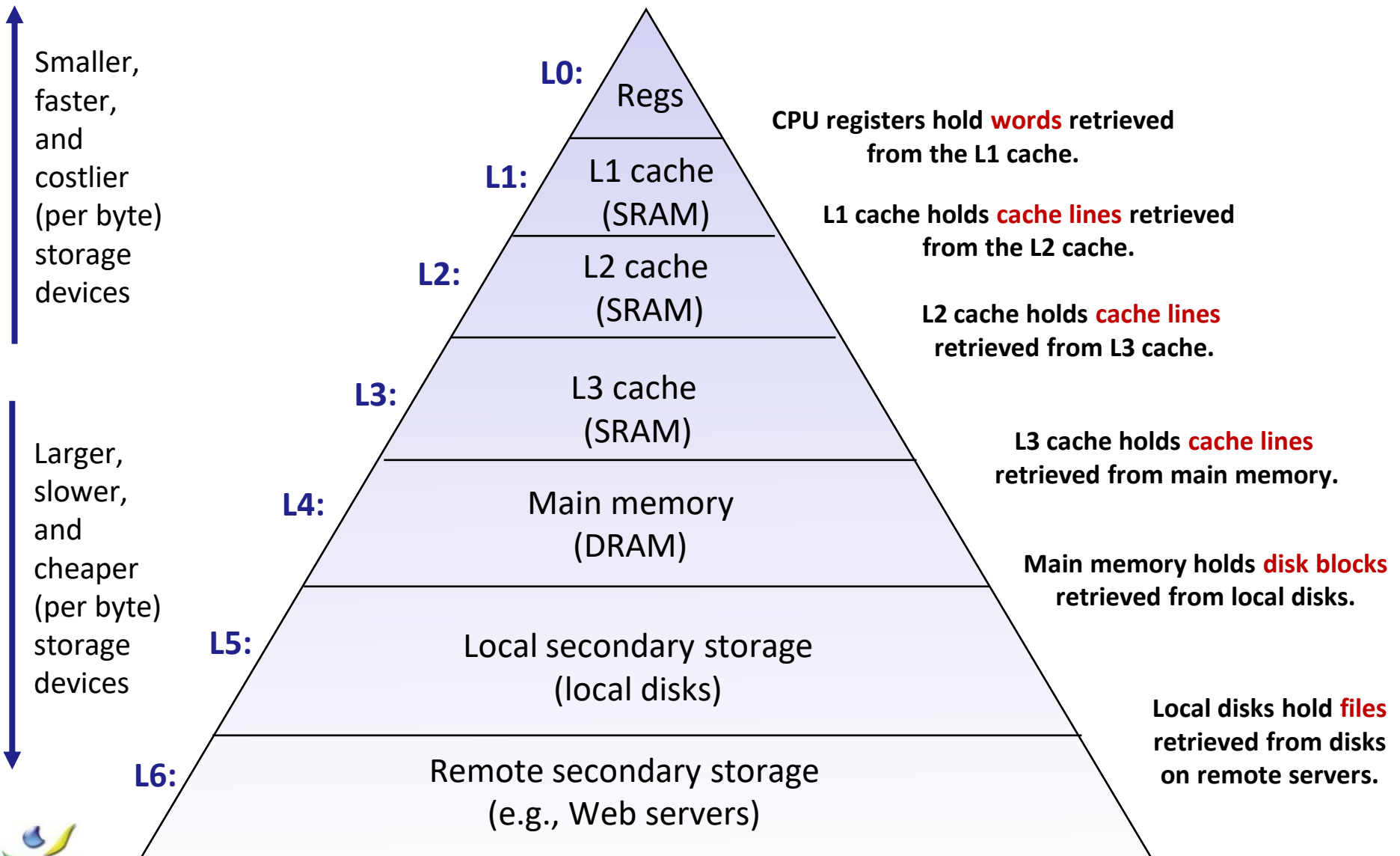
- 辅存

目的：整个存储系统在速度上接近
cache，在容量和价格上接近外存





存储系统的层次结构



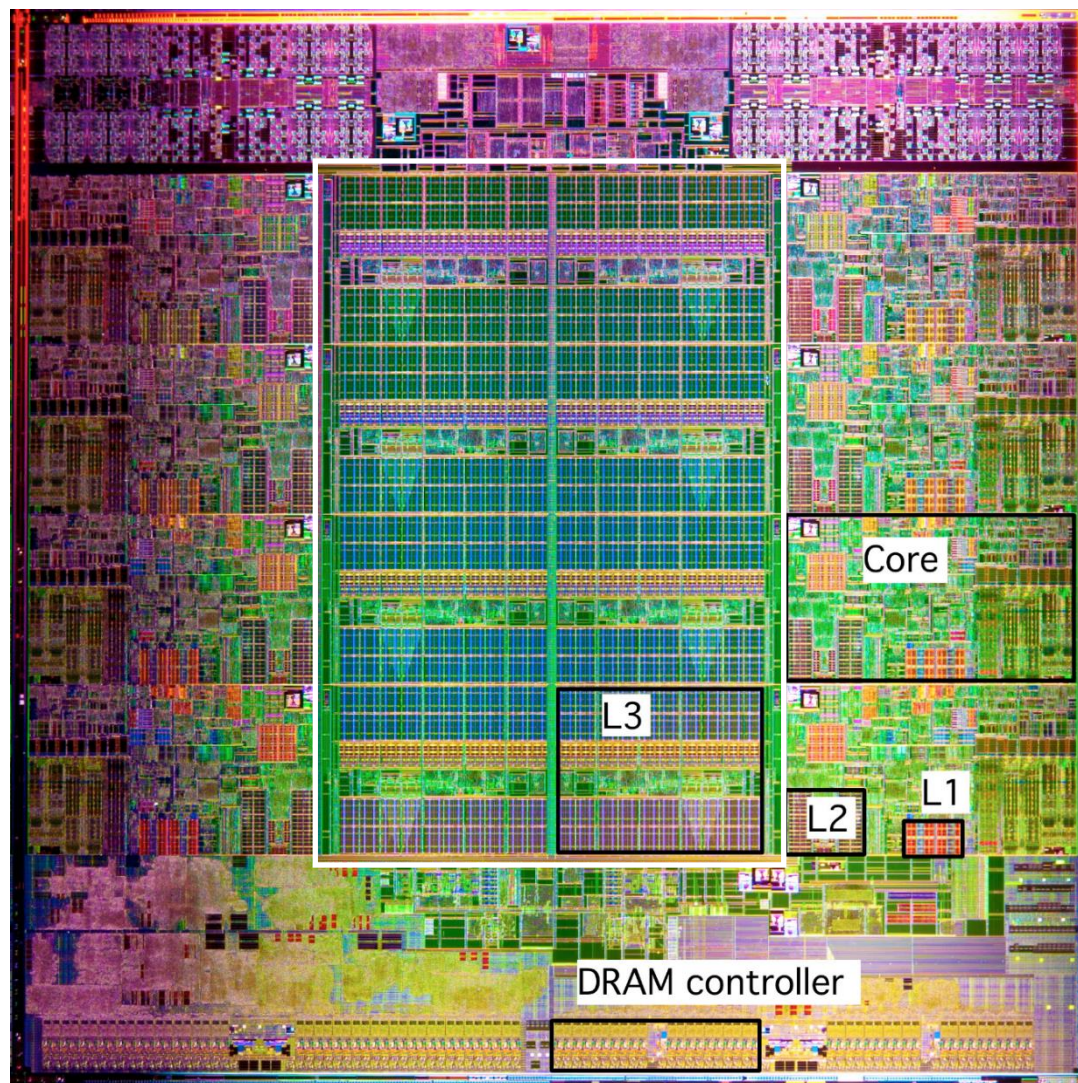


层次结构实例

Cache Type	What Cached	Where Cached	Latency (cycles)	Managed By
Registers	4-byte word	CPU registers	0	Compiler
TLB	Address translations	On-Chip TLB	0	Hardware
L1 cache	32-byte block	On-Chip L1	1	Hardware
L2 cache	32-byte block	On-Chip L2	10	Hardware
Virtual Memory	4-KB page	Main memory	100	Hardware+ OS
Buffer cache	Parts of files	Main memory	100	OS
Network buffer cache	Parts of files	Local disk	10,000,000	AFS/NFS client
Browser cache	Web pages	Local disk	10,000,000	Web browser
Web cache	Web pages	Remote server disks	1,000,000,000	Web proxy server



Intel Sandy Bridge Processor Die



8 cores

L1: 32KB Instruction + 32KB Data

L2: 256KB

L3: 3 ~ 20MB

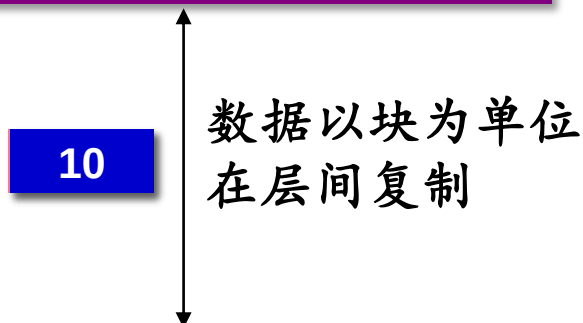
通过观察cache占用整个硅芯片面积的比例，可以看出cache的重要性



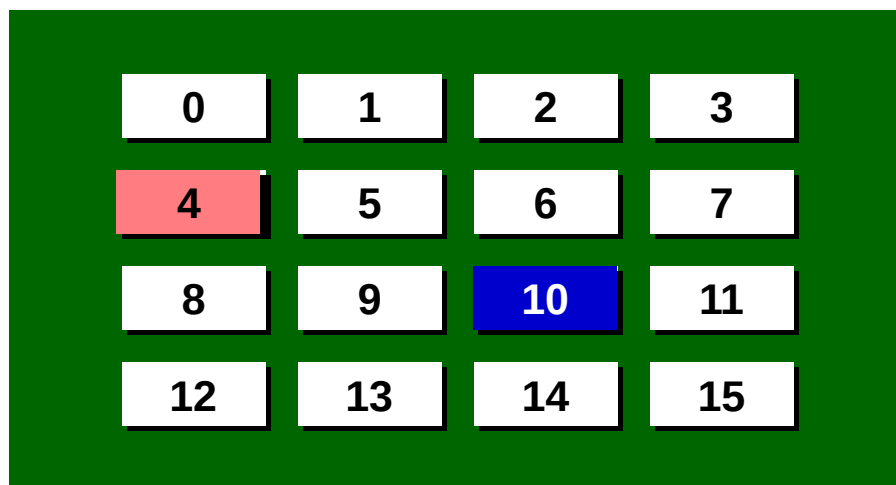
Cache基本原理 (1)



较小、较快、更贵的存储器
缓冲来自下层数据块的子集



Memory

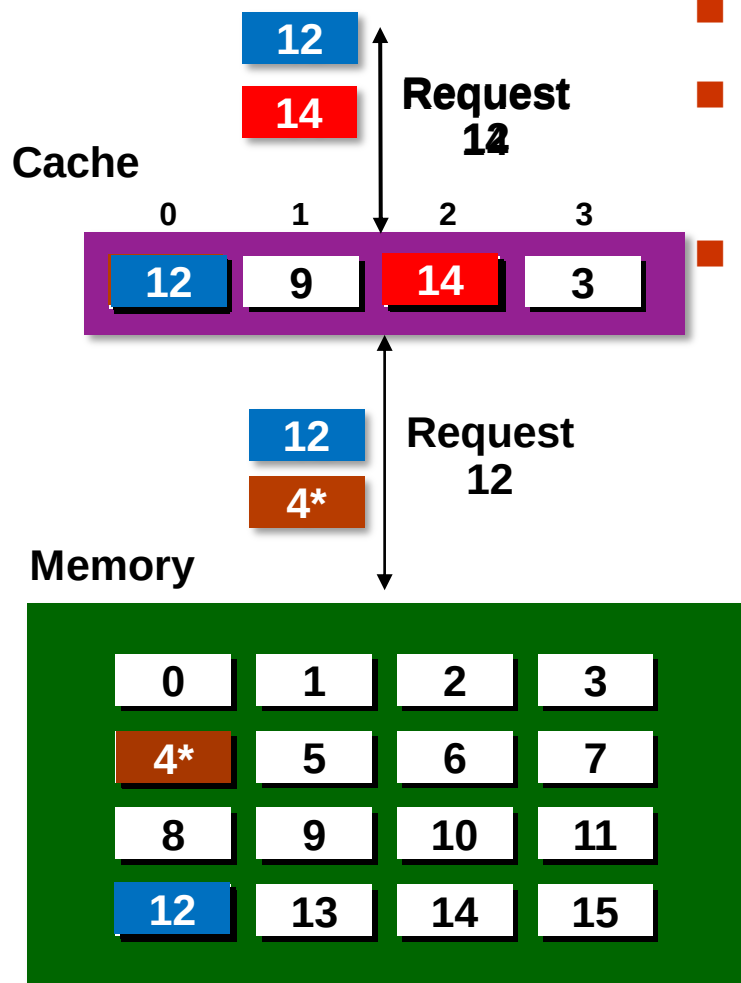


较大、较慢、更便宜的存储器
上的数据分成若干块





Cache基本原理 (2)



- 程序需要访问块b中的对象d.
- Cache命中 (Cache hit)
 - ◆ 程序在Cache找到b, 例如: 块14
- Cache缺失 (Cache miss)
 - ◆ 块b不在层k中, 所以cache必须下一层去取它, 例: 块12.
 - ◆ 如果层cache已满, 则当前的某块必须被替换。哪块被替换出去?
 - 替换策略: 哪一块被替换出? 例如: 最久未使用LRU策略
 - 放置策略: 新块放在何处? 例如: $b \bmod 4$





Cache缺失类型

- 冷启缺失/义务缺失 (Cold miss or Compulsory miss)
 - ◆ 当cache为空时就产生冷启缺失
- 冲突缺失 (Conflict miss)
 - ◆ 层 $k+1$ 的数据块通常只能映射到层 k 的有限位置上。尽管层 k 中cache足够大，若层 $k+1$ 有多个数据块全都映射到层 k 的同一位置上，则产生冲突缺失
- 容量缺失 (Capacity miss)
 - ◆ 当活动的cache块的数量超过cache的容量就产生容量缺失



动手做

下列有关RAM和ROM的叙述中，正确的是（ ）

I、 RAM是易失性存储器， ROM是非易失性存储器

II、 RAM和ROM都是采用随机存取的方式进行信息访问

III、 RAM和ROM都可用作Cache

IV、 RAM和ROM都需要进行刷新

A: 仅I和II

B: 仅II和III

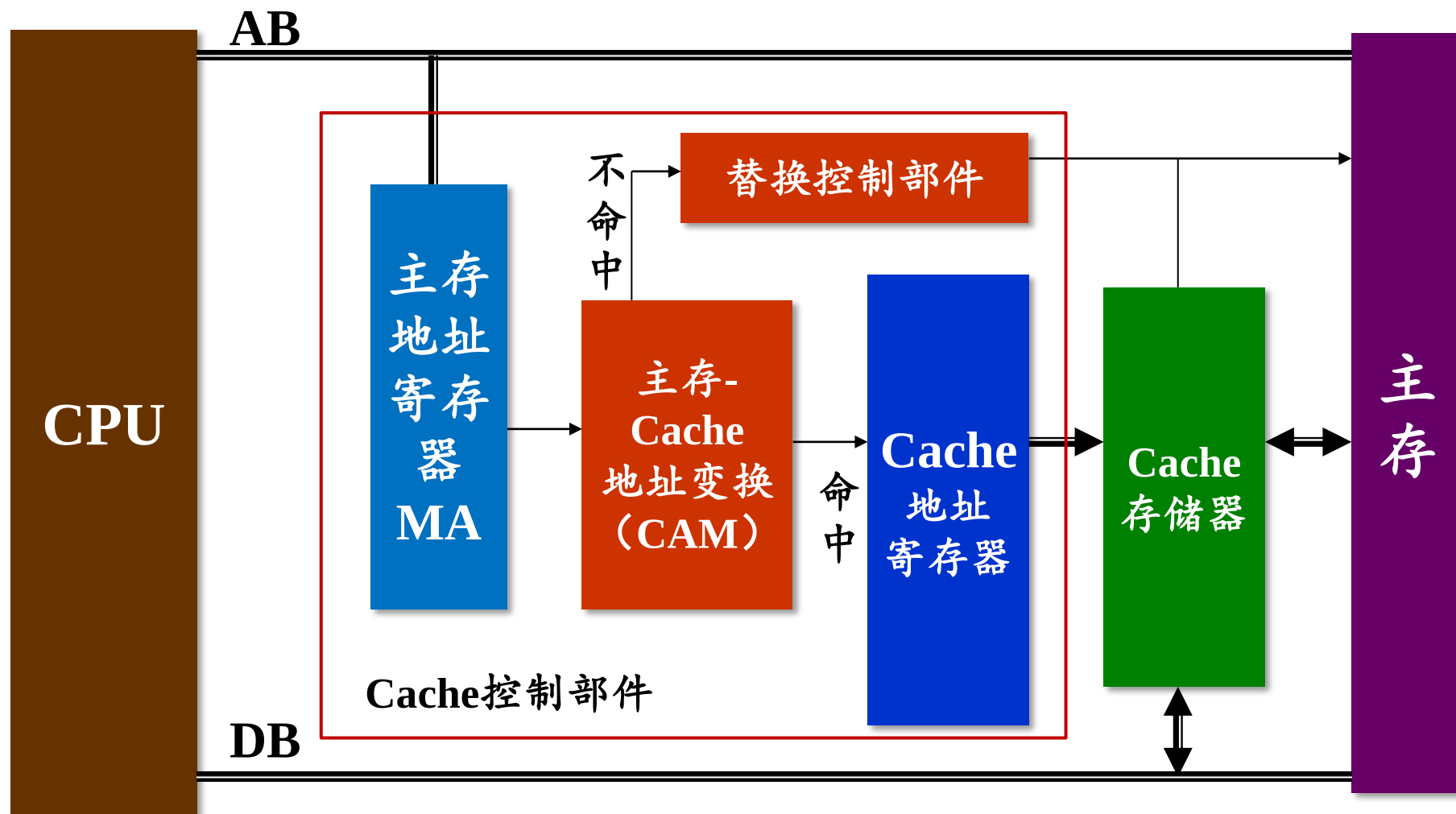
C: 仅I, II, III

D: 仅II, III, IV





Cache的基本工作原理





cache控制部件的功能

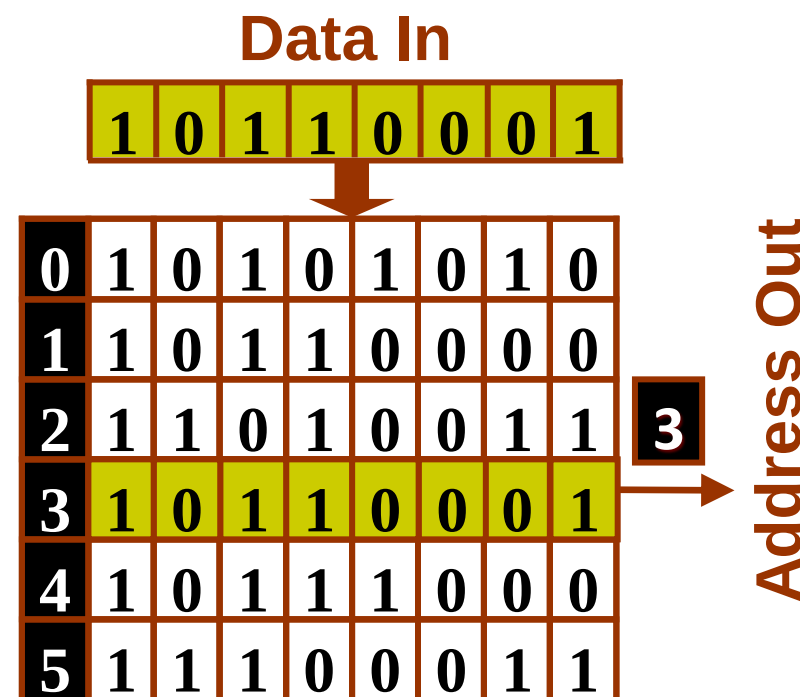
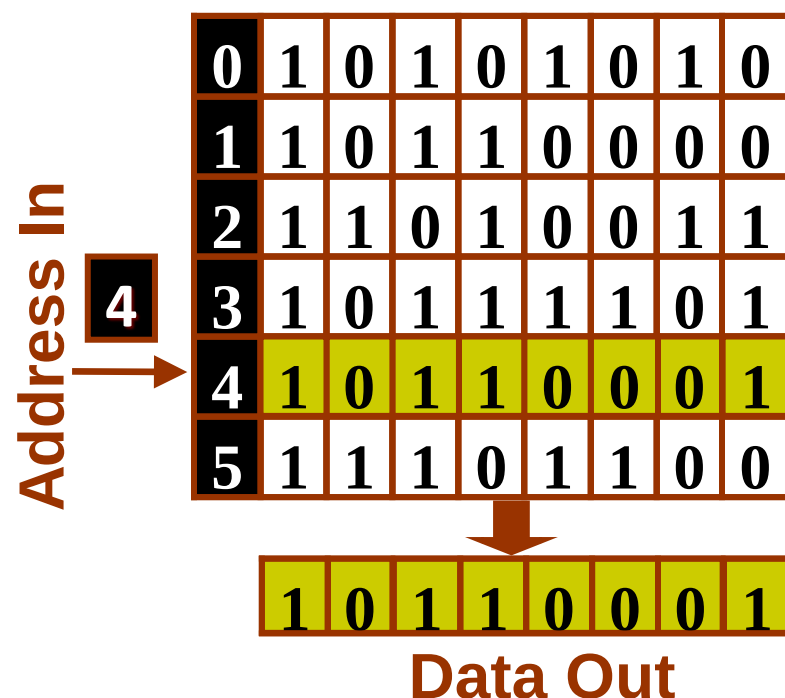
- 地址变换：如何将主存中的地址映射成**cache**中的地址之间
- 替换算法：在**cache**不命中时如何替换**cache**中的内容
- 更新算法：如何保持主存与**cache**的一致性





内容可寻址存储器CAM

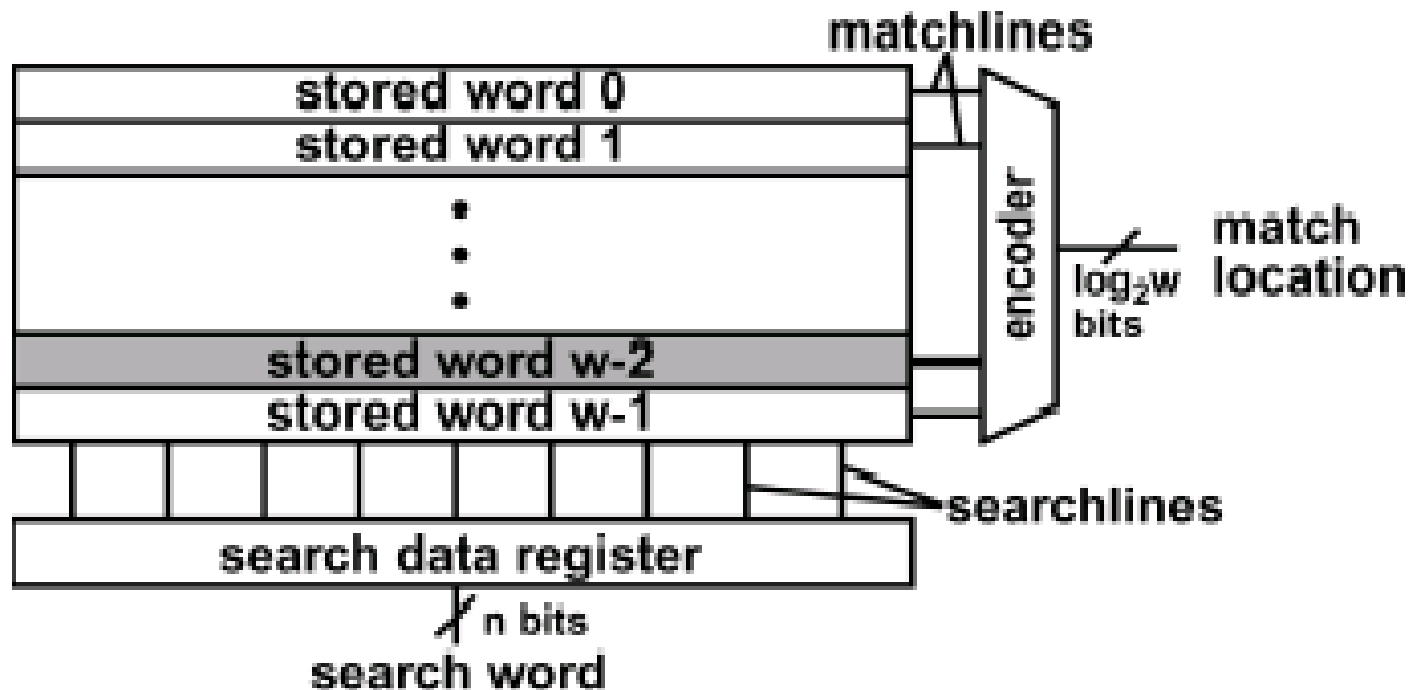
- CAM (Content Addressable Memory) 是一种用内容进行寻址的存储器。将一个输入数据项与存储的所有数据项同时进行比较，若存在匹配，输出该数据项对应的匹配信息（或地址）。同时也能按地址进行读和写。
- RAM vs. CAM





简化的CAM功能框图

- Search Word: 输入检索字
- 检索字在search lines上广播
- Match line: 检索字与存储字是否匹配
- Encoder输出匹配位置





二进制CAM单元

SL: search line

BL: bit line

WL: word line

ML: match line

Red: "1"

Blue: "0"

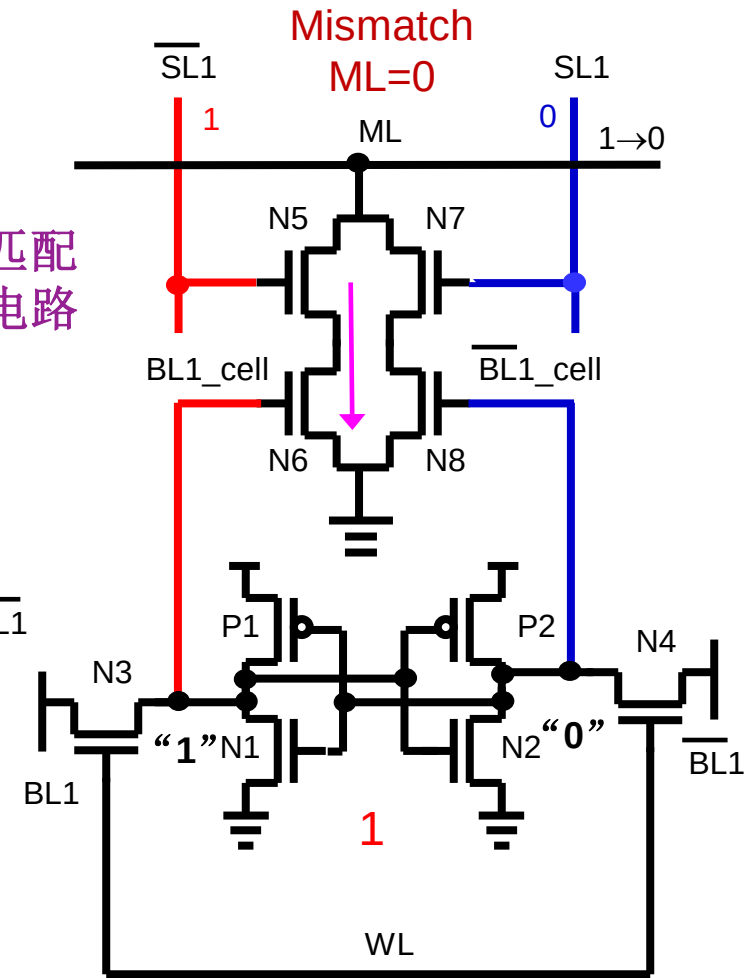
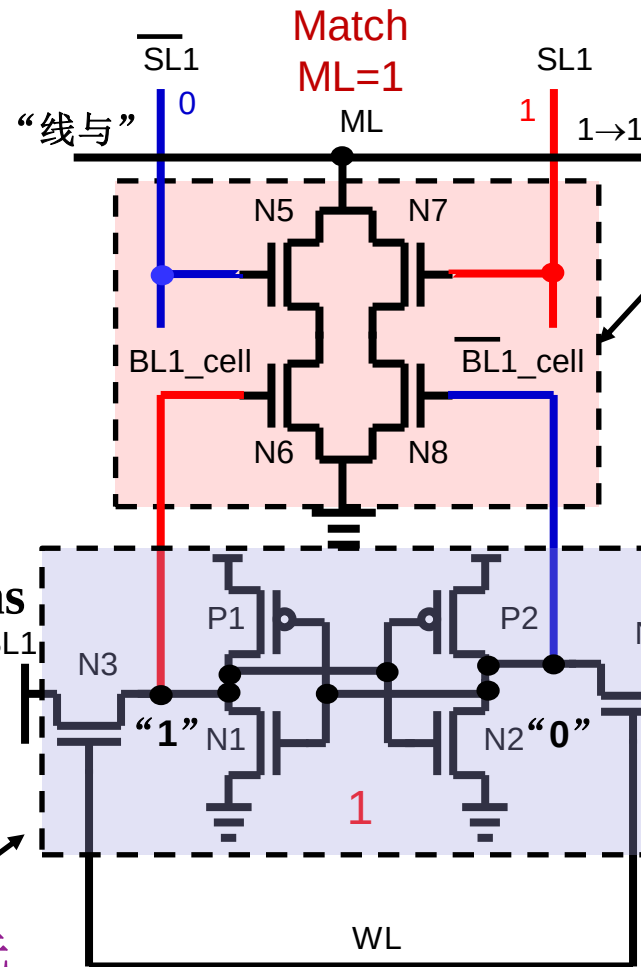
ML pre-charged to VDD

Match: ML remains at VDD

Mismatch: ML discharges

存储单元

匹配电路





主存和cache的分块

	容量	块（行）大小	块数
主存	2^n	2^b	$2^n/2^b=2^m$
cache	2^l	2^b	$2^l/2^b=2^c$

- 主存和cache间以数据块为单位进行复制，为便于管理，将主存和cache都划分为大小相等的数据块（也称**行**），所以，只需对块地址进行映射

主存地址（n位） = 主存块号（m位） + 主存块内地址（b位）

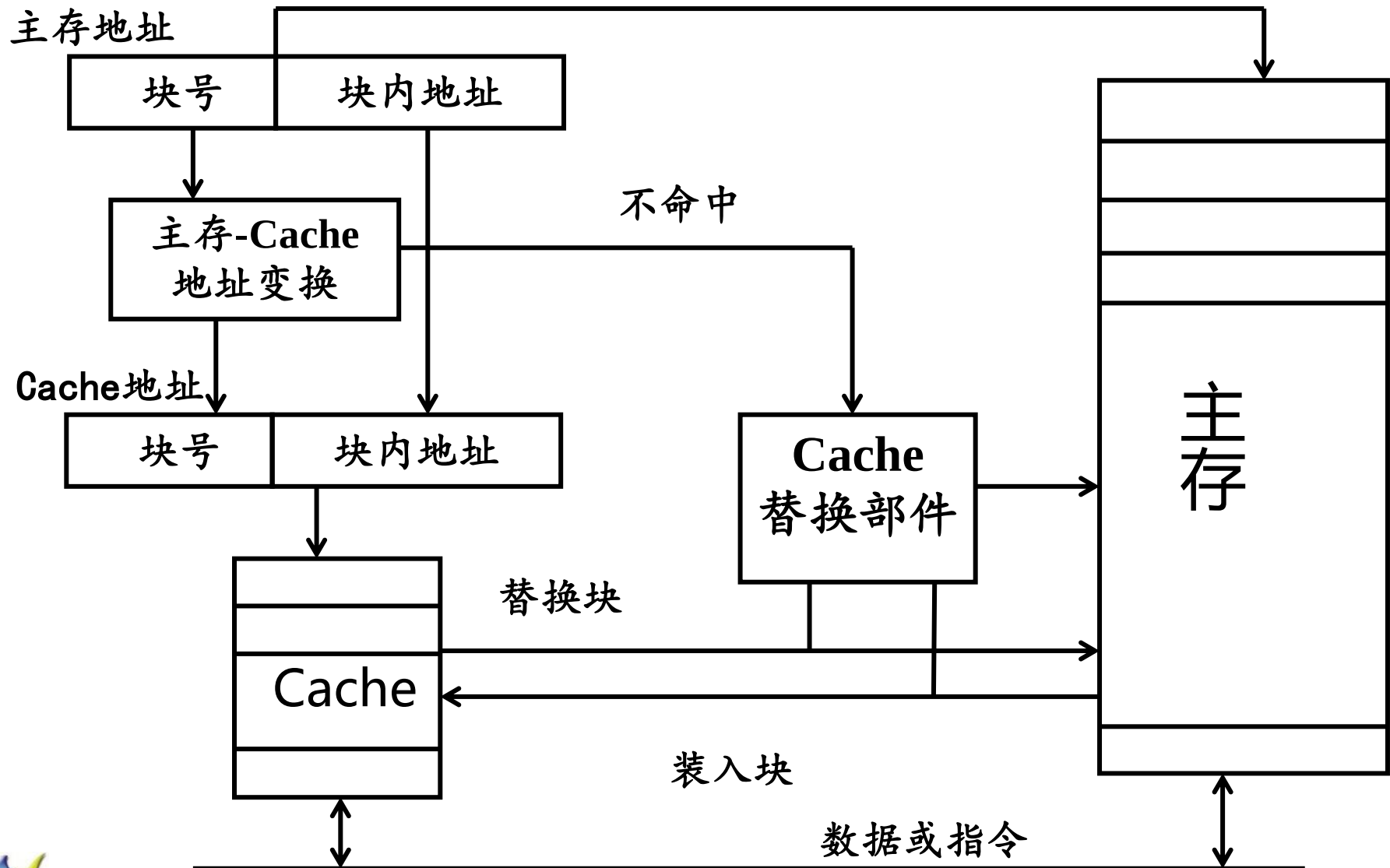
cache地址（l位） = cache块号（c位） + cache块内地址（b位）

其中， $c \ll m$





Cache的结构





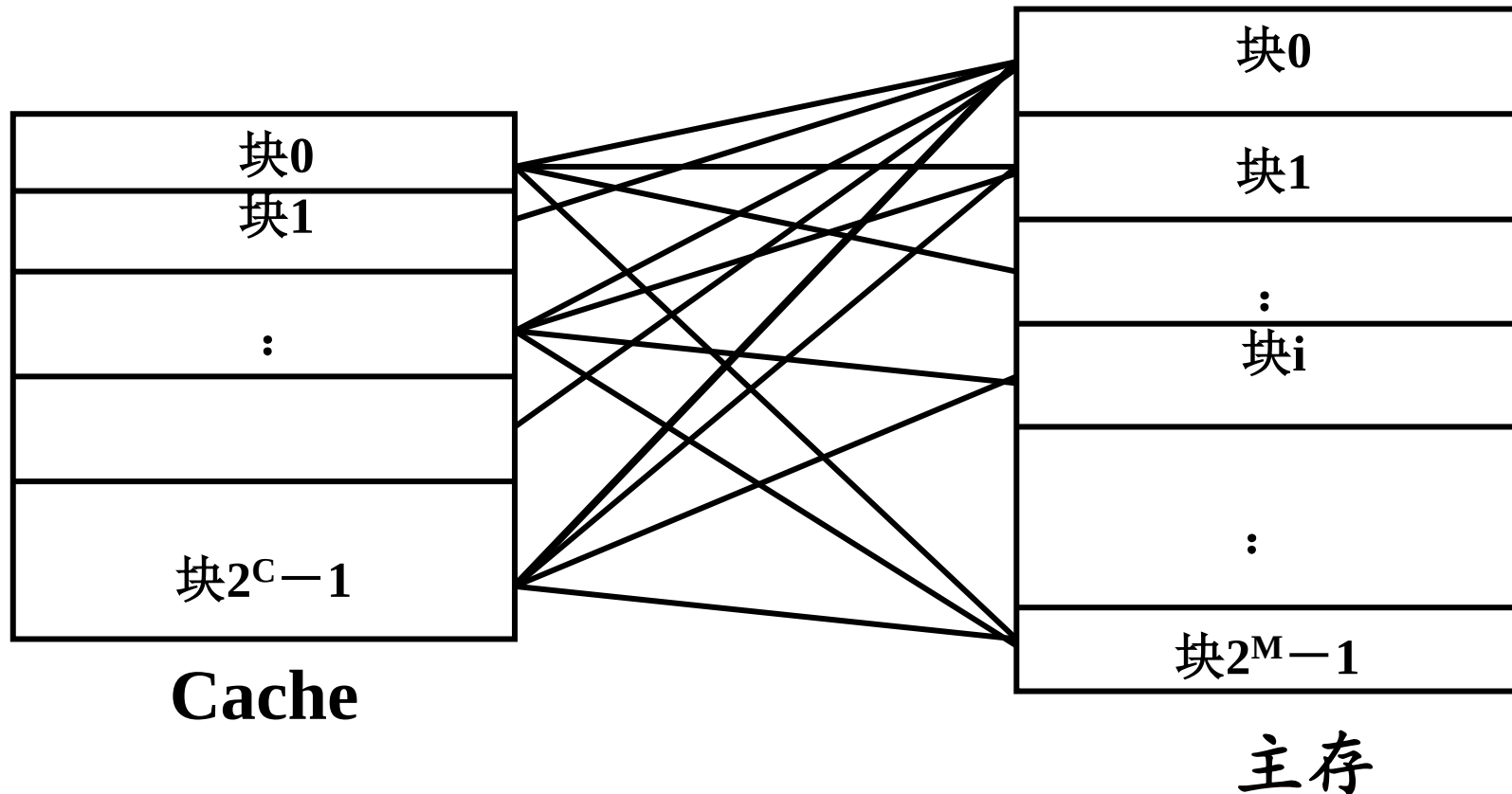
主存与Cache的地址映射

- 为了把主存块放到Cache中，必须应用某种方法把主存地址变换成Cache中的地址，称作**地址映射**（**地址变换**）
- 地址映射方式
 - ◆ 全相联映射（Fully Associative Mapping）
 - ◆ 直接映射（Direct Mapping）
 - ◆ 组相联映射（Set Associative Mapping）



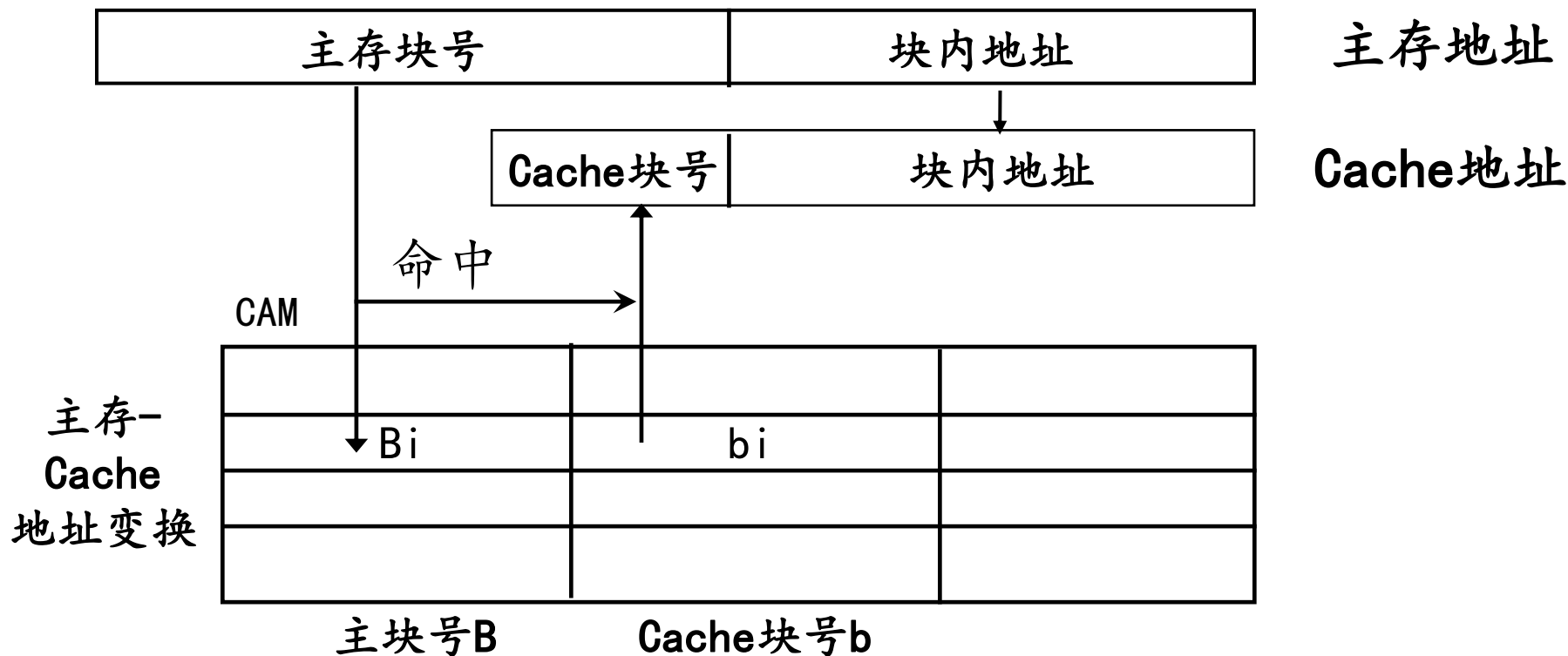


1、全相联映射（1）





全相联映射 (2)



优点：Cache的存储空间利用率高，命中率较高

缺点：需要CAM存储器，线路复杂，成本高

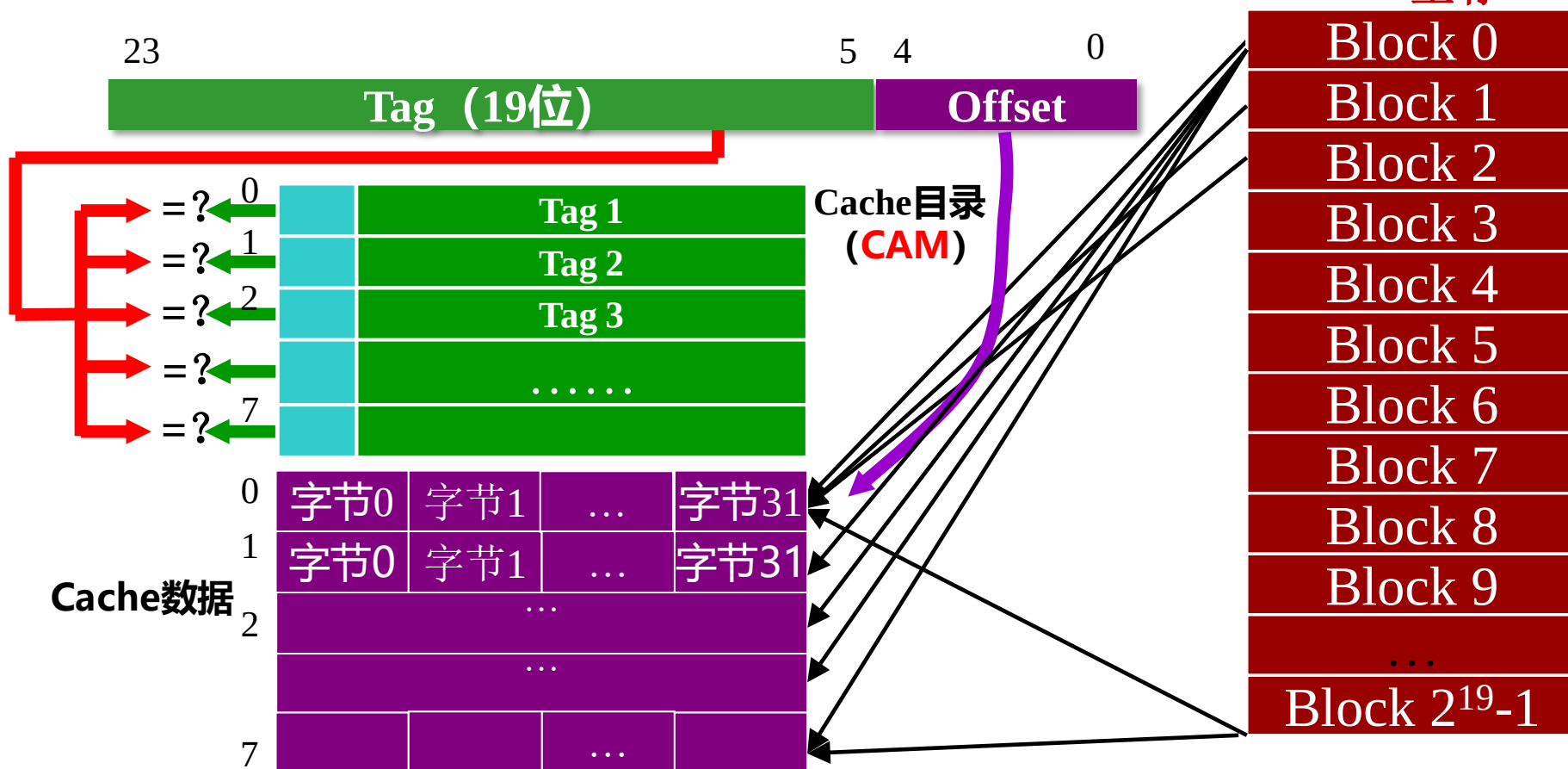




全相联映射Cache示例

- 主存储器地址宽度：24位；容量： 2^{24} bytes； $2^{24}/2^5=2^{19}$ 块
- Cache容量：256 bytes，块大小为 32 bytes； $256/32=8$ 块

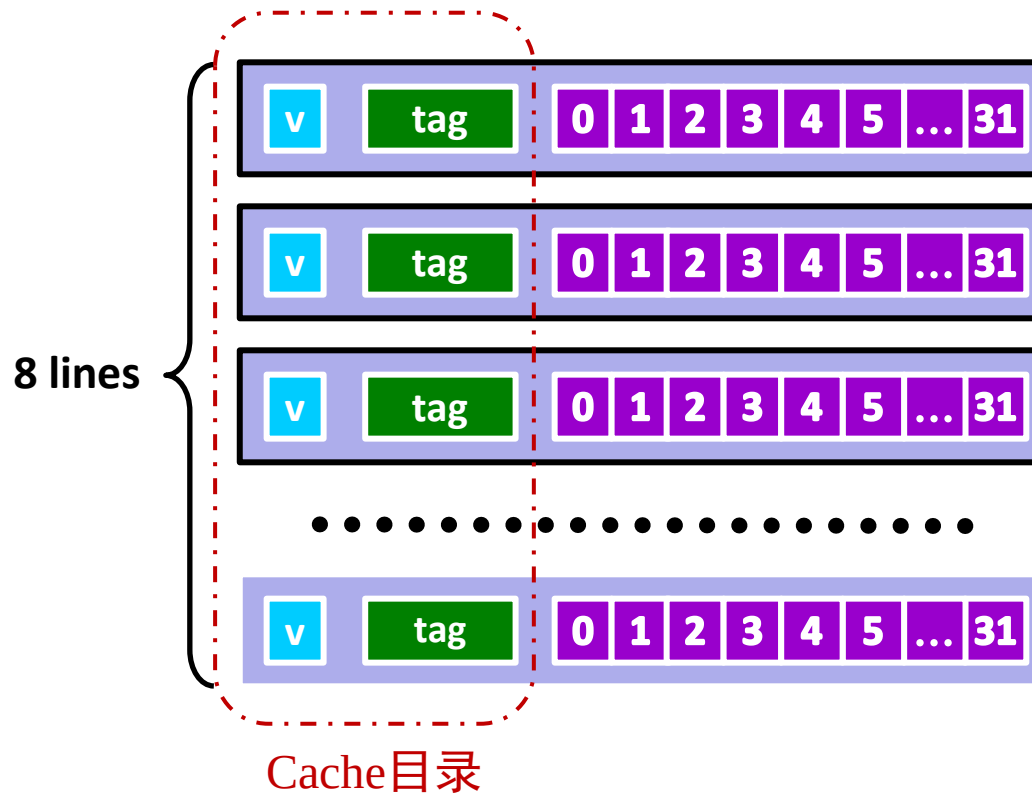
主存





Cache组织结构

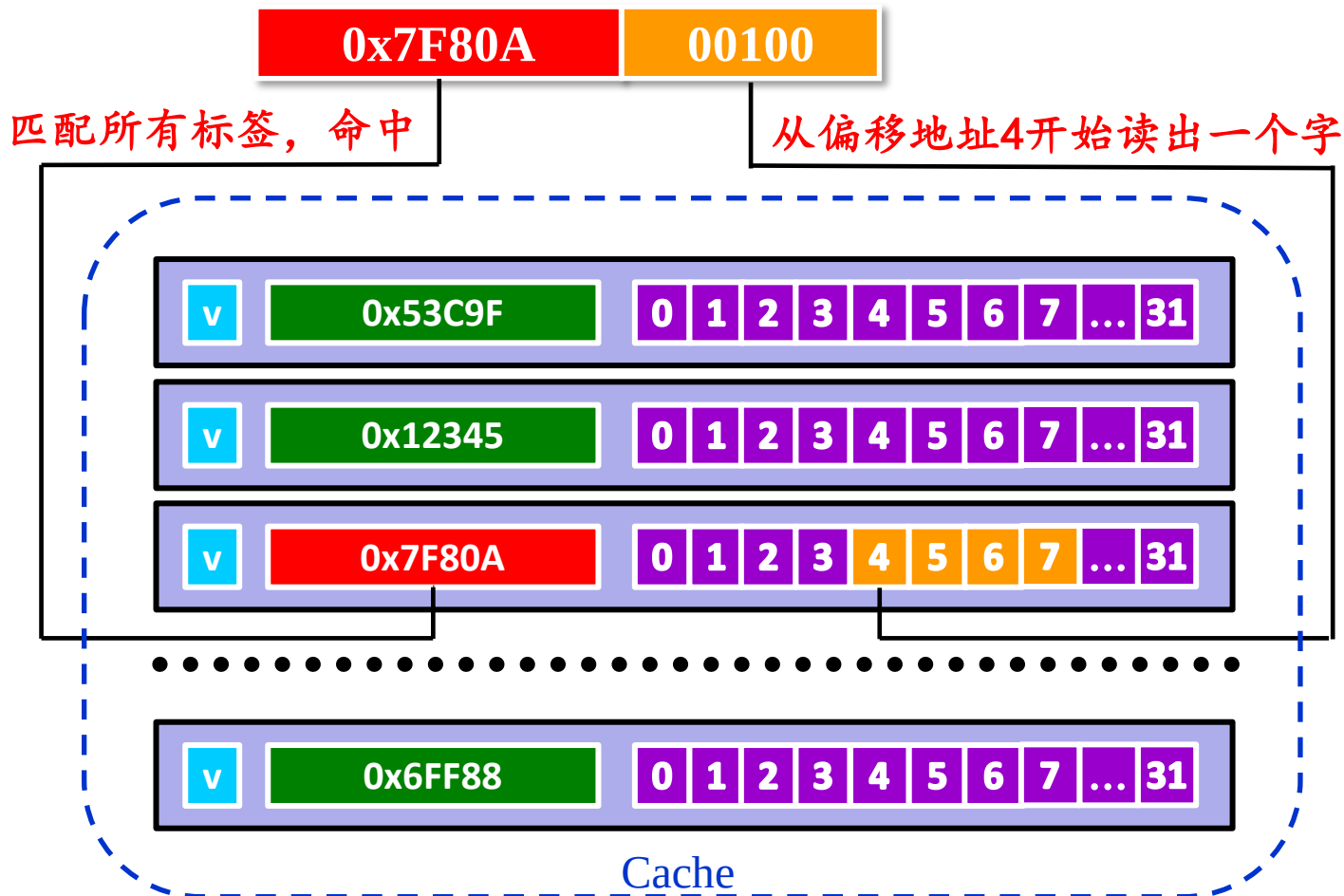
- 主存地址的tag字段与Cache目录中匹配后，意味着找到了所在的行，然后用offset访问数据
- 从图中可看出称之为“行”的原因





全相联映射Cache中的行匹配和字选择过程

- 假设CPU欲从0xFF0144地址读出一个字（32位）





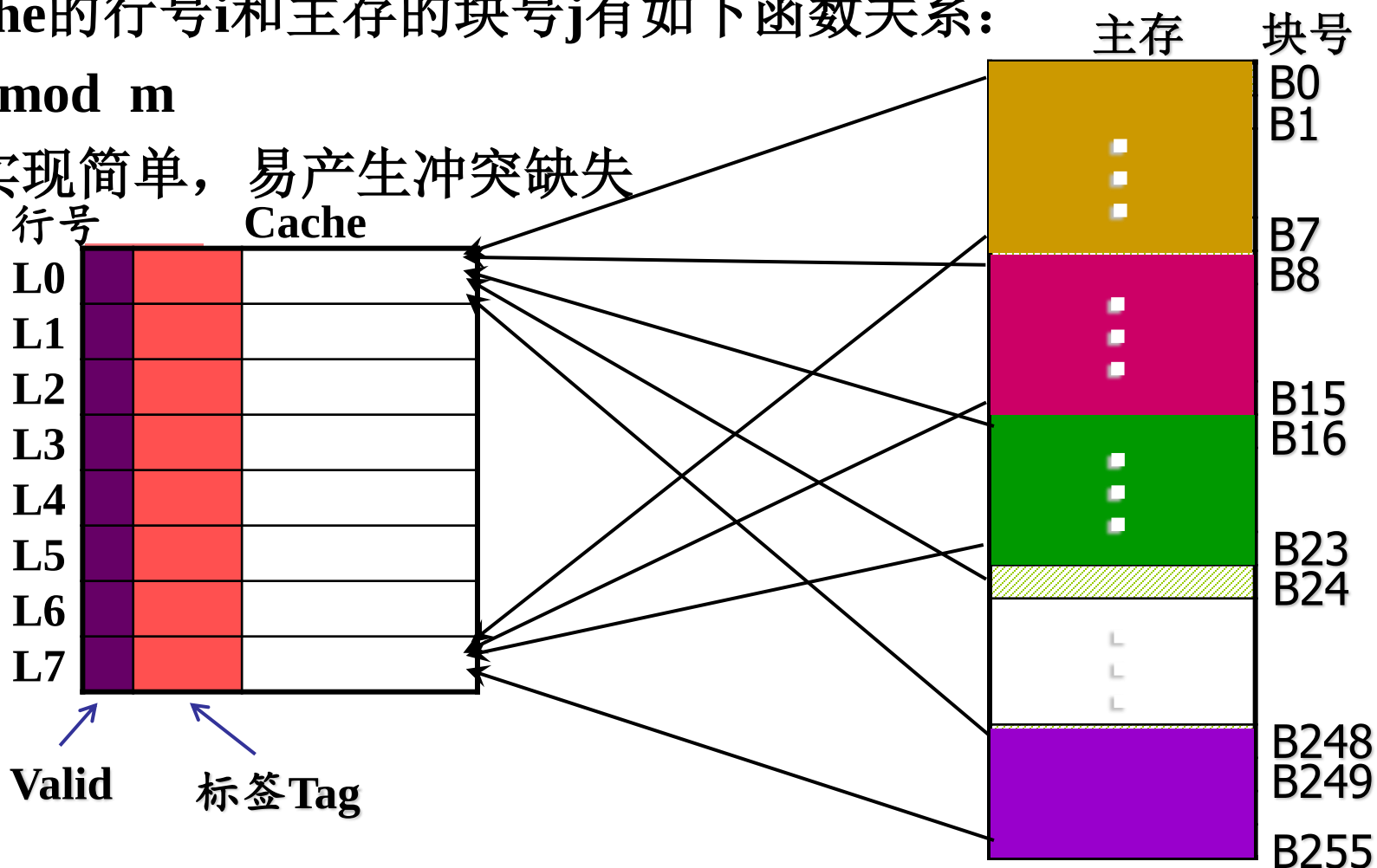
2、直接映射

- 直接映射是一种**多对一**的映射关系，一个主存块只能复制到Cache的一个特定行位置上去

- m 行Cache的行号 i 和主存的块号 j 有如下函数关系：

$$i = j \bmod m$$

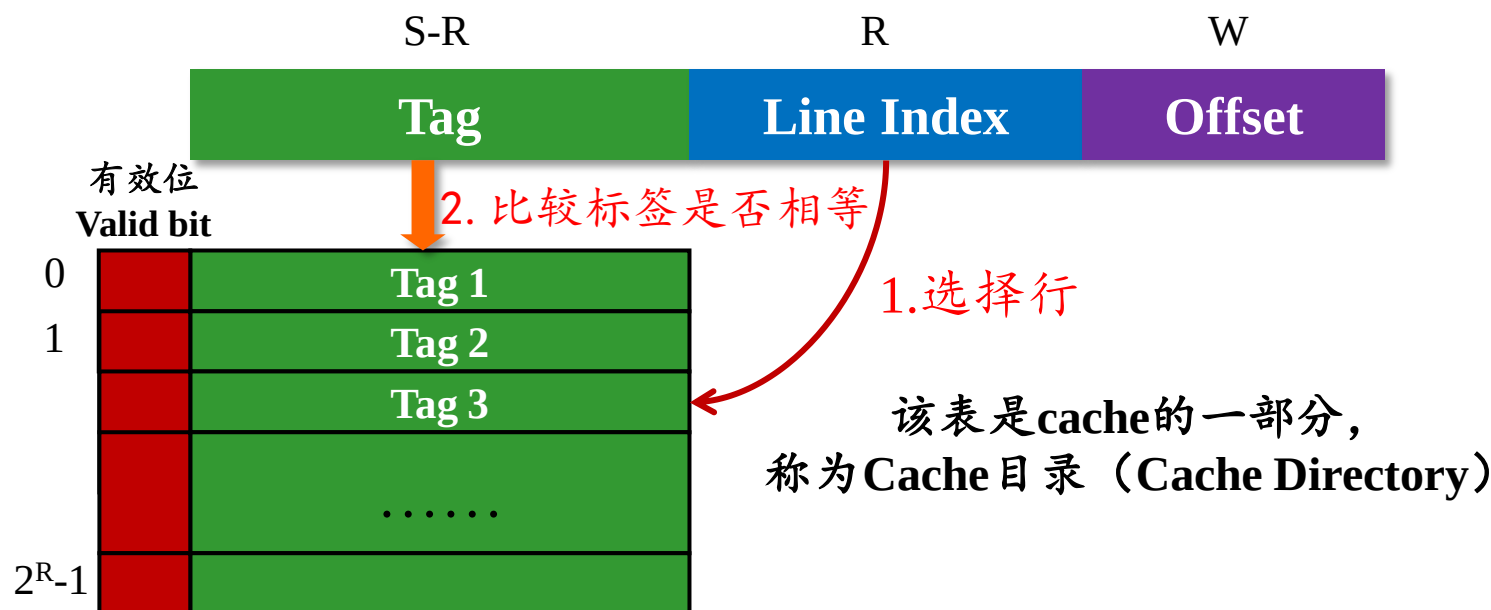
- 特点：实现简单，易产生冲突缺失





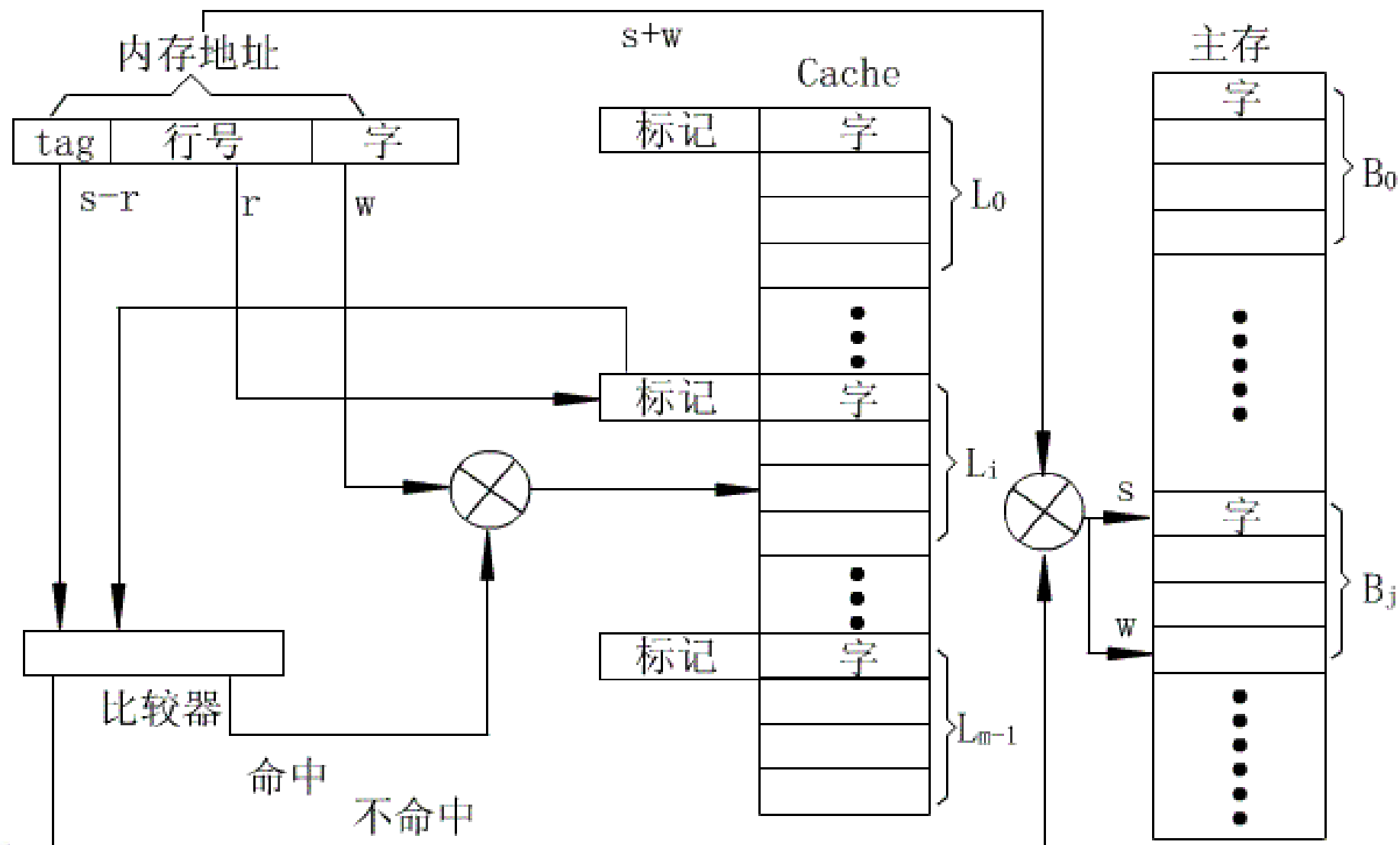
直接映射的Cache标签和行号

- 块大小 2^W 字节
- 假设存储器按字节寻址，其地址宽度为 $S+W$ 位
- 直接映射cache的空间为 2^R 个行
 - ◆ Cache行号（**Line**）：存储器地址中的 R 位。也称行索引
 - ◆ Cache标签（**Tag**）：存储器地址的 $S - R$ 位





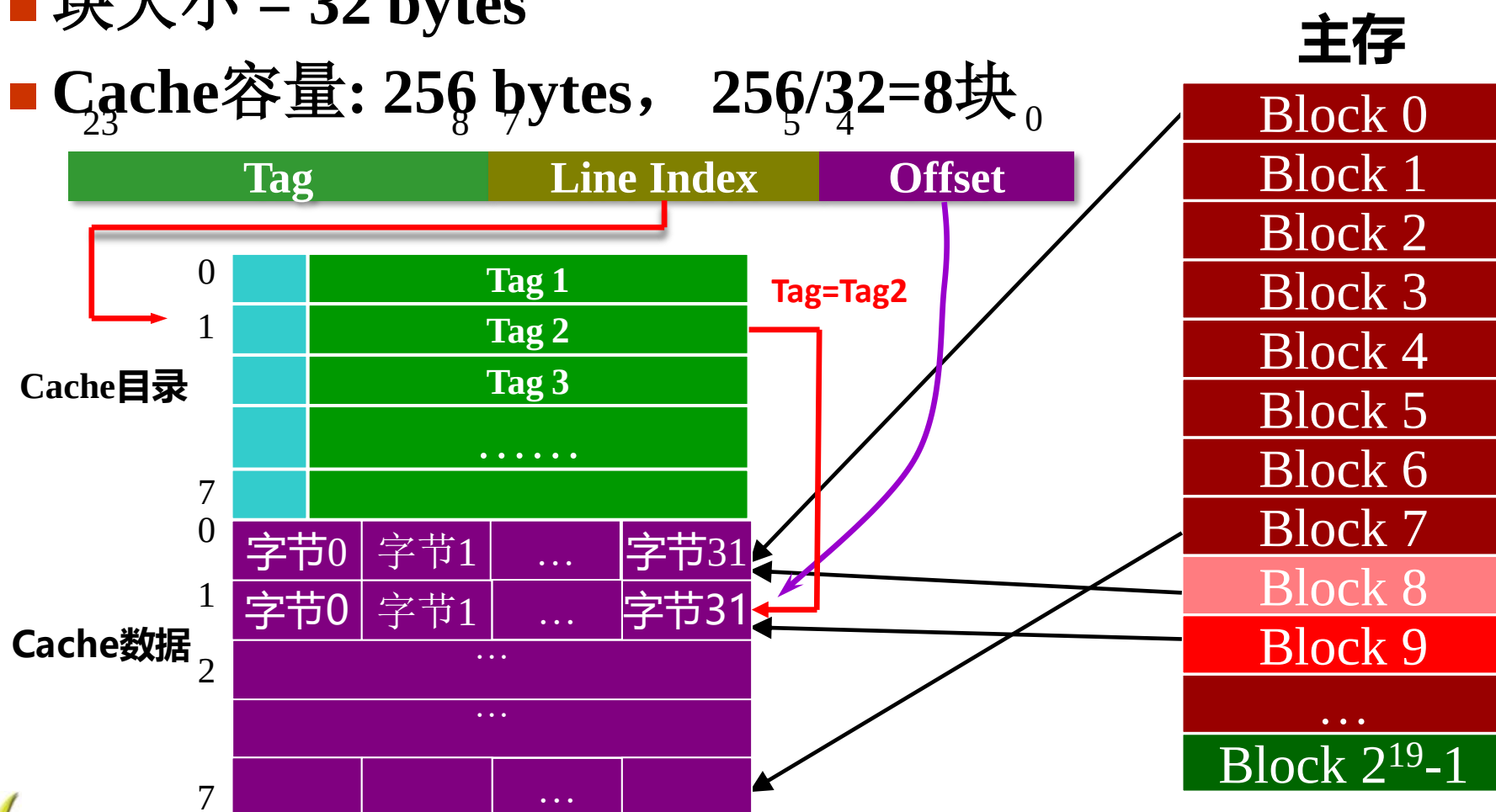
直接映射Cache的检索过程





直接映射Cache示例（1）

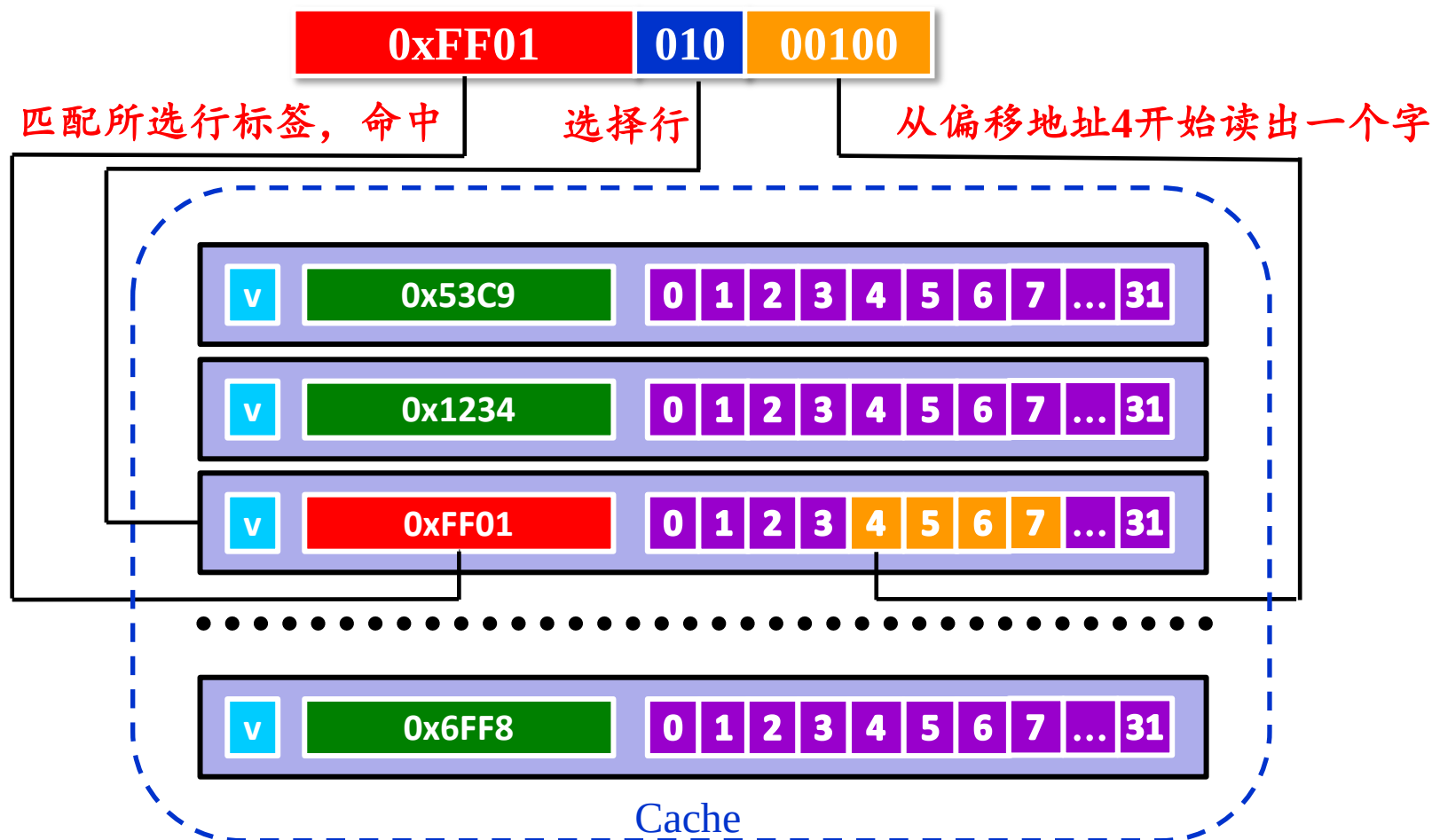
- 主存地址宽度：24位；容量 2^{24} bytes； $2^{24}/2^5=2^{19}$ 块
- 块大小 = 32 bytes
- Cache容量：256 bytes， $256/32=8$ 块





直接映射Cache示例（2）

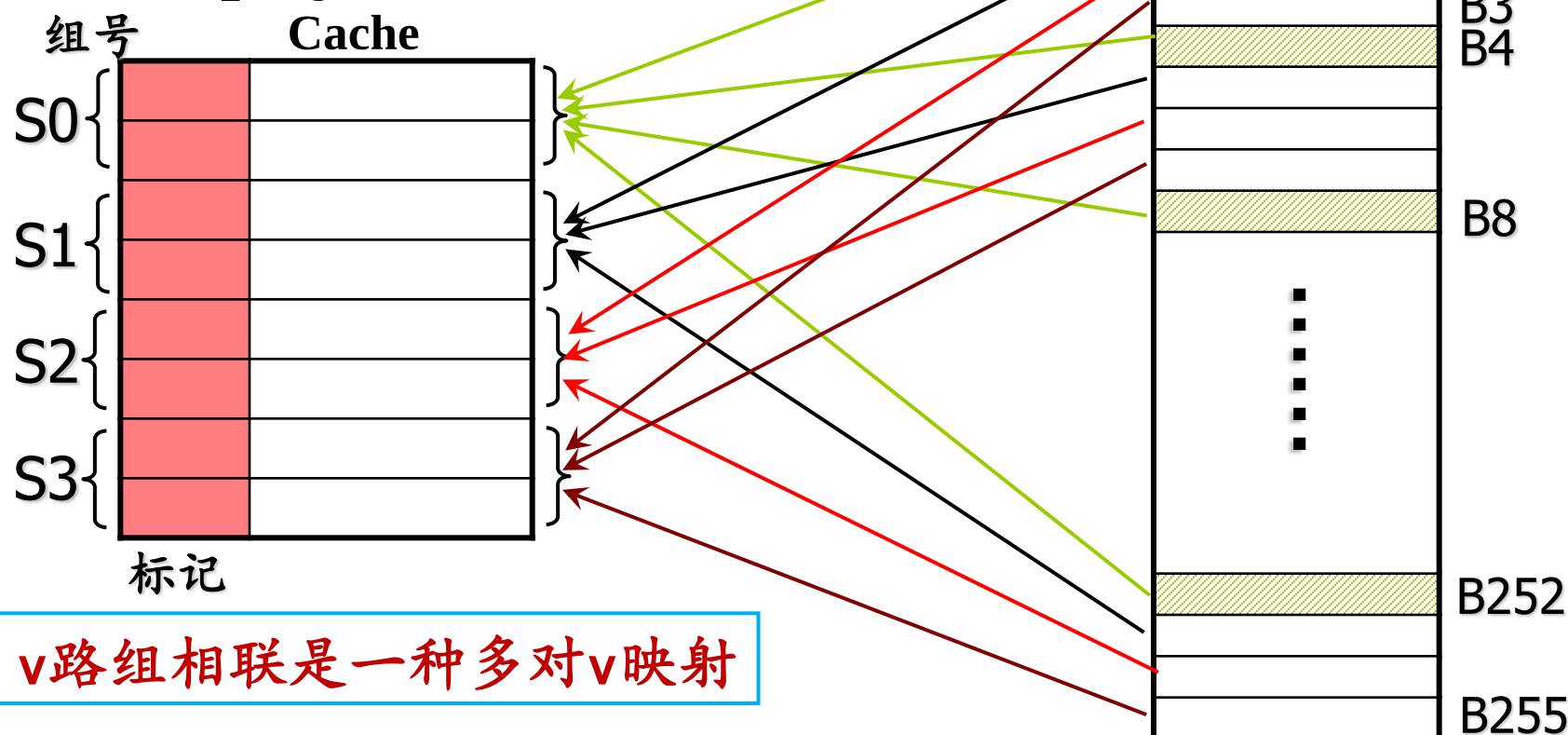
- 假设CPU欲从0xFF0144地址读出一个字（32位）





3、v路（v-way）组相联映射

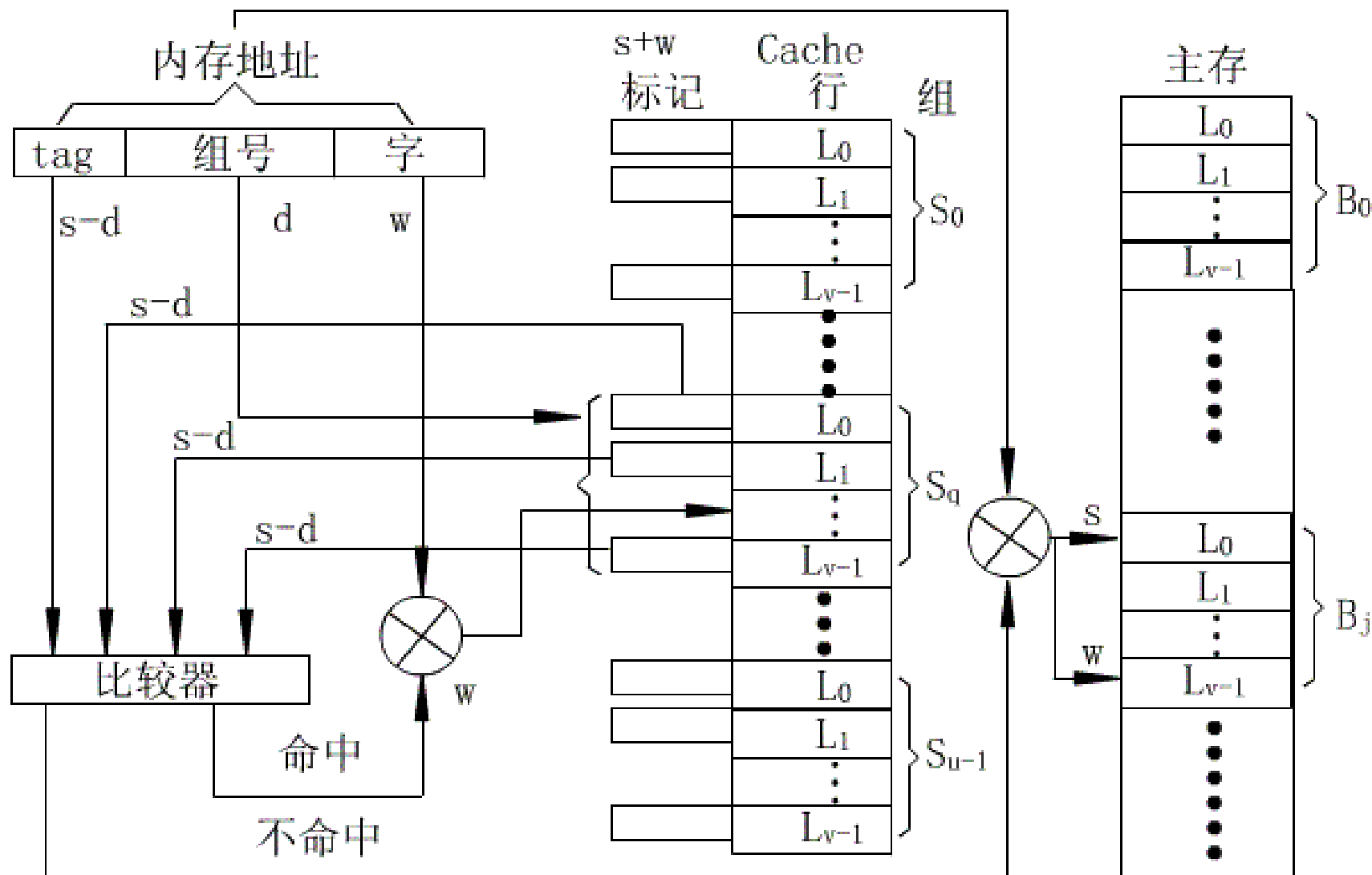
- 组相联映射方式是前两者的折衷方案，被普遍采用
- 将Cache分成u组，每组v行，主存块j存放到哪个组是固定的，放在组内的哪一行是任意的。
- 组号： $q = j \bmod u$ ；对应多个块



v路组相联是一种多对v映射



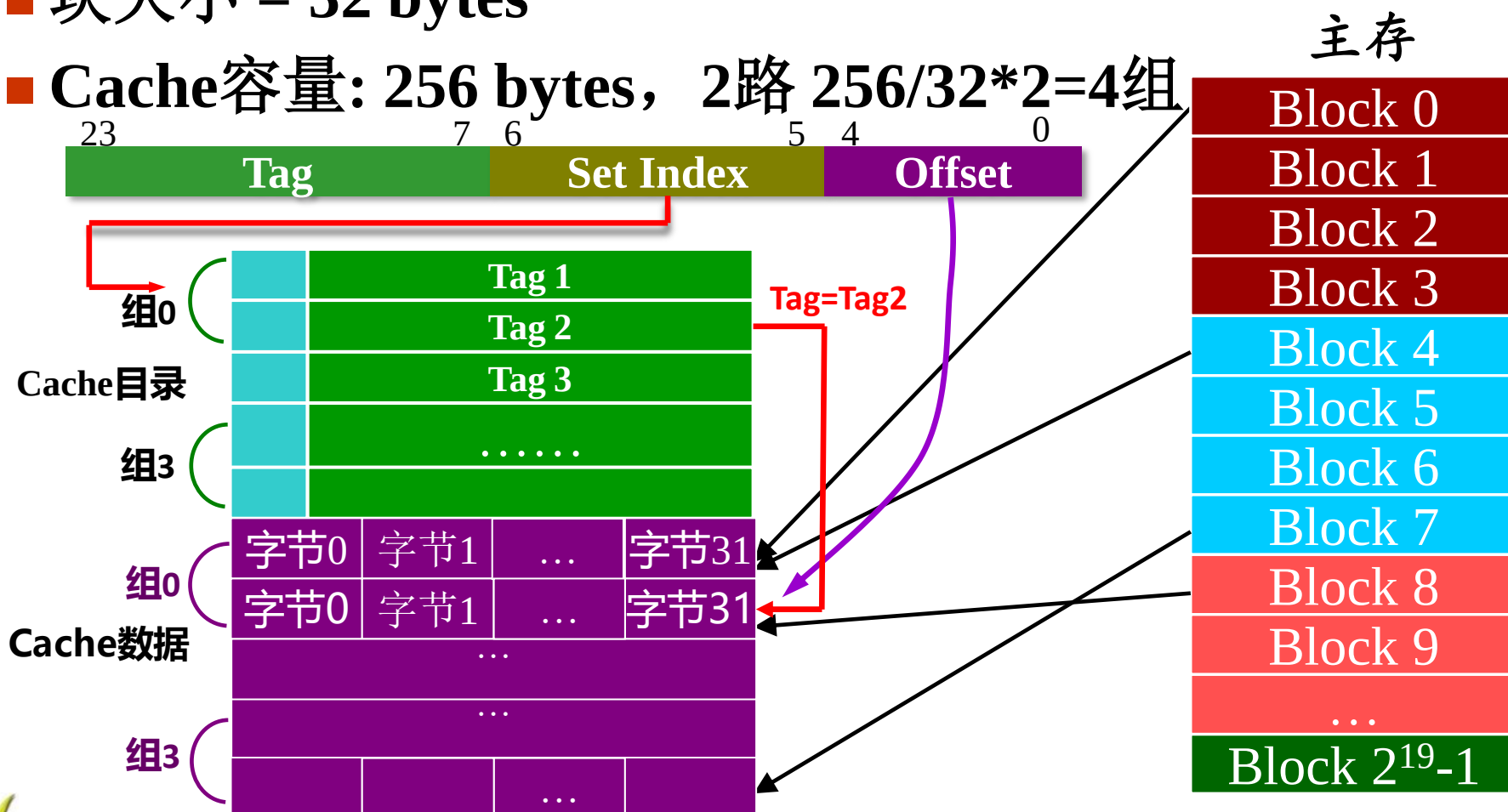
组相联映射的检索过程





2路组相联映射Cache示例（1）

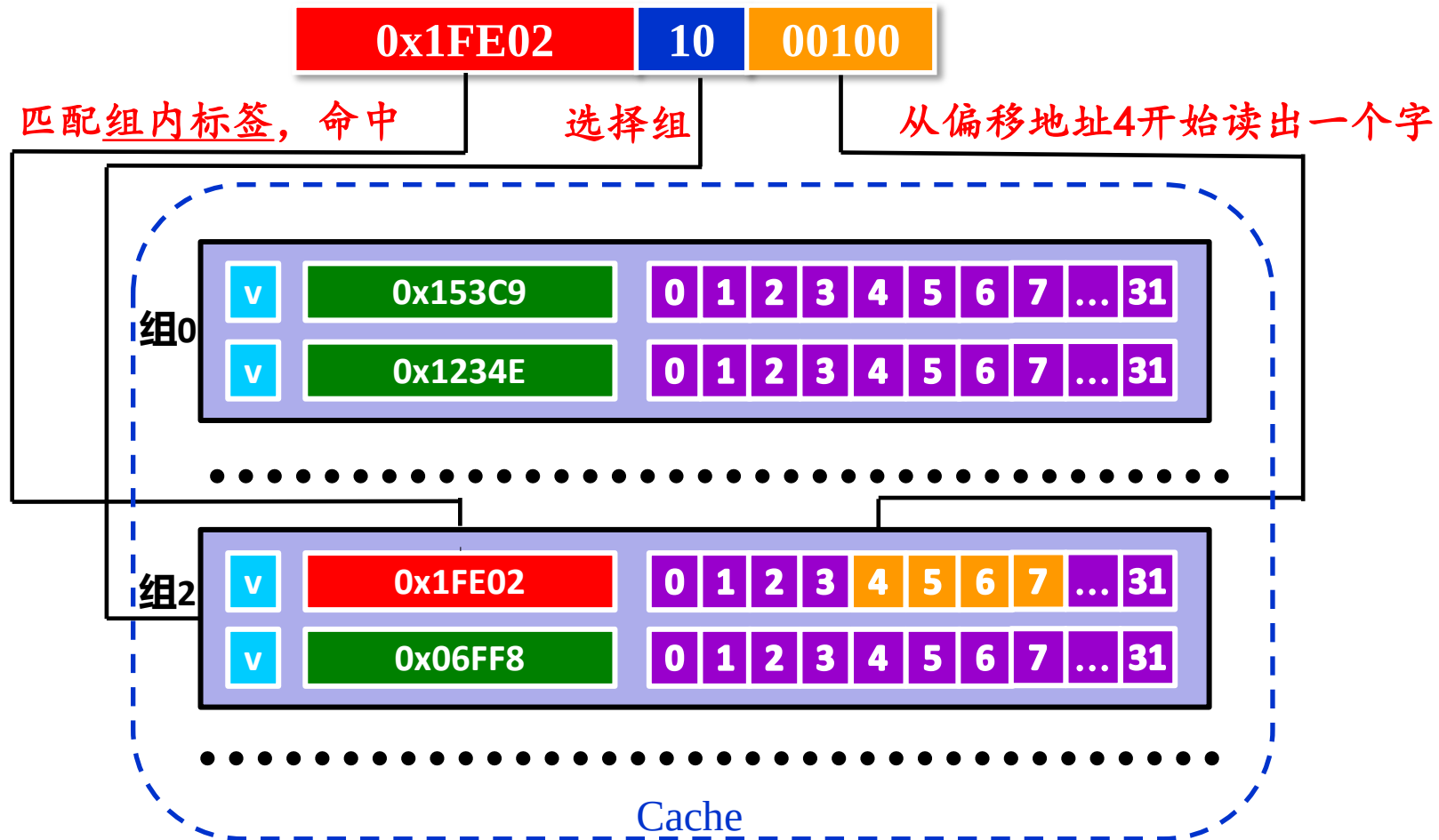
- 主存地址宽度：24位；容量 2^{24} bytes； $2^{24}/2^5=2^{19}$ 块
- 块大小 = 32 bytes
- Cache容量：256 bytes，2路 $256/32*2=4$ 组





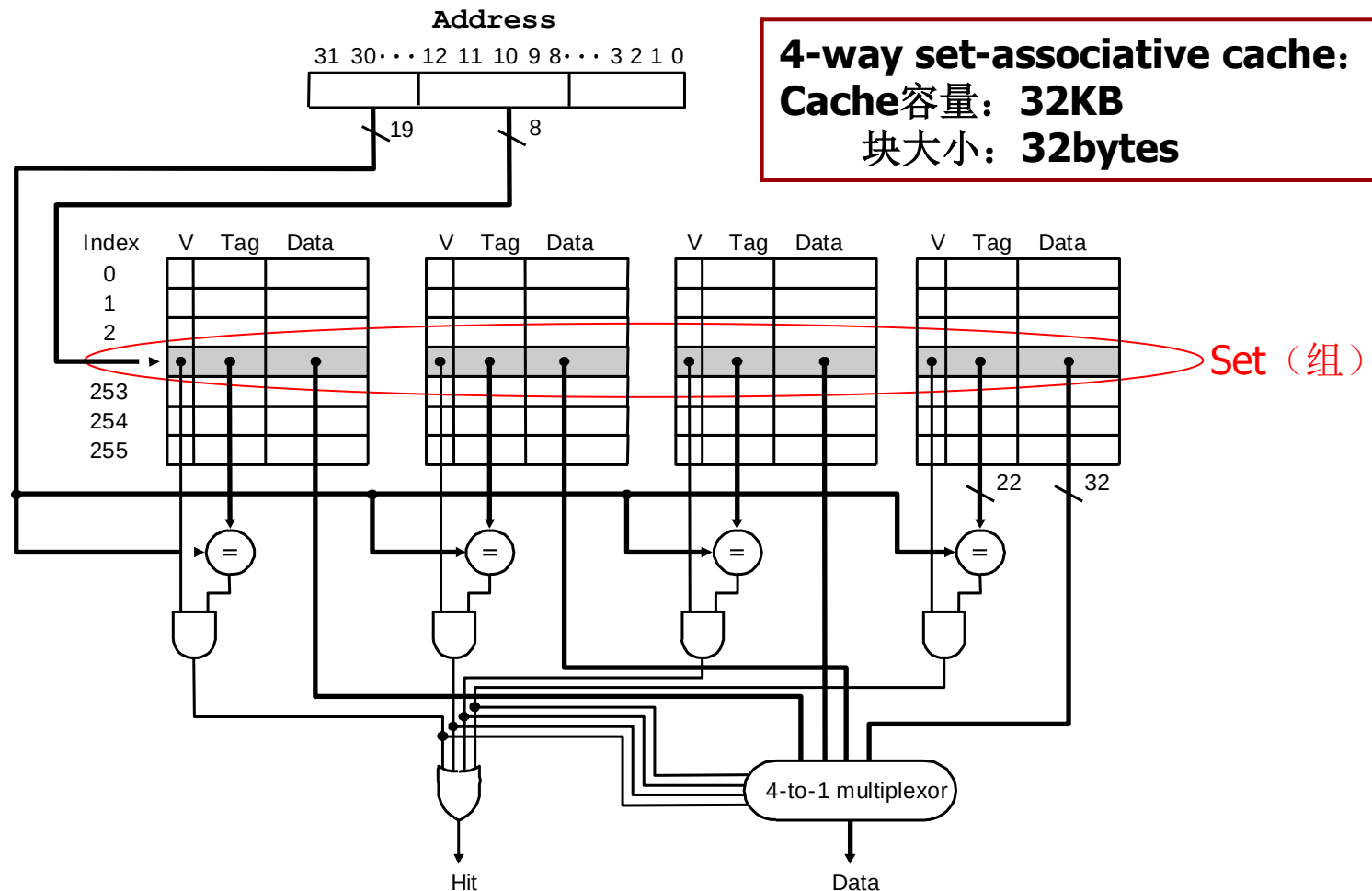
2路组相联映射Cache示例（2）

- 假设CPU欲从0xFF0144地址读出一个字（32位）





4路组相联映射的一个实现



4-way set-associative cache with 4 comparators and one 4-to-1 multiplexor





3种映射方式比较

■ 直接映射Cache:

- ◆ 每个存储器位置只能映射到唯一一个cache位置
- ◆ 在定位时，不需要进行任何判断
- ◆ 当前的数据块替换掉Cache中该位置上的前一个数据块

■ v路组相联Cache:

- ◆ 每个存储器位置可以选择v个cache位置之一，需要CAM

■ 全相联Cache:

- ◆ 每个存储器位置可以存放在cache任一位置，需要CAM





Cache冲突缺失和容量缺失的处理

- 从低一层存储器中取出新的数据块
- 若要给新块腾出空间，需要替换出某一个数据块，所以需制定策略确定替换出哪一个数据块

Cache块替换策略

- 随机替换法（RAND）
 - ◆ 硬件随机选择一个Cache数据块替换出去
- 先进先出（FIFO: First-In First-Out）
 - ◆ 把最先调入 cache数据块作为被替换对象
- 最少使用（LFU: Least Frequently Used）
 - ◆ 将最近一段时间内被访问次数最少的数据块换出。需为每个Cache行设置访问计数器
- 最久未使用（LRU: Least Recently Used）
 - ◆ 将近期内最长时间未被访问过的数据块换出。需要为每个Cache行设置计时器。一旦被访问，计时器就清零





例

- 一个4路组相联的Cache共有64块，主存共有8192块，每块32个字节。则主存地址的标记、组号和块内地址各是多少位？
- 块内地址：5位
- 组号：4位，因为Cache分为 $64/4=16$ 组
- 标记：9位， $13+5 - (4+5) = 9$





动手做

某计算机的Cache共有16块，采用2路组相联映射方式（即每组2块）。每个主存块大小为32字节，按字节编址。主存129号单元所在主存块应装入到的Cache组号是

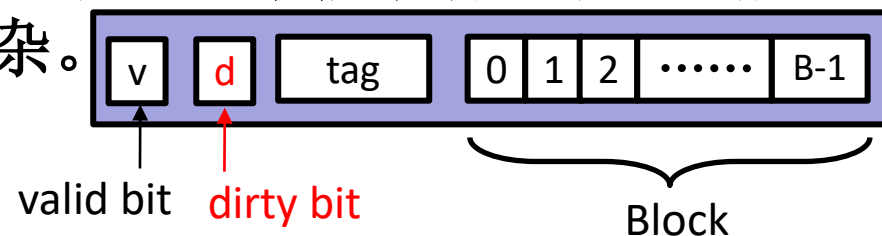
- A. 0
- B. 2
- C. 4
- D. 6





Cache写操作处理（1）

- 数据的多份拷贝存在于
 - ◆ L1、L2、L3、主存、磁盘
- cache写操作处理需要保证cache和存储器中**数据的一致性**
- 写命中（write hit）的情况下
 - ◆ 写穿透（Write Through）：写操作同时更新cache和低一层存储器。显然，每次写都会占用总线带宽
 - ◆ 写回（Write Back）：只写入Cache。当出现cache缺失需要将该cache数据块替换出时，才将该Cache块的数据写回到存储器中。在实现上需要增加一个修改标志位d。能显著减少总线流量，实现复杂。





Cache写操作处理（2）

■ 写缺失（write miss）的情况下

◆ 写分配（write-allocate）

加载相应的低一层中的块到cache，然后更新。试图利用写操作的局部性原理。但缺点是每次不命中都会导致一个块的传送

◆ 非写分配（non-write-allocate）

直接写到低一层中

■ 典型搭配

◆ Write-through + No-write-allocate

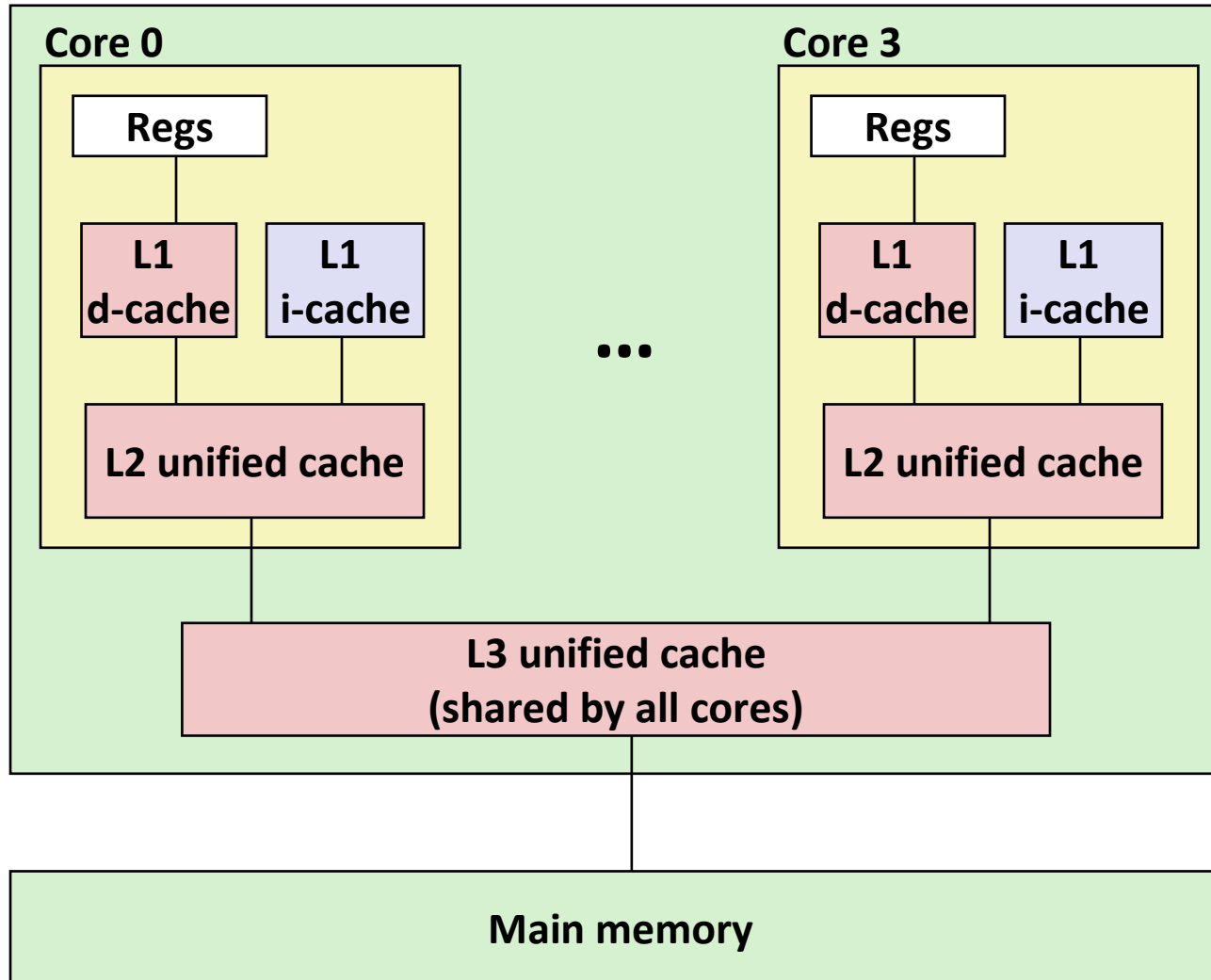
◆ Write-back + Write-allocate





Intel Core i7 Cache Hierarchy

Processor package



L1 i-cache and d-cache:

32 KB, 8-way,
Access: 4 cycles

L2 unified cache:

256 KB, 8-way,
Access: 10 cycles

L3 unified cache:

8 MB, 16-way,
Access: 40-75 cycles

Block size: 64 bytes for
all caches.





例: Core i7 L1 Data Cache

32 kB 8-way set associative

64 bytes/block

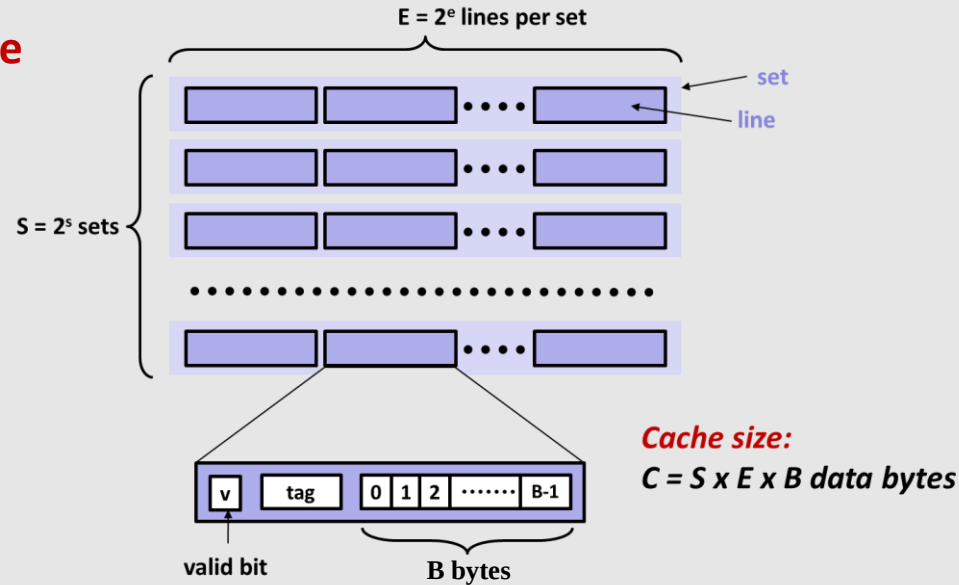
47 bit address range

$B =$

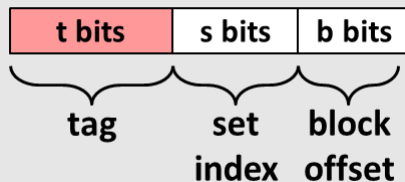
$S =$, $s =$

$E =$, $e =$

$C =$



Address of word:



Block offset: ? bits

Set index: ? bits

Tag: ? bits

Stack Address:

0x00007f7262a1e010

Block offset:

0x??

Set index:

0x??

Tag:

0x??





例: Core i7 L1 Data Cache

32 kB 8-way set associative

64 bytes/block

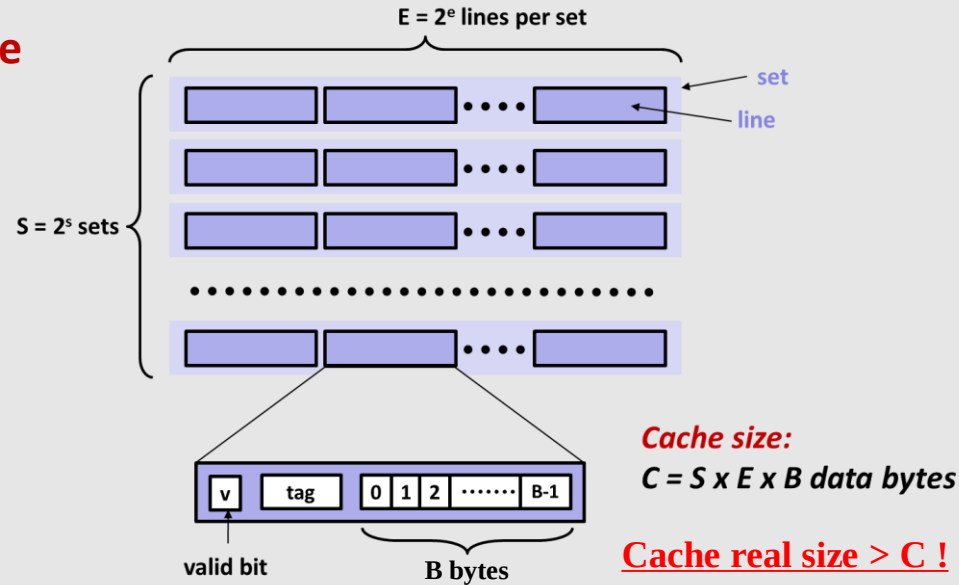
47 bit address range

$B = 64, b = 6$

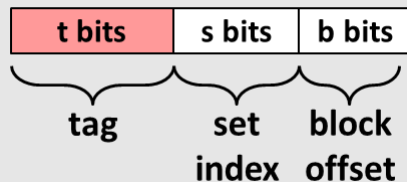
$S = 64, s = 6$

$E = 8, e = 3$

$C = 64 \times 64 \times 8 = 32,768$



Address of word:



Block offset: 6 bits

Set index: 6 bits

Tag: 35 bits

Stack Address:

0x00007F7262A1E010

0000 0001 0000

Block offset: **0x10**

Set index: **0x0**

Tag: **0x7f7262A1E**





Cache性能指标 (1)

■ 命中率 (Hit Rate) :

- ◆ 在一个程序执行期间, 设 N_c 表示在Cache中完成存取的总次数, N_m 表示在主存中完成存取的总次数, 则**命中率h**为:

$$h = N_c / (N_c + N_m)$$

- ◆ 若 t_c 表示命中时Cache访问时间, t_m 表示未命中时的主存访问时间, 则Cache/主存系统的**平均访问时间 t_a** 为:

$$t_a = ht_c + (1 - h) t_m \quad \text{显然, } t_a \text{ 越接近 } t_c \text{ 越好}$$

- ◆ 设 $r = t_m/t_c$ 表示主存慢于cache的倍率, e 表示**访问效率**, 则有

$$e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)r} = \frac{1}{r + (1-r)h}$$





Cache性能指标 (2)

- 命中时间 (Hit Time)：从cache传送一个字到CPU所需的时间
 - ◆ SRAM访问时间+ 确定命中/失效的时间 (组选择、行确认和字选择的时间)
 - ◆ 典型数据
 - 4 clock cycle for L1
 - 10 clock cycles for L2
- 缺失 (Miss)：需要从低一层的块中找回数据
 - ◆ 缺失率 (Miss Rate) = $1 - \text{命中率}$
 - L1 cache缺失率一般为 3-10%，L2 cache缺失率就更小
 - ◆ 缺失损失 (Miss Penalty)：加载低一层中的数据块到当前层的时间+ 将数据交付给处理器的时间。也称为缺失处罚
 - 从主存加载的典型时间是50-200时钟





“缺失率” 指标更真实

- 假设一个有cache的系统
 - ◆ Cache命中时间：1个时钟周期
 - ◆ 缺失处罚：100个时钟周期
- 命中率从97%提高到99%，性能改善多少？
- 平均访问时间：
 - ◆ 97%命中： $1 + 100 \times 0.03 = 4$ 时钟周期
 - ◆ 99%命中： $1 + 100 \times 0.01 = 2$ 时钟周期

2倍提升！





例4

CPU执行一段程序时，Cache完成存取的次数为1900次，主存完成存取的次数为100次，已知Cache存取周期为50ns，主存存取周期为250ns，求Cache/主存系统的效率和平均访问时间。

解：

$$h = \frac{N_c}{N_c + N_m} = \frac{1900}{1900 + 100} = 0.95$$

$$r = \frac{t_m}{t_c} = \frac{250ns}{50ns} = 5$$

访问效率：

$$e = \frac{1}{r + (1-r)h} = \frac{1}{5 + (1-5) \times 0.95} = 83.3\%$$

平均访问时间：

$$t_a = \frac{t_c}{e} = \frac{50ns}{0.833} = 60ns$$





动手做

假设某计算机的存储系统由**Cache**和主存组成，某程序执行过程中访存1000次，其中访问**Cache**缺失（未命中）50次，则**Cache**的命中率是

- A. 5%
- B. 9.5%
- C. 50%
- D. 95%





动手做

采用指令Cache与数据Cache分离的主要目的是

- A 减低Cache的缺失损失
- B 提高Cache的命中率
- C 减低CPU平均访问时间
- D 减少指令流水线资源冲突





例 8 (1)

假设处理器基本的CPI为1.0，所有访问在一级Cache中均命中，时钟频率为5GHz。假定主存访问时间为100ns，其中包括缺失处理时间。设一级cache中**每条**指令缺失率为2%。如果增加一个二级cache，命中或缺失访问的时间都是5ns，而且容量大到使访问主存的缺失率减为0.5%，问处理器速率提高了多少？

解：访问主存的缺失损失折算为主频的时钟周期数为：

$$100\text{ns} \div 0.2\text{ns} = 500 \text{ 个时钟周期}$$

仅有一级cache的处理器器的实际CPI为：

$$\begin{aligned}\text{CPI} &= \text{基本CPI} + \text{每条指令中存储器停顿的时钟周期} \\ &= 1.0 + 2\% \times 500 \\ &= 11\end{aligned}$$





例 8 (2)

增加二级cache后，访问二级cache的缺失损失折算为主频的时钟周期数为：

$$5\text{ns} \div 0.2\text{ns} = 25 \text{ 个时钟周期}$$

有一、二级cache的处理器的实际CPI为：

$$\begin{aligned}\text{CPI} &= \text{基本CPI} + \text{每条指令的一级停顿} + \text{每条指令的二级停顿} \\ &= 1 + 2\% \times 25 + 0.5\% \times 500 \\ &= 4\end{aligned}$$

性能提升：

$$11 \div 4 = 2.75 \text{ 倍}$$





3.7 虚拟存储器





虚存的引入

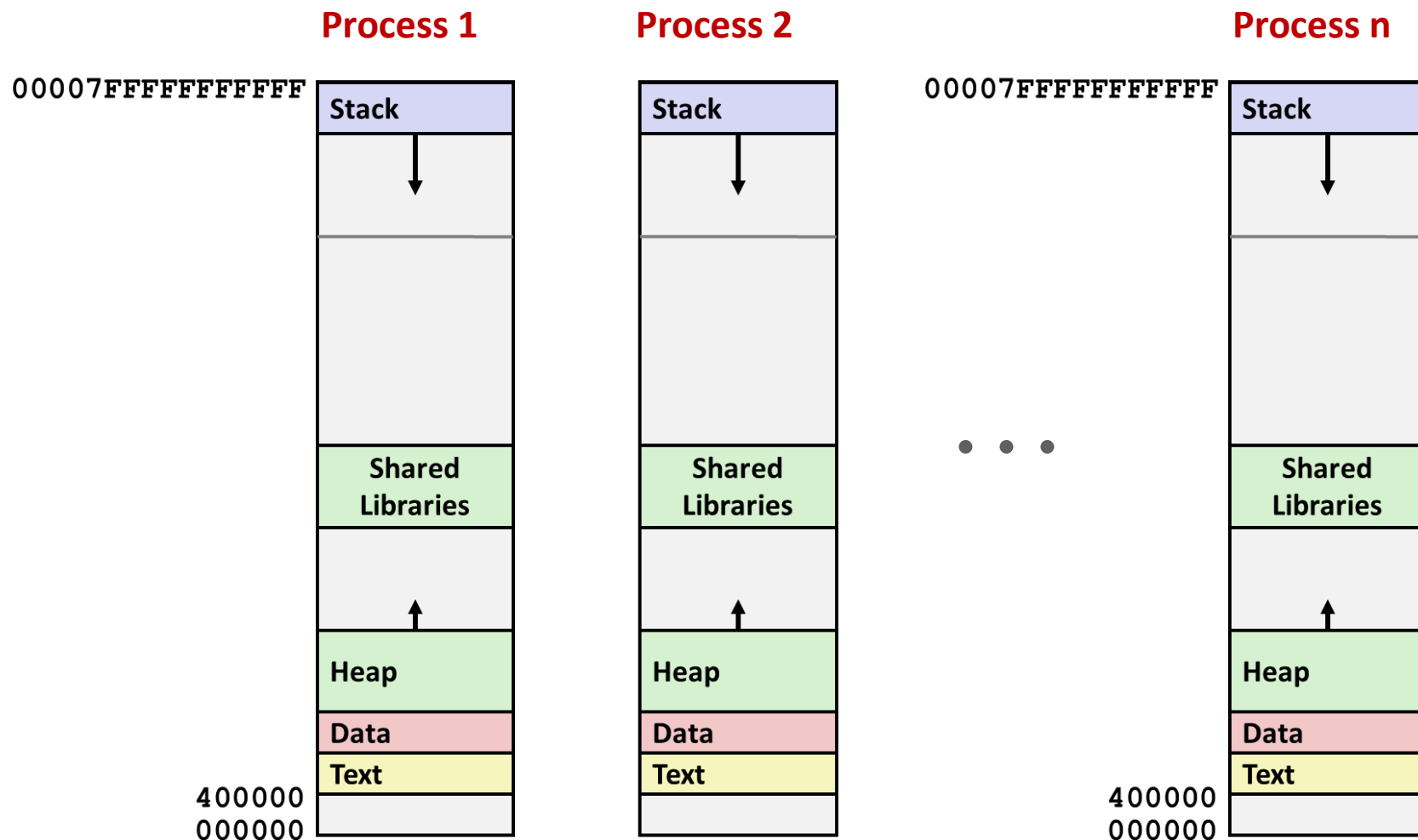
- 程序所需的存储器容量可能大于计算机系统实际配备的主存储器容量
- 在编写程序时希望**独立编址**
 - ◆ 无需考虑程序是否能在物理存储器中存放得下
 - ◆ 无需考虑程序应该存放在什么物理位置
- 在程序运行时分配给每个程序一定的运行空间
 - ◆ 由地址转换部件（硬件或软件）将编写程序时使用的地址（逻辑地址）转换成实际内存的地址（物理地址）

目的：给程序员一个比实际内存大得多的编址空间





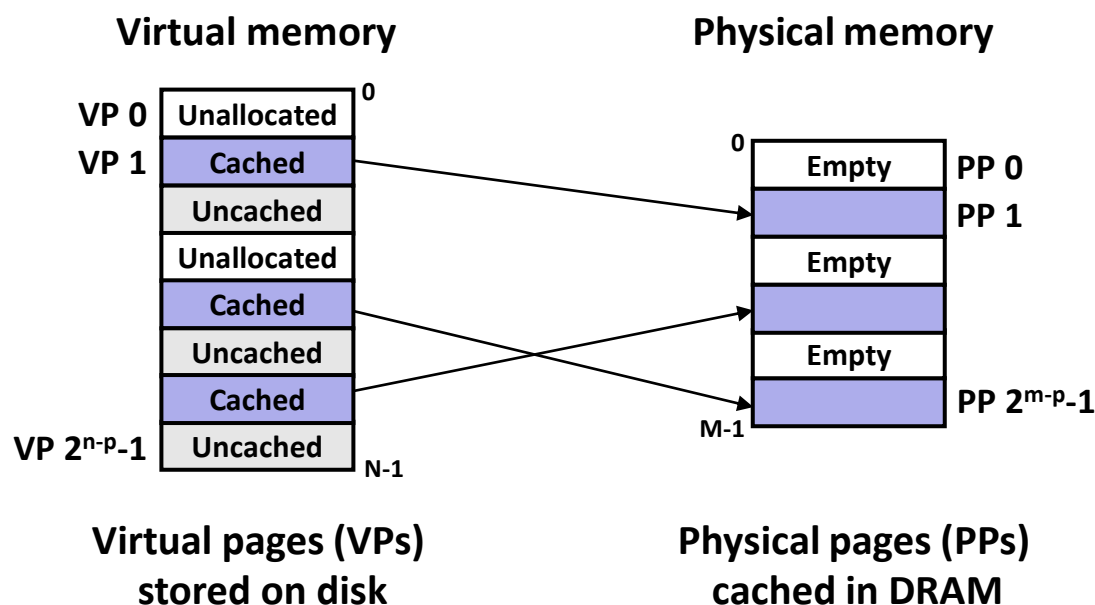
虚存能为每个进程提供统一的地址空间





基本概念 (1)

- 虚拟存储器只是一个容量非常大的存储器的逻辑模型，以透明的方式为用户提供了一个比实际主存空间大得多的程序地址空间
- 虚拟存储器被组织为一个由存放在磁盘上的N个连续字节大小的单元组成的数组（N为虚存的大小）





基本概念（2）

- 虚拟存储器指的是主存—外存层次
- 用户编写程序时使用的地址称为**虚拟地址**（Virtual Address）或逻辑地址（Logical address），其对应的存储空间称为虚存空间或逻辑地址空间
- **物理地址**（Physical Address）或称实地址（Real Address）是由CPU地址引脚送出的、用于访问主存的地址，其对应的存储空间称为物理地址空间或主存空间
 - ◆ 若CPU地址总线的宽度为 m 位，那么物理地址空间的大小用 2^m 来表示
- 程序进行虚拟地址到物理地址的转换过程称为程序再定位





主存—外存层次结构特点

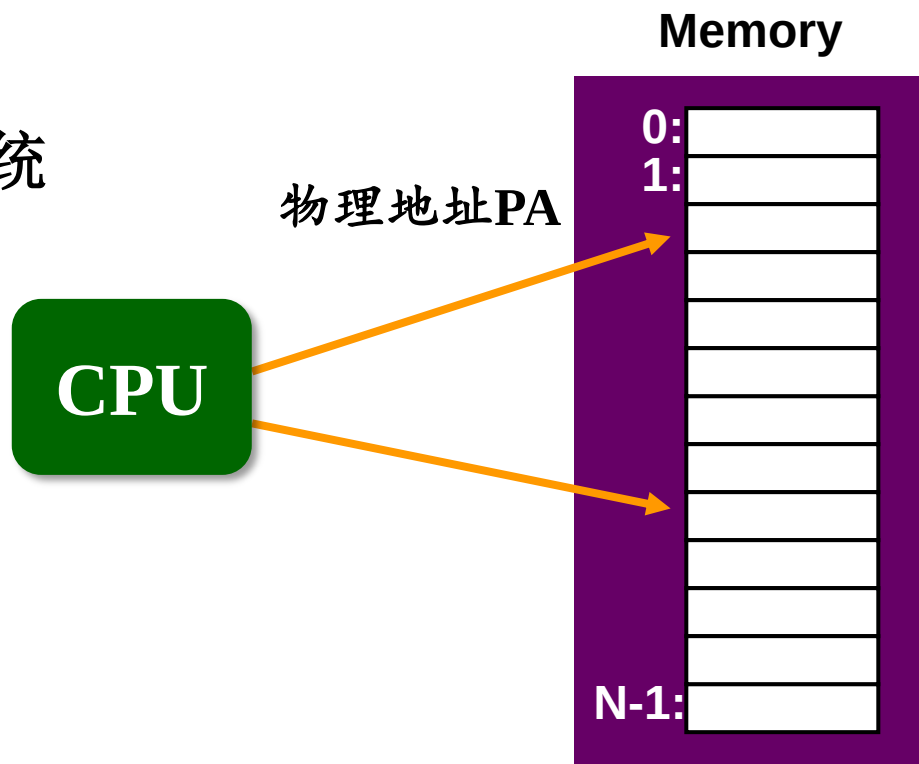
- 与Cache-主存层次有很多相似之处
 - ◆ 地址变换
 - ◆ 替换策略
 - ◆ 基于程序局部性原理
- 主存—外存层次和Cache—主存层次遵循的原则
 - ◆ 把程序中最近常用的部分驻留在高速的存储器中
 - ◆ 一旦这部分变得不常用了，把它们送回到低速的存储器中
 - ◆ 这种换入换出是由硬件或操作系统完成的，对用户是透明的
 - ◆ 力图使存储系统的性能接近高速存储器，价格接近低速存储器
- 常用的3种存储管理方式
 - ◆ 段式存储管理
 - ◆ 页式存储管理
 - ◆ 段页式虚拟存储管理





只有物理存储器的系统

- 处理器产生的地址直接指向物理存储器中的存储单元（如：字节）
- 实例
 - ◆ 早期PC机
 - ◆ 几乎所有的嵌入式系统
 - ◆



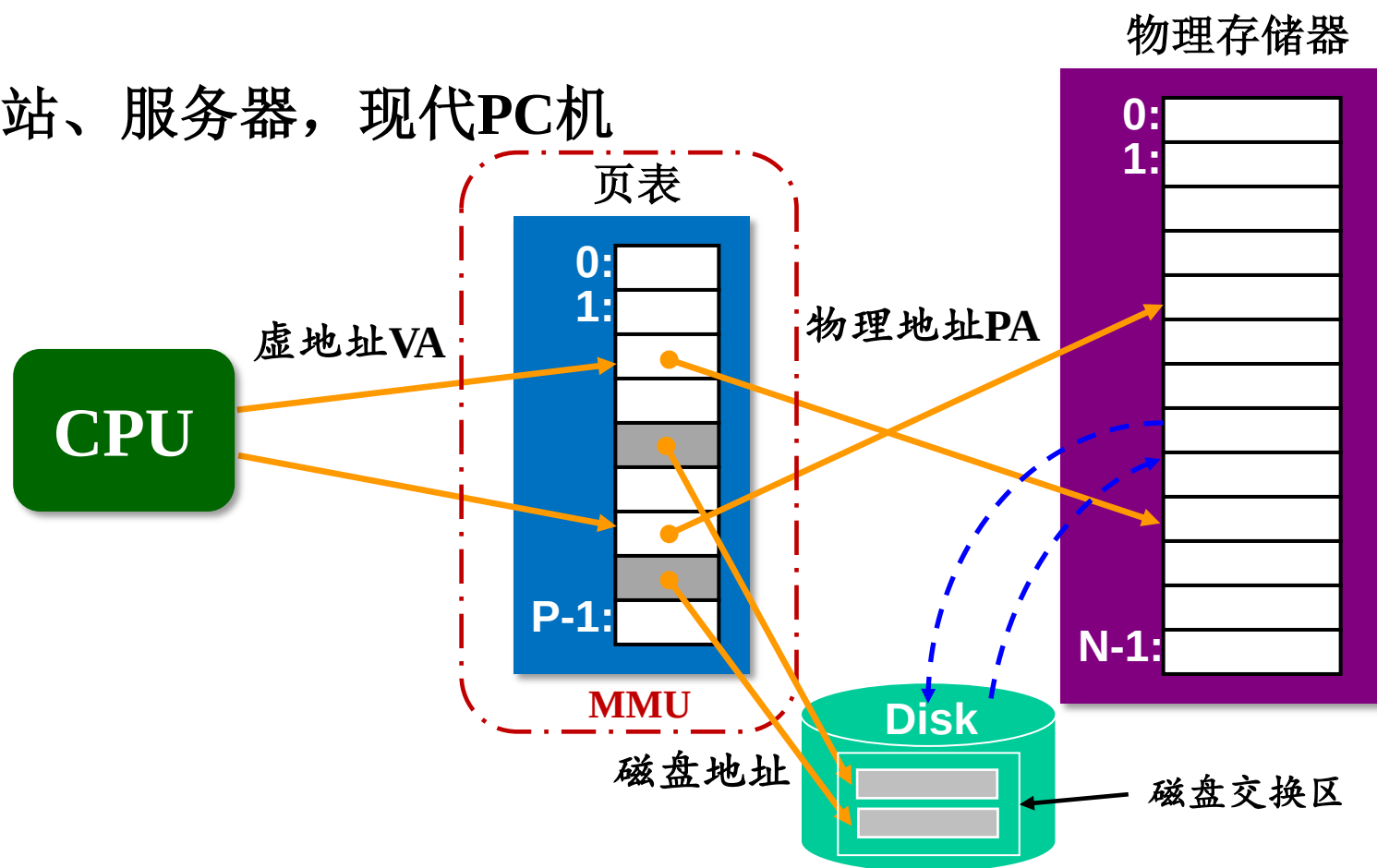
具有虚拟存储器的系统

■ 地址翻译：通过操作系统管理的页表可将虚拟地址转换为物理地址

■ 实例：

◆ 工作站、服务器，现代PC机

◆





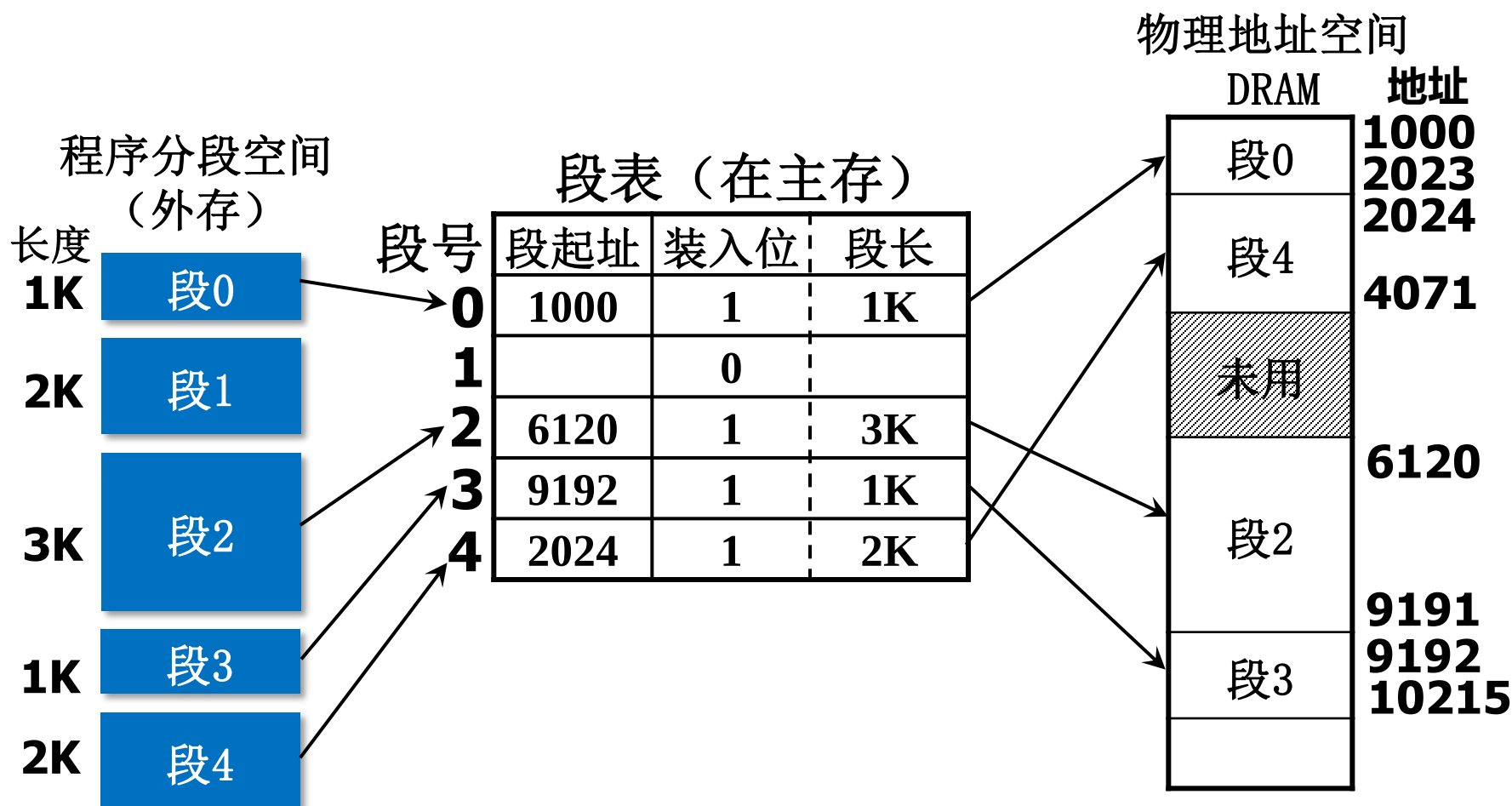
1. 段式存储管理

- 段是利用程序的模块化性质，按照程序的逻辑结构划分成的多个相对独立部分，作为独立的逻辑单位
- 用**段表**来指明各段在主存中的位置，各段都有它的名称、段起点、段长等
- 主存按段进行分配与管理的存储管理方式称之为**段式管理**
- 段式管理有利于按段实现信息共享和内存保护，但易造成主存中出现不好用的碎块，会影响主存储器的利用效率





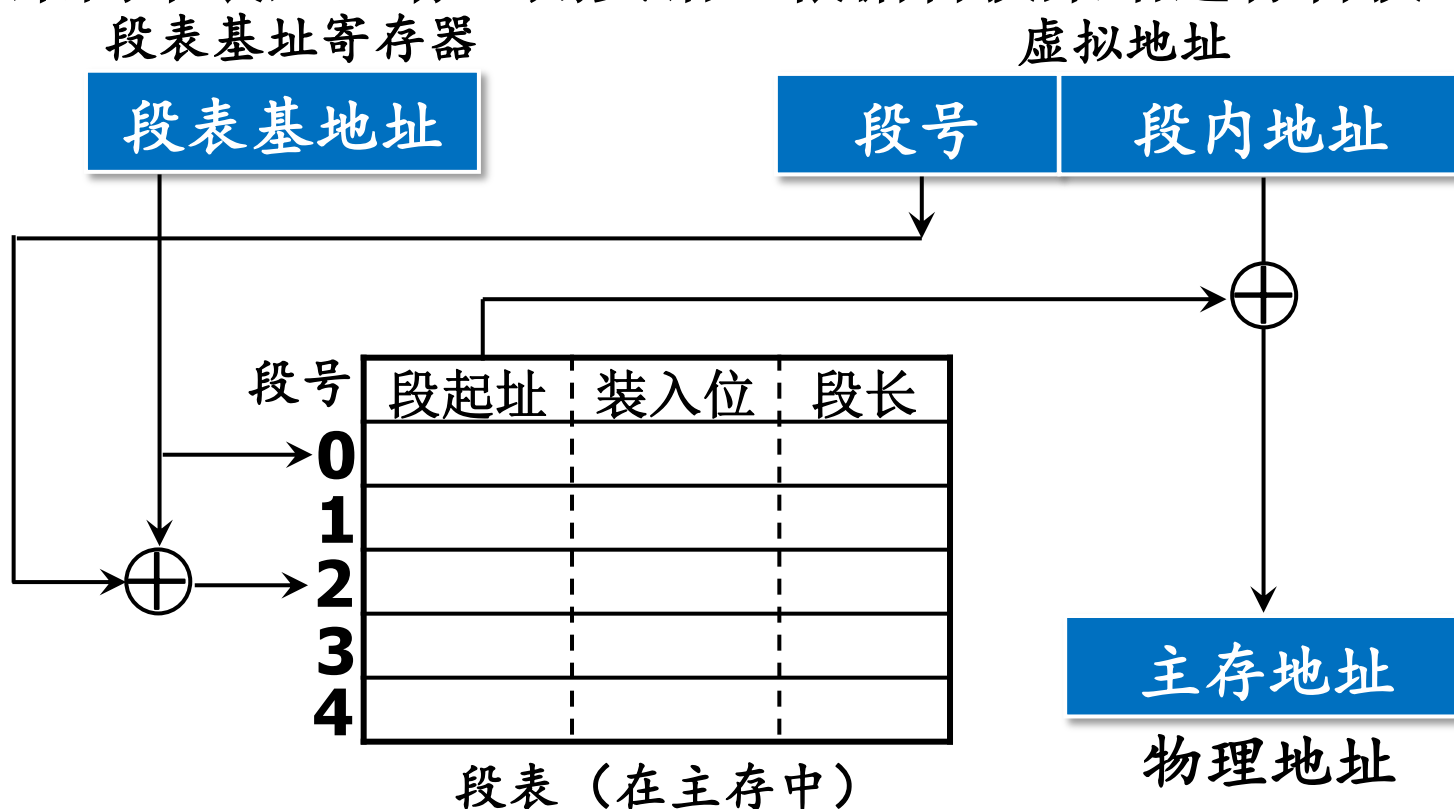
段式虚拟存储器





虚拟/物理地址翻译

- 根据**段表基址寄存器**和段号在段表中找到相应的段记录
- 如果对应的段已装入主存，则称段命中，根据段起址和段内地址得到物理地址
- 如果尚未装入主存，则页错，根据替换策略进行替换





2. 页式虚拟存储器

- 把虚拟空间和主存空间都分成**大小相同的页**（为二的整数幂个字），称为页面，并以页为单位进行虚存与主存间的信息交换。
- 虚拟地址被分为：虚拟页号、页内地址；物理地址被分为：物理页号（**页框号**）、页内地址，尽管它们页号会不同，但使用相同的页内地址。
- 与段式存储管理不同，页不是程序本身的结构特性，而是从管理的角度人为划分的结果。
- 页式管理的缺点是处理、保护和共享都不如段式管理方便。

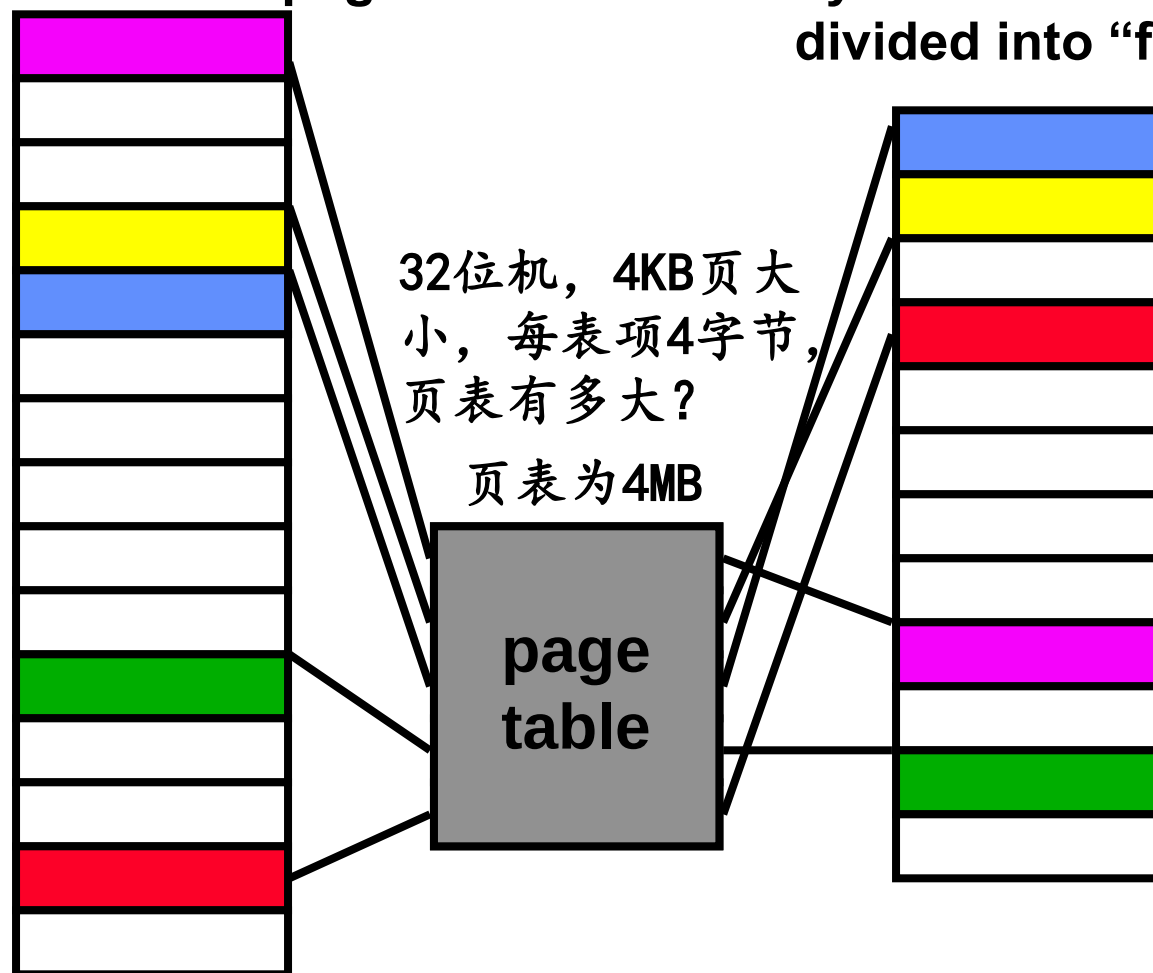




页式存储管理示意图

Logical Address Space
divided into “pages”

Physical Address Space
divided into “frames”



页表大小：页
表中表项的数
量等于虚拟地
址空间中页的
数量



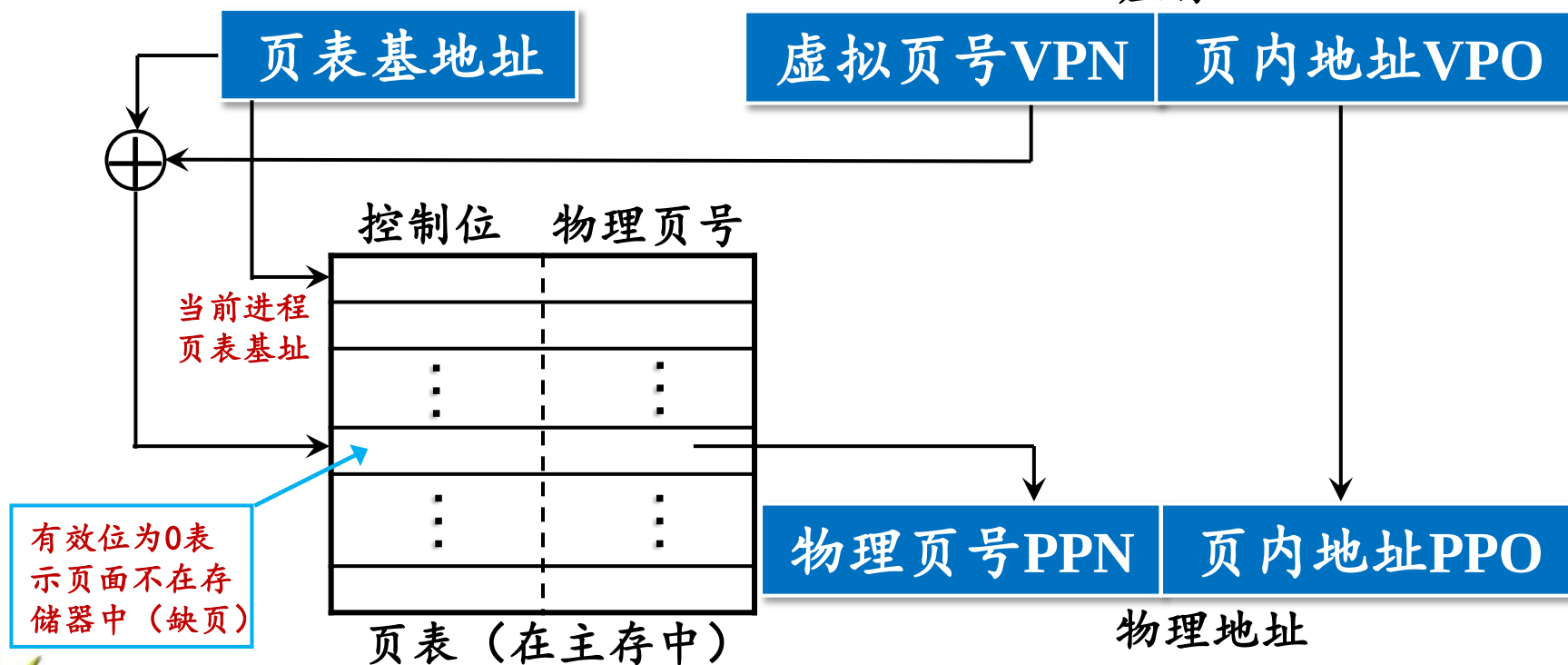


虚拟/物理地址翻译

- 在页表中，每个表项内容至少要包含该逻辑页所在主存的页框号（物理页号）
- 此外，在页表的表项中还包括装入位（有效位）、修改位、替换控制位和保护位等组成的控制字段

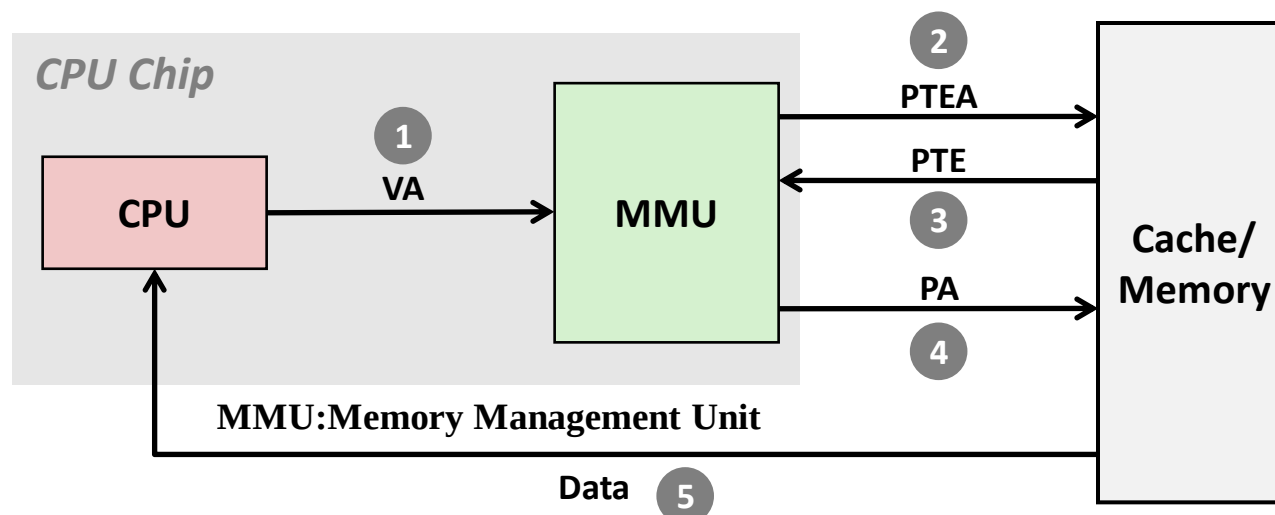
页表基址寄存器（x86中为CR3）

虚存地址





页面命中

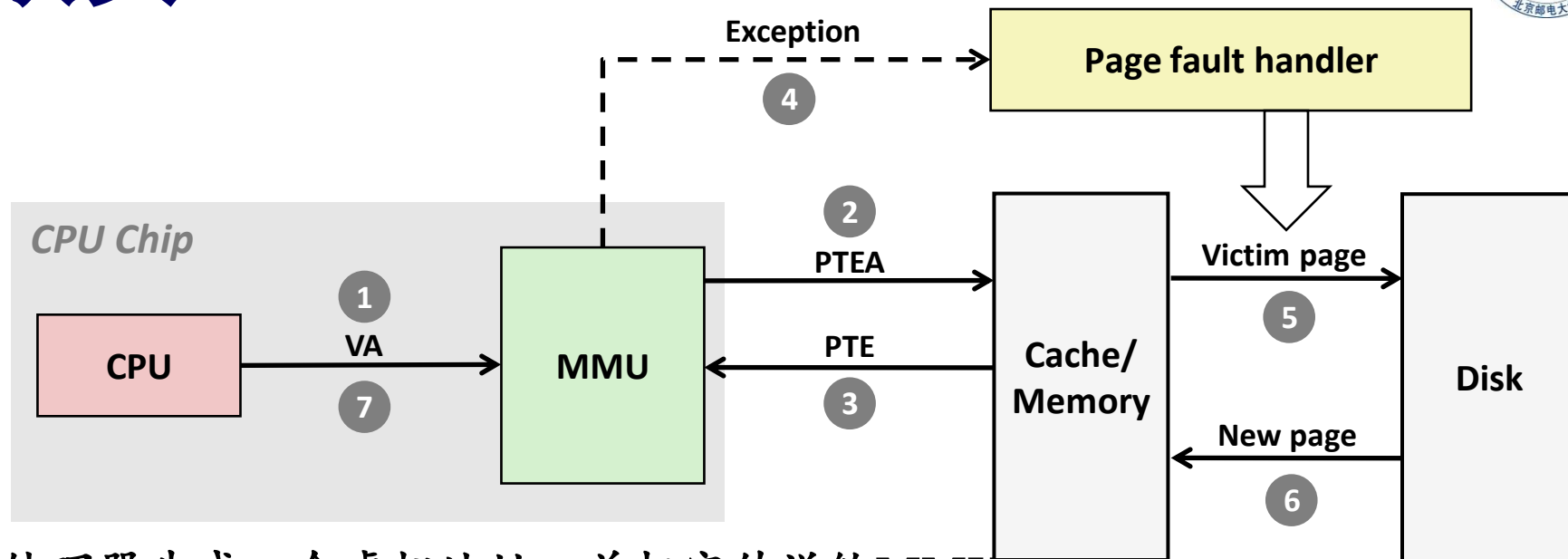


VA: virtual address, PA: physical address, PTE: page table entry

- 1) 处理器生成一个虚拟地址，并把它传送给MMU
- 2) MMU生成页表项（PTE）地址，并从cache/主存请求得到该页表项
- 3) cache/主存向MMU返回PTE
- 4) MMU 构造物理地址，并把它传送给cache/主存
- 5) cache/主存返回所请求的数据字给处理器



缺页



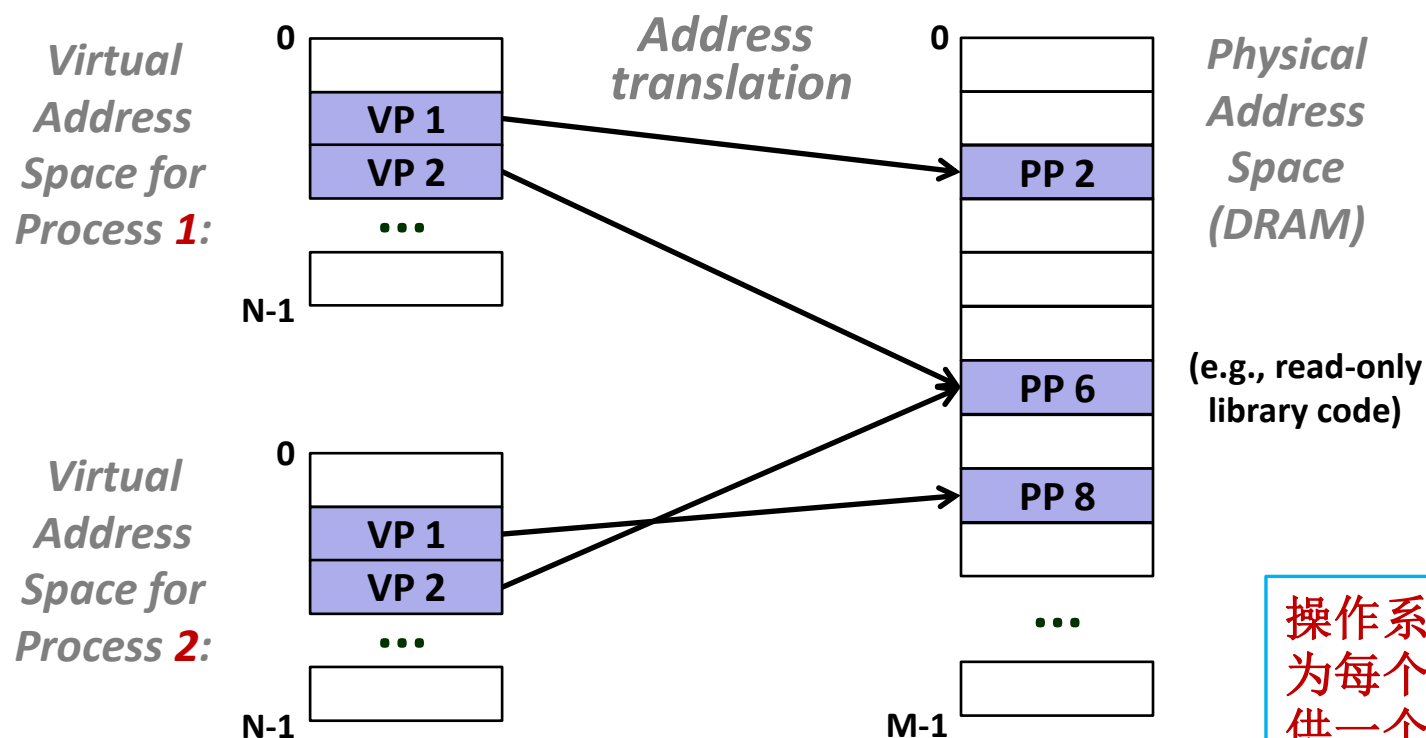
- 1) 处理器生成一个虚拟地址，并把它传送给MMU
- 2) MMU生成PTE地址，并从cache/主存请求得到该页表项
- 3) cache/主存向MMU返回PTE
- 4) PTE中的有效位为0，所以MMU触发了一次异常，传递CPU控制权到操作系统内核中的缺页异常处理程序
- 5) 缺页处理程序确定出物理内存中的牺牲页，如果这个页面已经被修改过，则把它换出到磁盘
- 6) 缺页处理程序调入新的页面，并更新内存中的PTE
- 7) 缺页处理程序返回到原来的进程，**再次执行**导致缺页的**指令**。CPU将引起缺页的虚拟地址重新发给MMU。因为虚拟页面现在已缓存在物理内存中，所以**肯定命中**





内存管理 (1)

- 每个进程都具有相同的虚拟地址空间
 - ◆ 将存储器视为一个线性数组



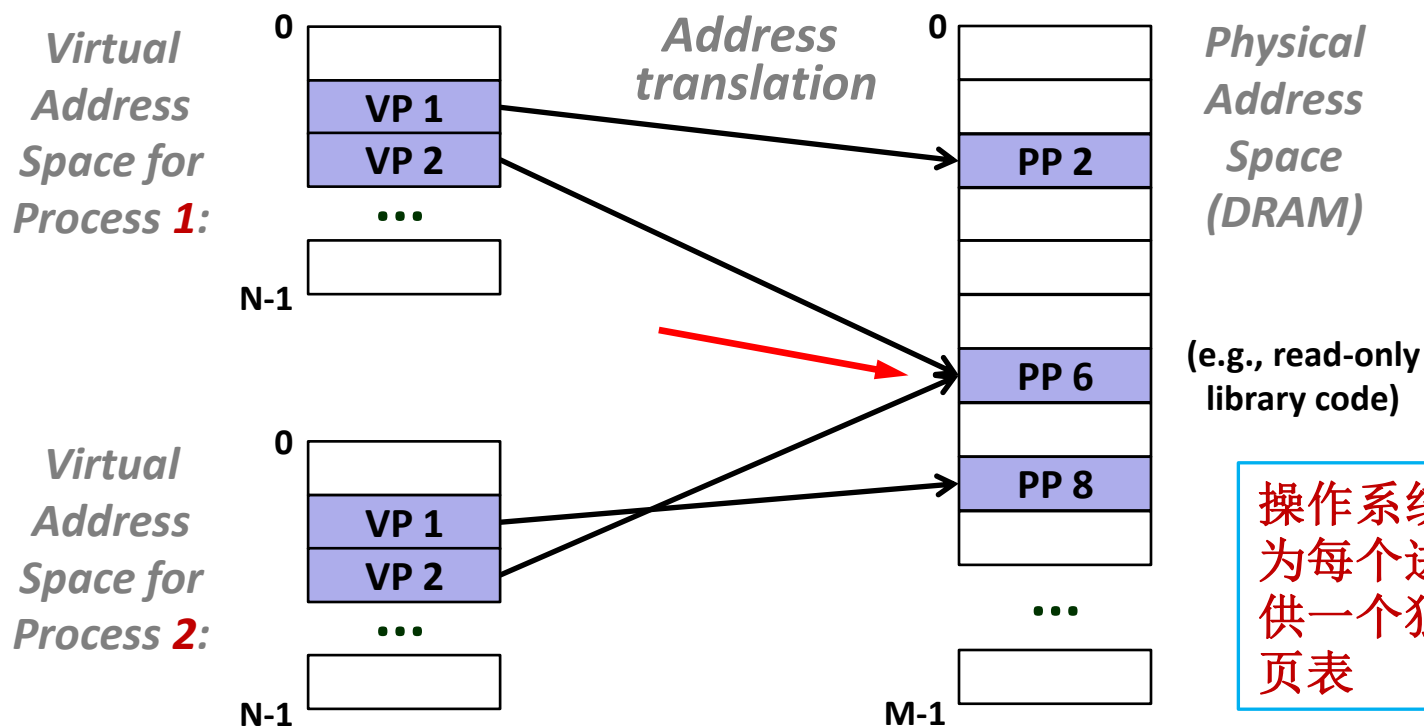
操作系统必须
为每个进程提
供一个独立的
页表





内存管理 (2)

- 简化存储器分配相同的虚拟地址空间
 - ◆ 每个虚拟页可以映射到任意物理页
 - ◆ 同一个虚拟页在不同的时间可以存储在不同的物理页
- 进程间共享代码和数据



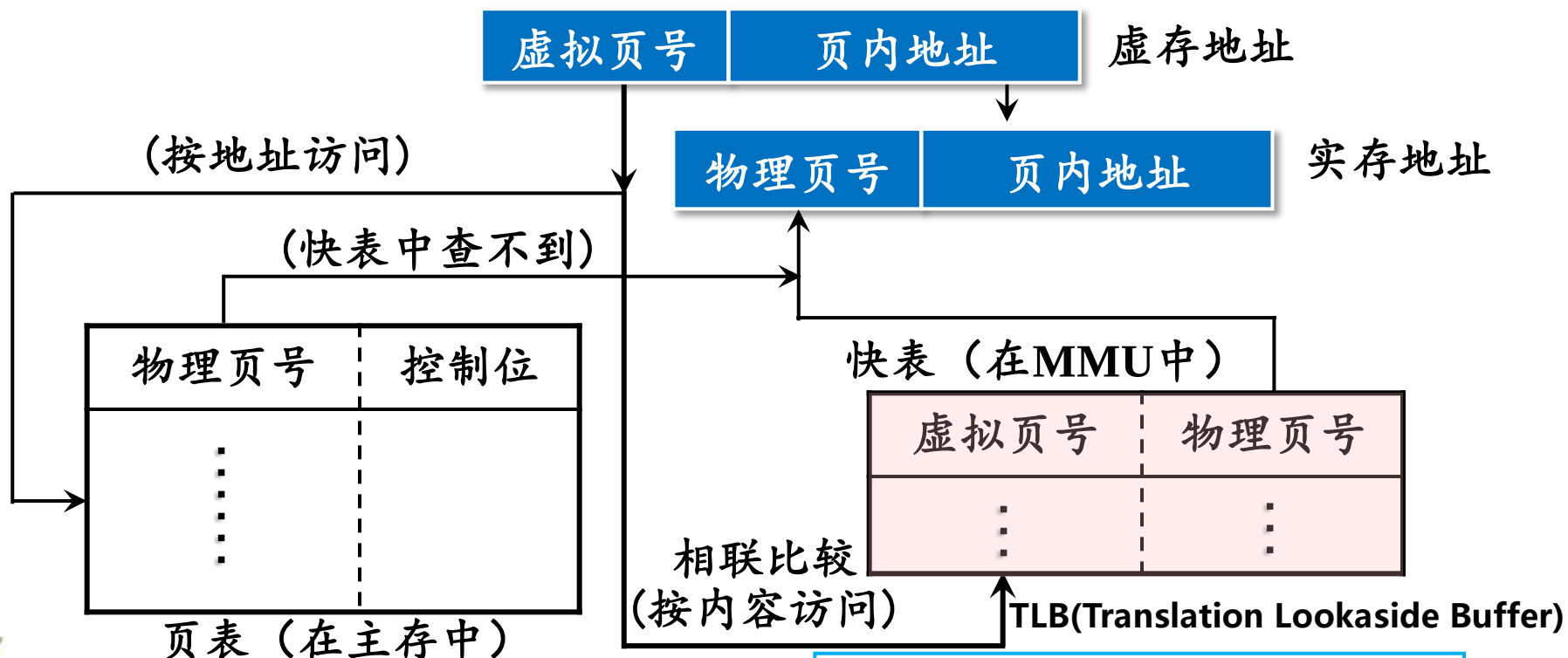
操作系统必须
为每个进程提
供一个独立的
页表





用TLB加速虚拟/物理地址翻译

- 快表TLB是一个特殊的cache，它比页表小很多，是页表的一个更小的副本
- 查表时，用虚拟页号同时去查快表和页表，当在快表中命中时，迅速将对应的物理页号送入实存地址寄存器
- 如果在快表中查不到，那就要花费一个访主存时间查页表，从中查到物理页号送入实存地址寄存器，并用替换算法替换快表中内容



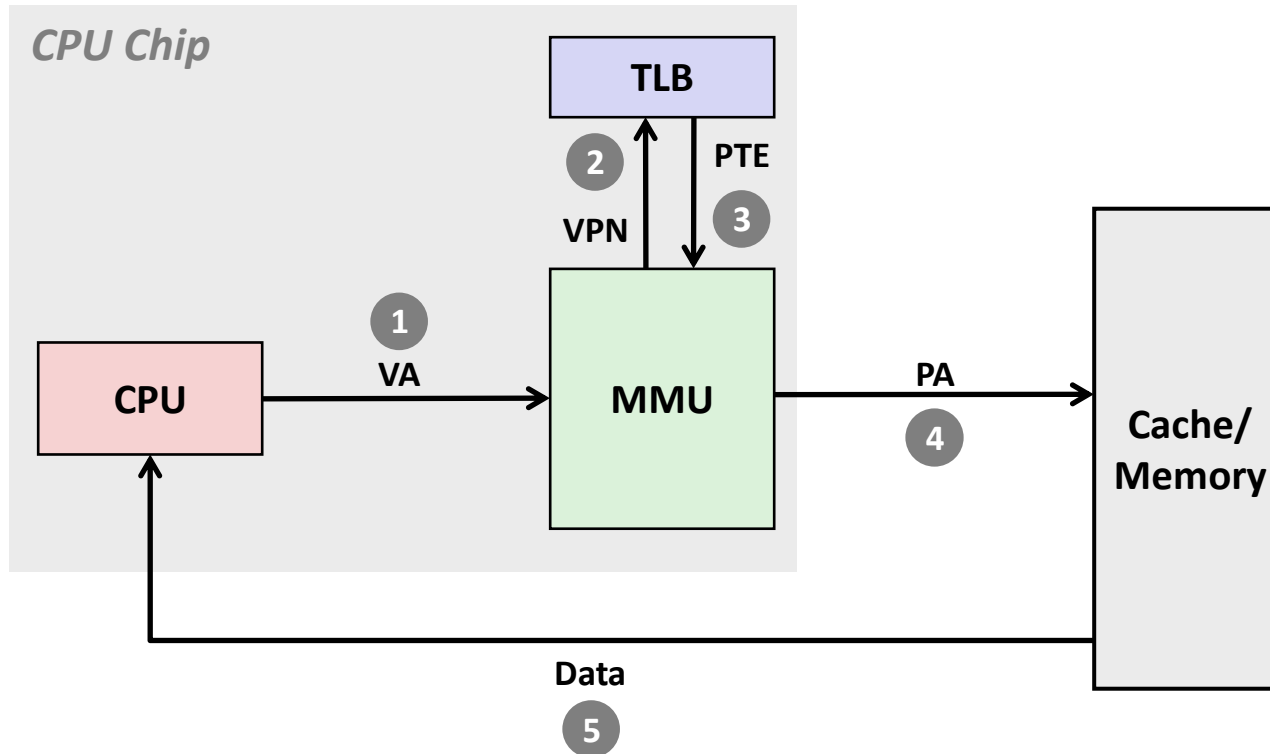
页表项到TLB中有3种映射方式





例

TLB命中



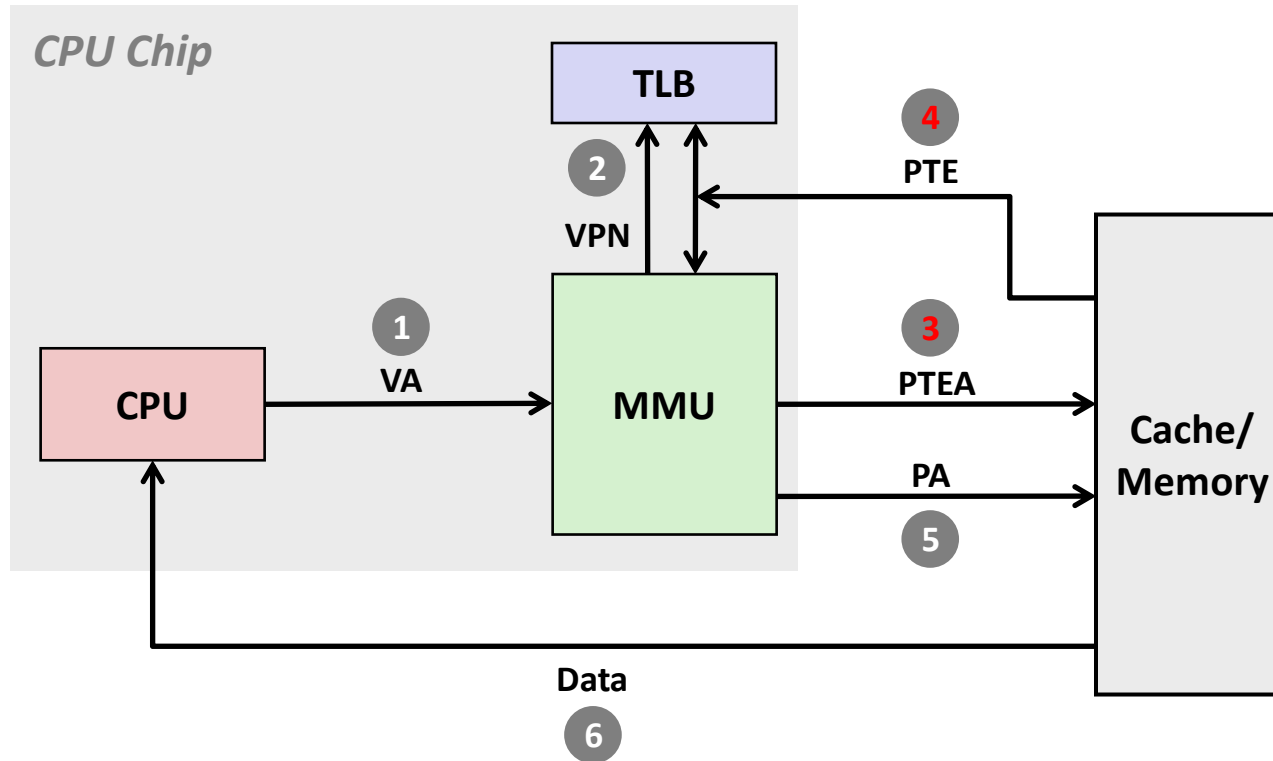
VA: virtual address, PA: physical address, PTE: page table entry





例

TLB未命中



VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address





例1

■ 虚拟地址

	虚拟页号	页内地址
①	015	324
②	007	128
③	848	516

- 1) CPU按虚地址①访问主存时，主存的实地址是多少？
- 2) CPU按虚地址②访问主存时，主存的实地址是多少？
- 3) CPU按虚地址③访问主存时，主存的实地址是多少？

快表TLB（全相联映射）

标记	页框号（物理页号）
032	42
025	38
007	96
006	60
004	40
015	80
005	50
034	70





例2

- 某计算机主存地址空间大小为256MB，按字节编址。虚拟地址空间大小为4GB，采用页式存储管理，页面大小为4KB，TLB（快表）采用全相联映射，有4个页表项，内容如下表所示。

有效位	标记	页框号	...
0	FF180H	0002H	...
1	3FFF1H	0035H	...
0	02FF3H	0351H	...
1	03FFFH	0153H	...

问对虚拟地址**03FF F180H**进行虚实地址变换的结果是多少？

解： $2^{12}=4\text{KB}$ ， $2^{32}=4\text{GB}$ ， $2^{28}=256\text{MB}$

所以，页内地址为12位；虚拟地址：虚拟页号20位 页内地址12位

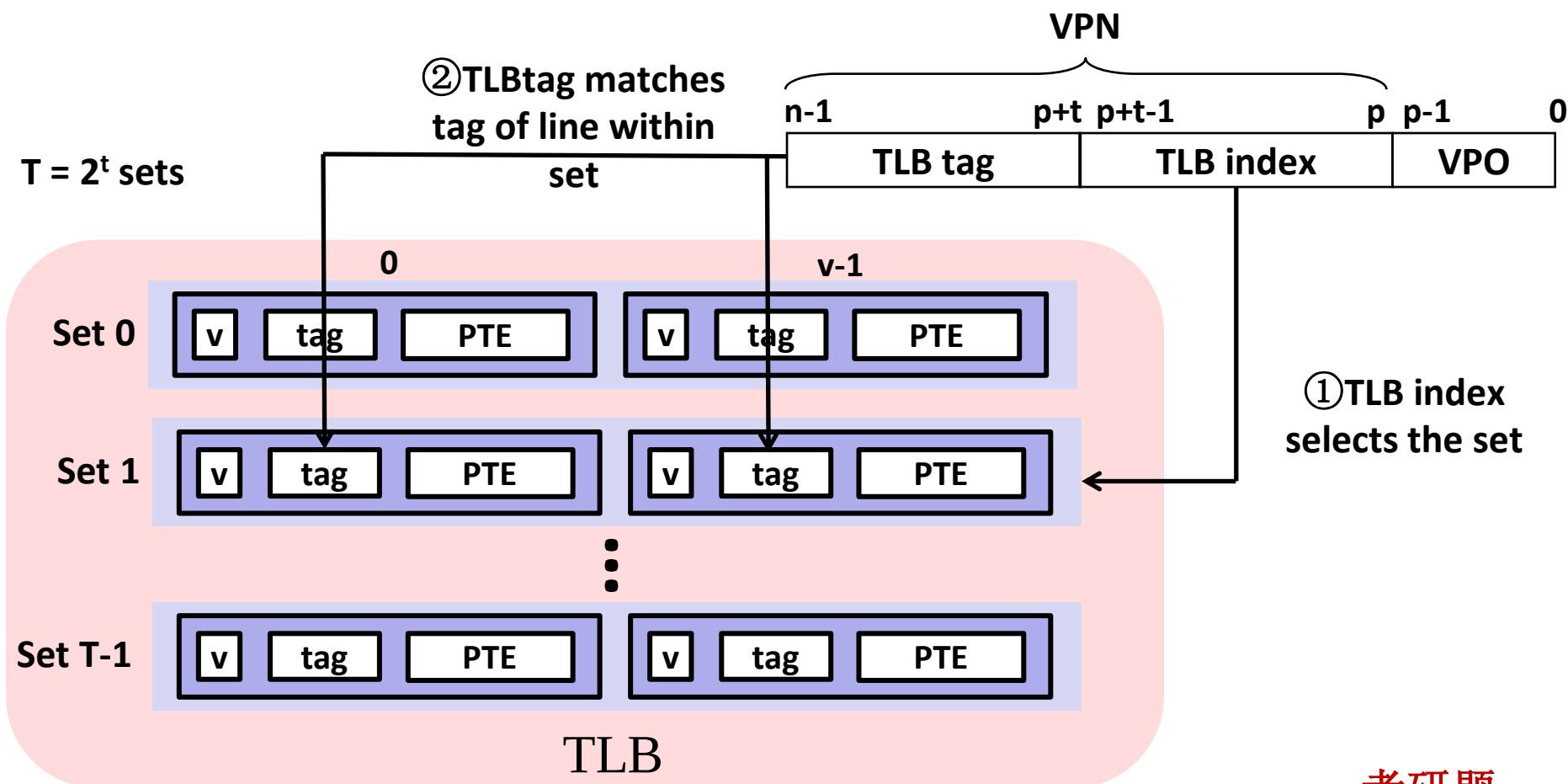
虚拟地址**03FF F180H** → 物理地址:0153 180H





补充 v路组相联TLB的访问

■ MMU用虚拟地址的VPN部分访问TLB



考研题





多级页表

考虑如下系统

■ 4KB页大小，48位虚拟地址空间，每个页表项8字节

■ 页表大小？

◆ $2^{48} \times 2^{-12} \times 2^3 = 2^{39} = \mathbf{512GB}$

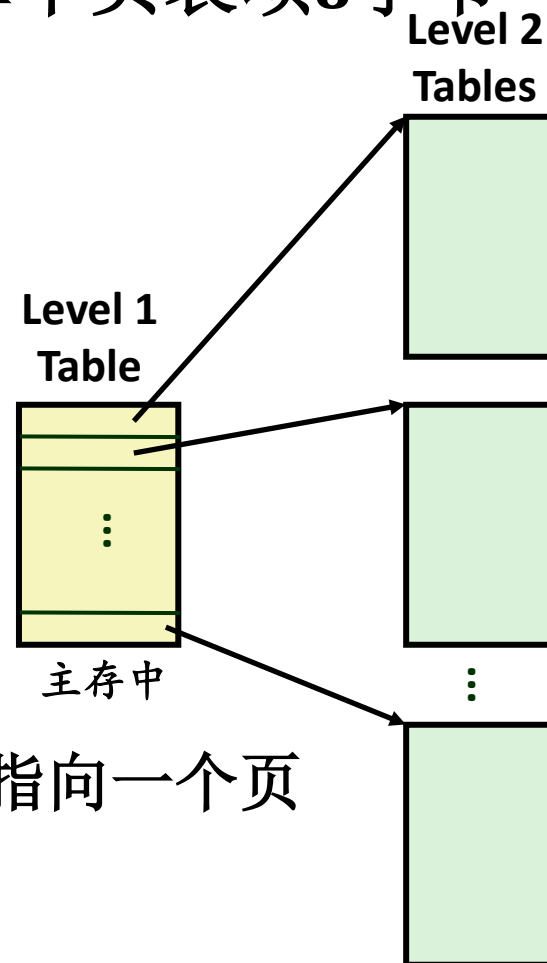
■ 页表必须放在主存中，如何解决？

■ 多级层次结构页表

■ 二级页表

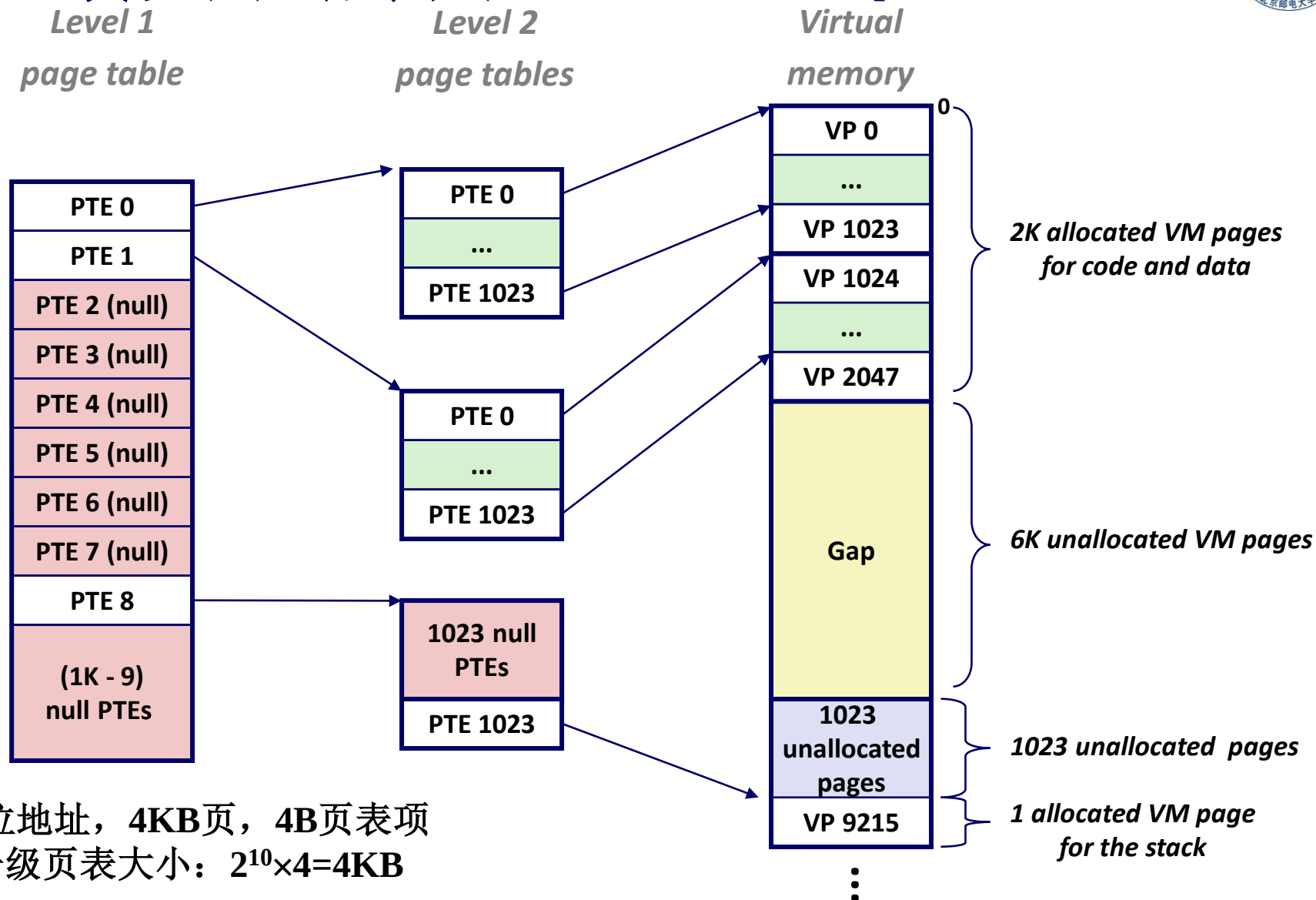
◆ 一级页表（驻留主存）：每个表项指向一个二级页表

◆ 二级页表（可被调入调出）：每个表项指向一个页



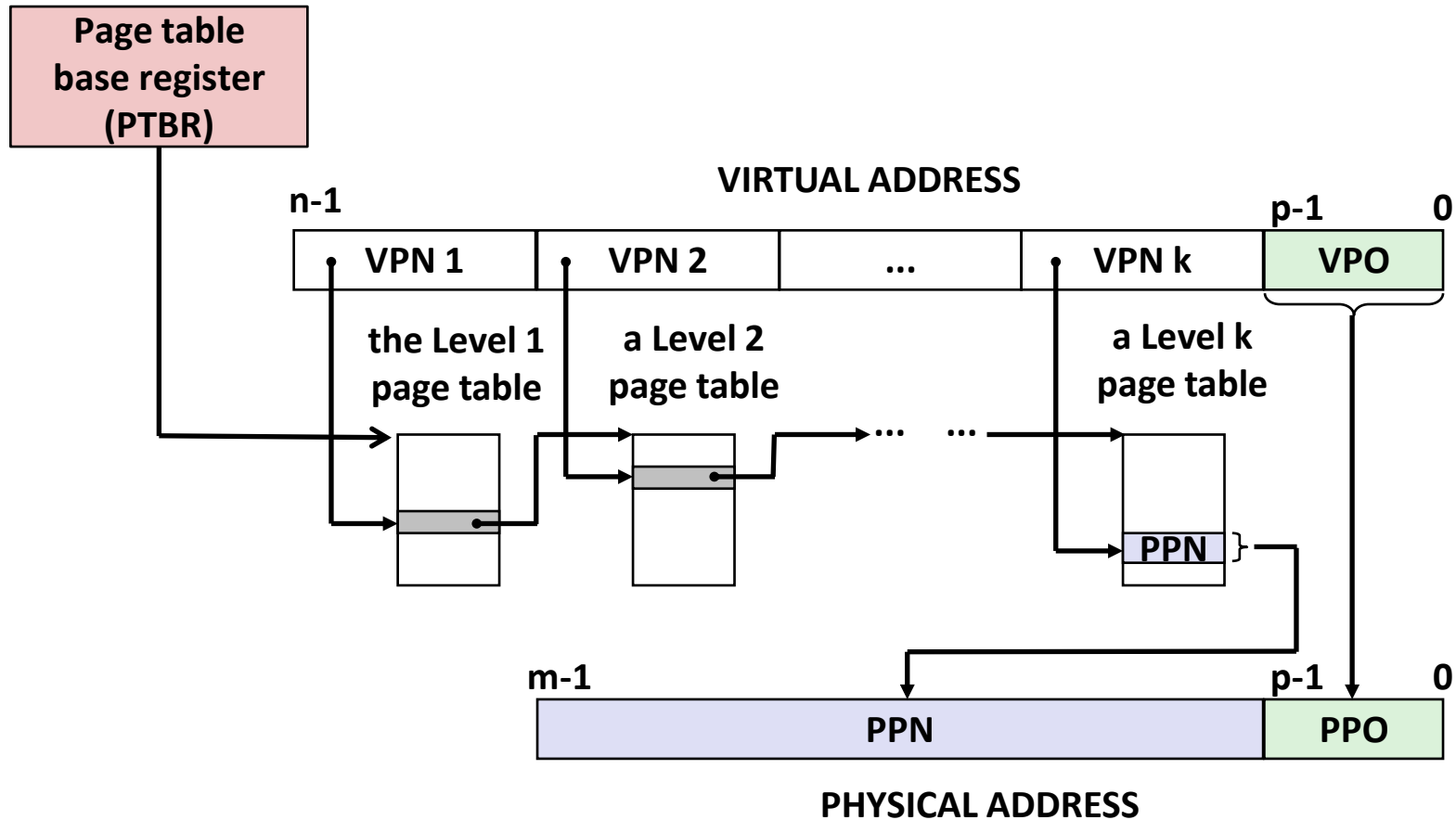


二级页表层次结构（32位地址）





使用k级页表的地址翻译





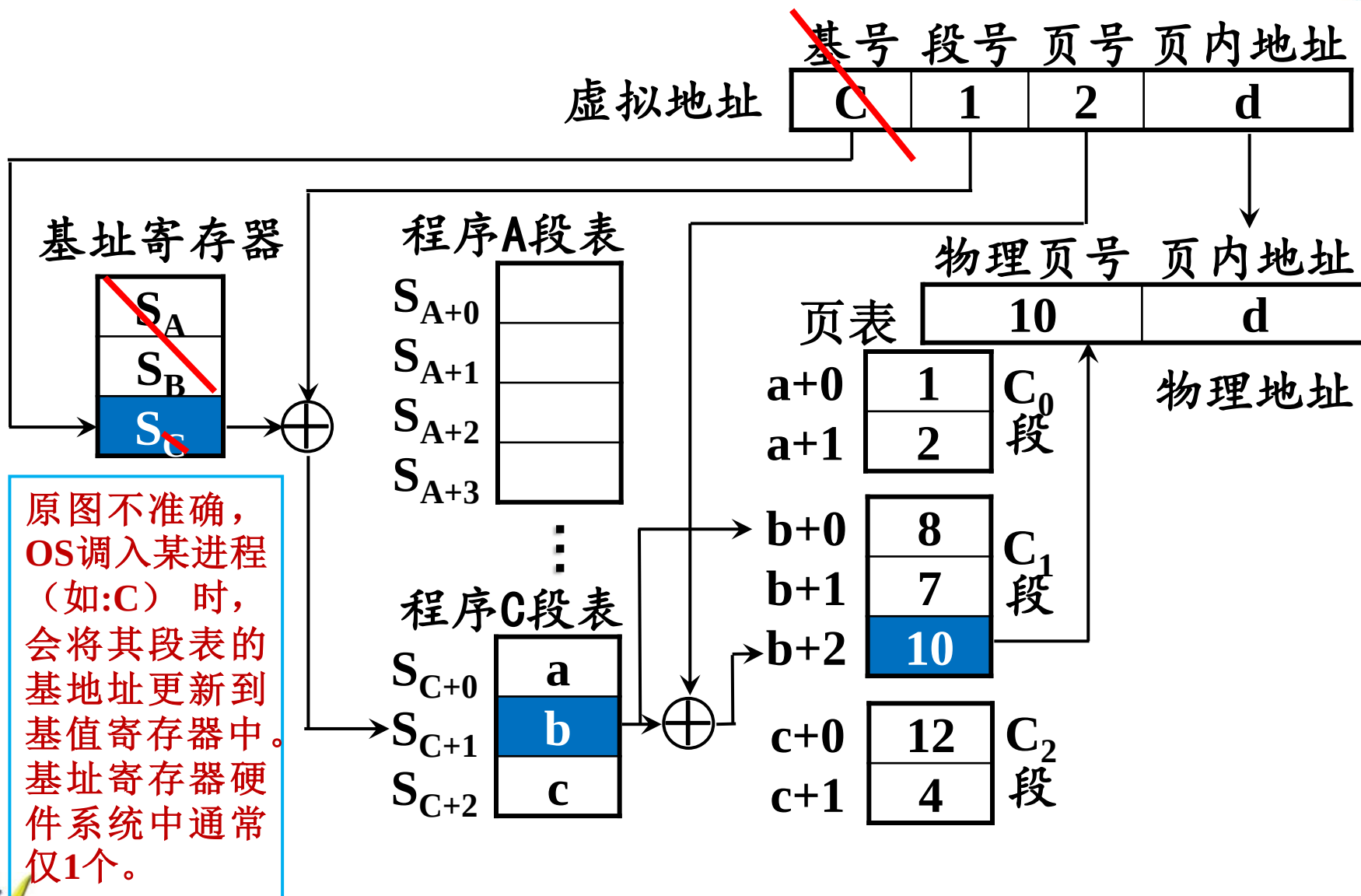
3. 段页式虚拟存储器

- 是段式虚拟寄存器和页式虚拟寄存器的结合
- 程序先按逻辑单位分段，再把每段分成固定大小的页
- 操作系统对主存的调入调出是**按页面**进行的，但它又可以按段实现共享和保护
- 兼有页式和段式系统的优点
- 缺点是在地址映射过程中需要**多次查表**。地址映射通过一个段表和一组页表来进行





例10 虚拟/物理地址翻译





页面替换算法（1）

- 当CPU要用到的数据或指令不在主存时，产生缺页异常（中断），要求从外存调进包含有这条指令或数据的页面
- 假如主存页面已全部被占满，那么要使用一定的替换算法来替换主存中的一页以接纳新调进的页面
- 页面替换策略与Cache中行替换策略的区别是：
 - ◆ 缺页至少要涉及一次磁盘存取，读取所缺的页，缺页使系统蒙受的损失要比cache未命中大得多
 - ◆ 页面替换是由操作系统软件实现的





页面替换算法（2）

■ 常用的替换策略

- ◆ LRU（最久未使用）算法
- ◆ LFU（最少使用）算法
- ◆ FIFO（先进先出）算法
- ◆ 或将两种算法结合起来使用

■ 写回处理

- ◆ 对于将被替换出去的页面，假如该页调入主存后没有被修改，就不必进行写回，否则就把该页重新写入外存，以保证外存中数据的一致性
- ◆ 为此，在页表的每一行需设置1个修改位





例11

- 假设主存只允许存放a、b、c三个页面，逻辑上构成a进c出的FIFO队列。某次操作中进程访存的序列是0、1、2、4、2、3、0、2、1、3、2（虚页号）。若分别采用FIFO算法、~~FIFO~~ + LRU算法，请用列表法分别求出两种替换策略情况下主存的命中率。

		0	1	2	4	2	3	0	2	1	3	2	命中率
	a	0	1	2	4	4	3	0	2	1	3	3	2/11=18.2%
	b		0	1	2	2	4	3	0	2	1	1	
	c			0	1	1	2	4	3	0	2	2	
						命中						命中	
	a	0	1	2	4	<u>2</u>	3	0	<u>2</u>	1	3	<u>2</u>	3/11=27.3%
	b		0	1	2	4	2	3	0	2	1	3	
	c			0	1	1	4	<u>2</u>	3	0	<u>2</u>	1	
						命中			命中			命中	





虚存与Cache的比较

■ 虚存

- ◆ “主存-辅存层次”，主要目的是解决**存储容量问题**
- ◆ 单位时间内数据交换次数较少，但每次交换的数据量大，达几十至几千字节。

■ Cache

- ◆ Cache主要目的是解决**存储速度问题**，使存储器的访问速度不太影响CPU的运行速度。
- ◆ 单位时间内数据交换的次数较多，每次交换的数据量较小，只有几个到几十个字节。





cache-主存和主存-辅存两个层次的相同点

■ 出发点相同：

- ◆ 为了提高存储系统的性能-价格比而构造的层次型存储体系
- ◆ 力图使存储系统的性能接近高速存储器，而价格和容量接近低速存储器。

■ 原理相同：

- ◆ 利用了程序运行时的局部性原理，把最近常用的信息块从相对低速而大容量的存储器调入相对高速而小容量的存储器。





cache-主存和主存-辅存两个层次的不同点

- 侧重点不同：
 - ◆ cache主要解决主存与CPU的速度差异问题
 - ◆ 虚存主要解决存储容量的问题
- 数据通路不同：
 - ◆ CPU与cache和主存之间均有直接访问通路，cache不命中时可直接访问主存
 - ◆ 辅存与CPU之间不存在直接的数据通路，主存不命中时只能通过调页解决
- 透明性不同：
 - ◆ cache的管理完全由**硬件**完成，对系统程序和应用程序均透明
 - ◆ 虚存管理由**软件**（操作系统）和硬件共同完成。虚存对系统程序不透明，而只对应用程序透明
- 未命中时的损失不同：
 - ◆ 主存未命中时系统性能损失要远大于cache未命中时的损失





小结：虚存的访问过程

- 虚存空间的用户程序按照虚地址编程并存放在辅存中
- 程序运行时，由地址变换机构依据当时分配给该程序的实地址空间把程序的一部分调入实存
- 每次访存时，首先判断该虚地址所对应的部分是否在实存中：
 - ◆ 是，则进行地址转换并用实地址访问主存
 - ◆ 否，则按照某种算法将辅存中的部分程序调度进内存，再按同样的方法访问主存
- 每个程序的虚地址空间可以远大于实地址空间
- 应用程序可以透明地使用整个虚存空间
 - ◆ 如果主存的命中率很高，虚存的访问时间就接近于主存访问时间

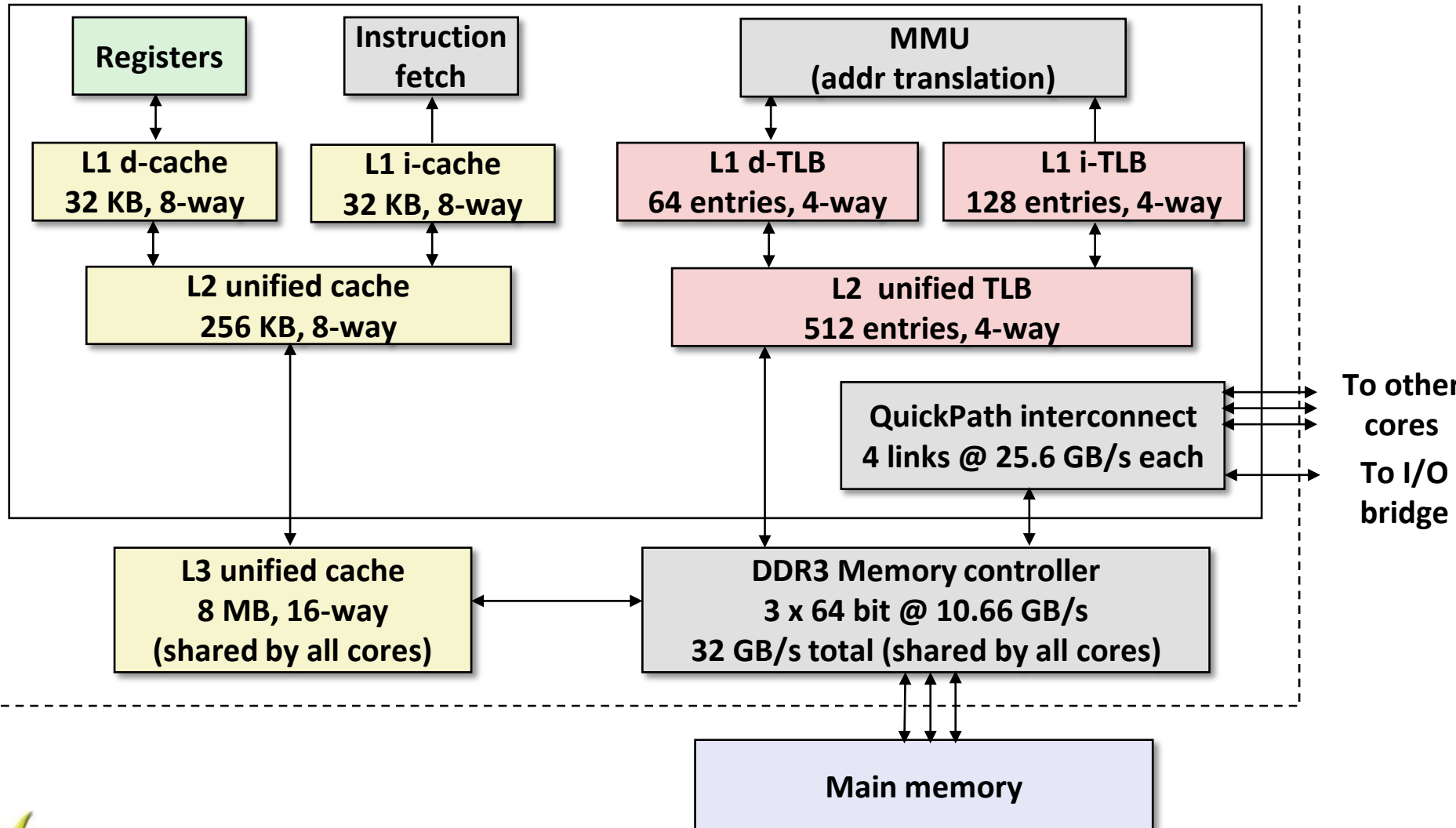




实例：Core i7存储器系统

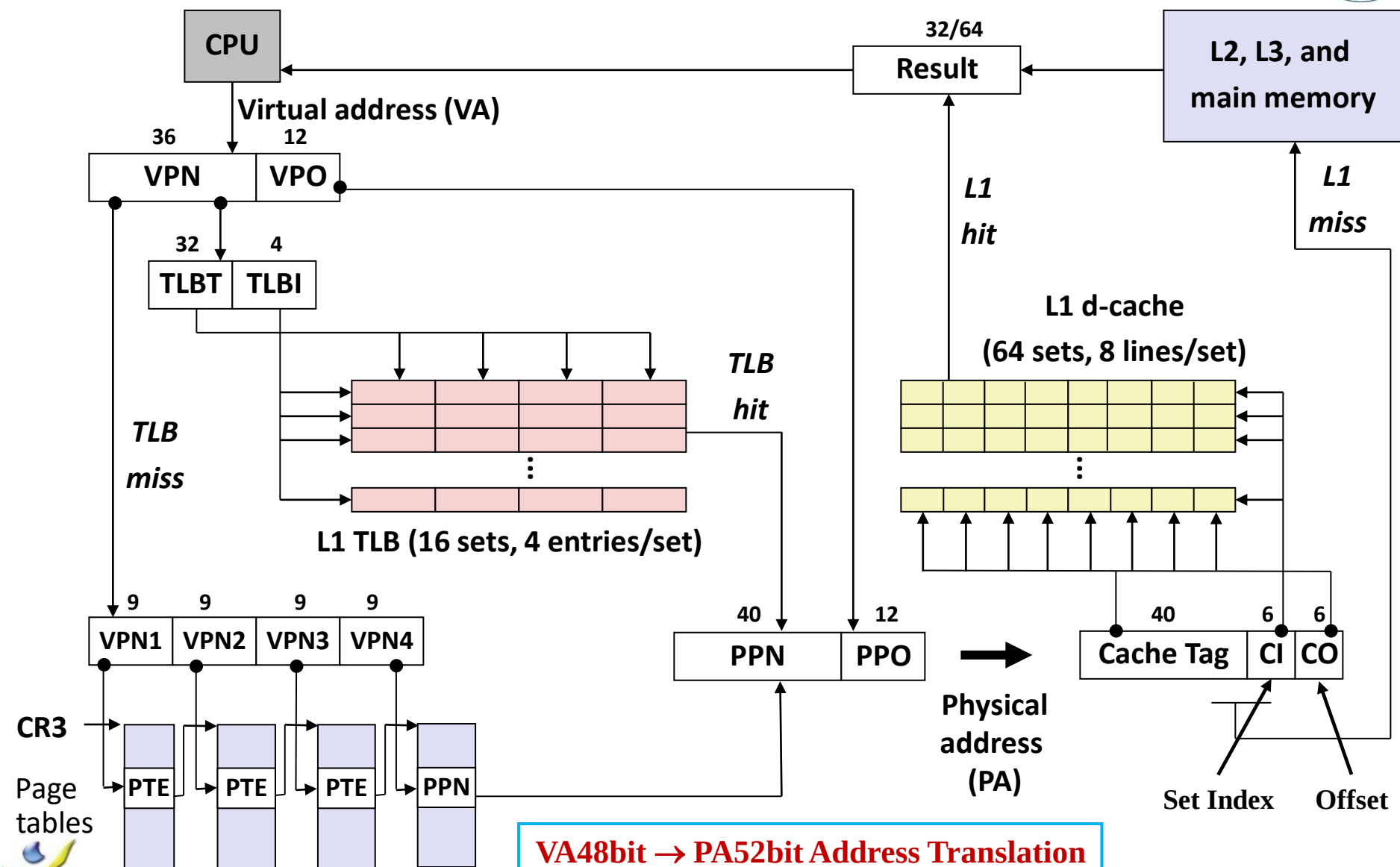
Processor package

Core x4



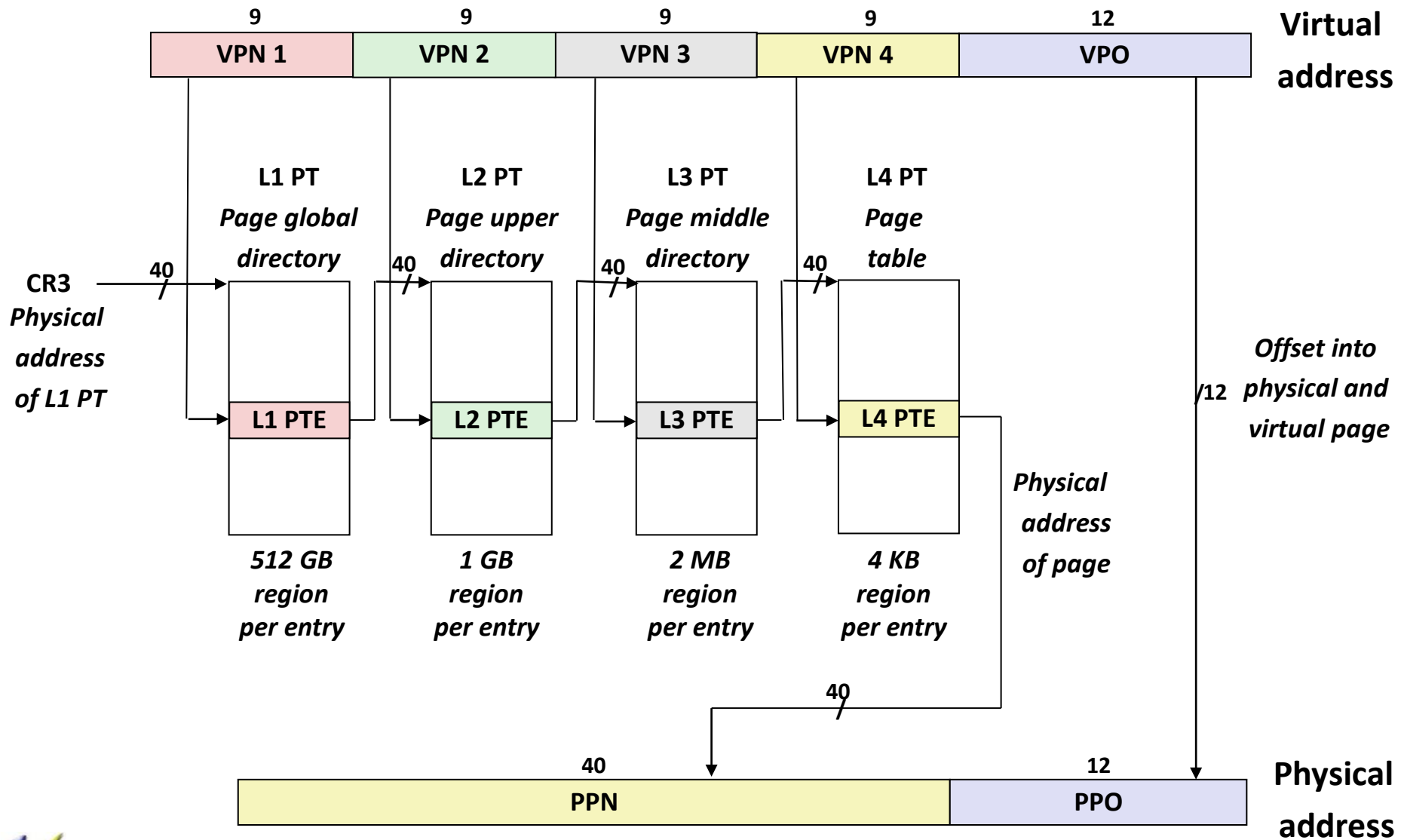


端到端的地址翻译





Core i7 页表翻译





Core i7 1-3级页表项

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused		Page table physical base address												Unused
							G	PS		A	CD	WT	U/S	R/W	P=1
Available for OS (page table location on disk)															P=0

Each entry references a 4K child page table. Significant fields:

P: Child page table present in physical memory (1) or not (0).

R/W: Read-only or read-write access access permission for all reachable pages.

U/S: user or supervisor (kernel) mode access permission for all reachable pages.

WT: Write-through or write-back cache policy for the child page table.

A: Reference bit (set by MMU on reads and writes, cleared by software).

PS: Page size either 4 KB or 4 MB (defined for Level 1 PTEs only).

Page table physical base address: 40 most significant bits of physical page table address (forces page tables to be 4KB aligned)

XD: Disable or enable instruction fetches from all pages reachable from this PTE.





Core i7 4级页表项

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page physical base address				Unused	G		D	A	CD	WT	U/S	R/W	P=1
Available for OS (page location on disk)															P=0

Each entry references a 4K child page. Significant fields:

P: Child page is present in memory (1) or not (0)

R/W: Read-only or read-write access permission for child page

U/S: User or supervisor mode access

WT: Write-through or write-back cache policy for this page

A: Reference bit (set by MMU on reads and writes, cleared by software)

D: Dirty bit (set by MMU on writes, cleared by software)

Page physical base address: 40 most significant bits of physical page address
(forces pages to be 4KB aligned)

XD: Disable or enable instruction fetches from this page.





鲲鹏920处理器片上系统的存储系统

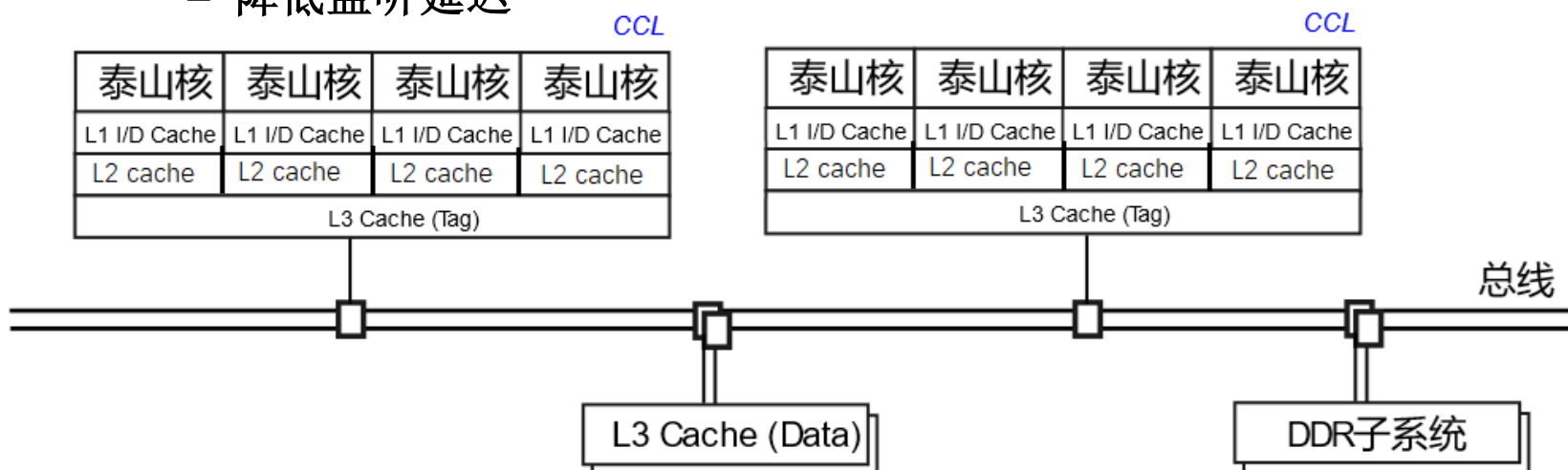




鲲鹏920处理器片上系统的内存存储系统

■ 多级Cache：片内共享L3 Cache架构

- ◆ 每个内核私有的L1 Cache和L2 Cache
- ◆ 每个超级内核集群配置共享L3 Cache
 - 系统级片上第三级高速缓存
 - L3 Cache数据块：处在内核集群之外
 - L3 Cache标记块：协议处理模块，集成在每个内核集群中
 - 降低监听延迟





泰山核内Cache和TLB特性

- 片内Cache: 两级片内Cache
 - ◆ 64KB L1指令Cache, 行大小均为64B
 - ◆ 64KB L1 数据Cache, 行大小均为64B
 - ◆ 每个内核私有的8路组相联512KB L2 Cache, 行大小64B
- 两级TLB系统:
 - ◆ 48表项全相联L1指令TLB
 - ◆ 32表项全相联L1数据TLB
 - ◆ 1152表项的4路组相联统一L2 TLB, 每个内核共享
- Cache和TLB差错校验:
 - ◆ L1数据Cache和L2 Cache: 支持ECC保护和SECDED差错校验
 - Single Error Correction- Double Error Detection
 - ◆ L1指令Cache和L2 TLB: 支持奇偶校验保护





鲲鹏920处理器片上系统L3 Cache

- 在系统启动期间，L3_DATA的全部存储空间都可以被当作一块**SRAM**使用
- 采用组相联结构，固定使用**Write-Back**策略
- 支持随机替换、**DRRIP**（动态重引用区间预测）和**PLRU**（**Pseudo LRU**）三种替换算法
- **Cache**行大小固定为128字节

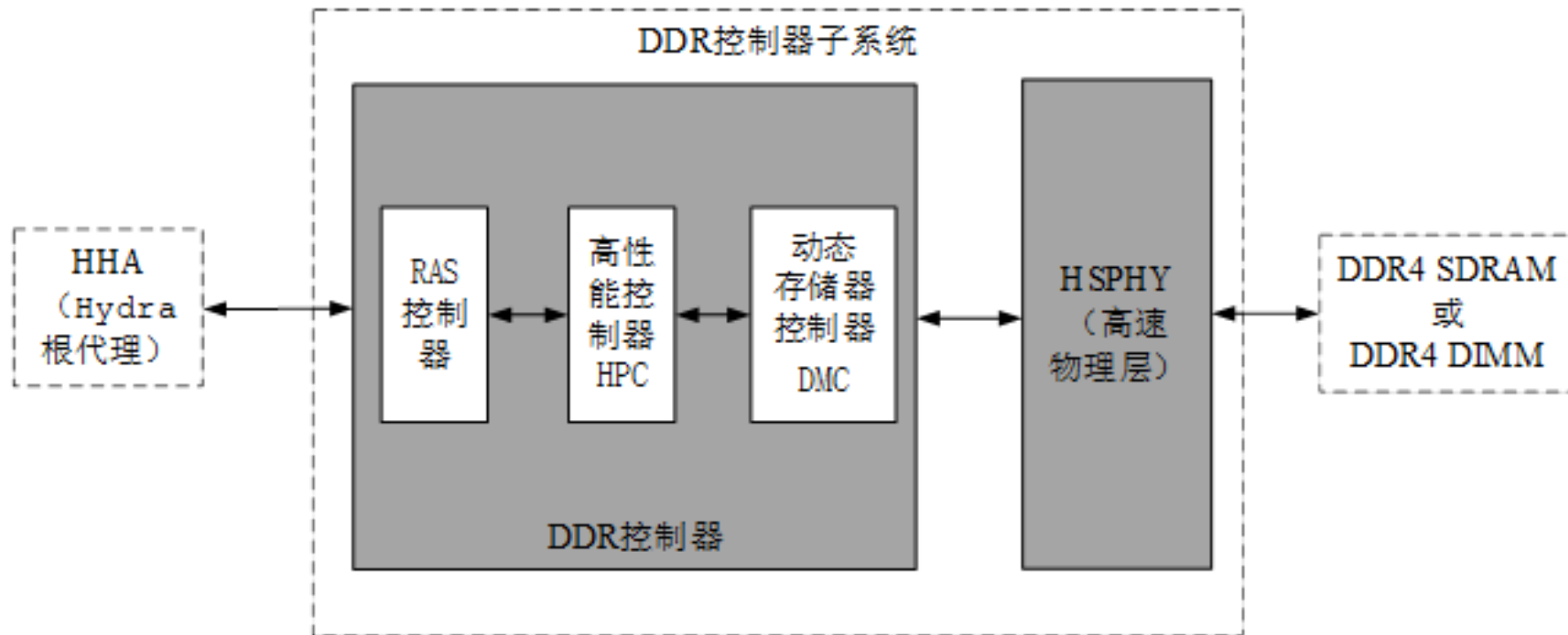




鲲鹏920处理器片上系统的主存系统

■ 鲲鹏920系统的主存储系统的组成部件

- ◆ Hydra根代理（HHA）
- ◆ **DDR SDRAM Controller（DDRC）**
- ◆ 片外DDR存储器颗粒或DDR DIMM内存条



Hydra协议： HiSilicon定义的支持在多芯片系统之间保持Cache一致性的规范





鲲鹏920处理器片上系统的主存系统

■ DDR SDRAM Controller

- ◆ 完成片上网络传输的访问请求到DDR SDRAM存储器的时序转换
 - 地址译码、调度、协议转换、接口时序处理等
- ◆ 8个DDR控制器子系统对应8个独立的DDR通道
 - 支持DDR3 SDRAM和DDR4 SDRAM存储器的颗粒或DIMM内存条
 - 每个DDR控制器子系统支持最多2个DIMM内存插槽
 - 每个DIMM插槽支持最多4个物理存储体





鲲鹏920处理器片上系统的地址映射与变换

- **TaiShan V110处理器**：支持48位的虚拟地址和48位的物理地址
- **处理器内核集群（CCL）**：访问存储器地址根据MMU的页表确定
- **I/O CLusters（ICL）**：访问的存储器地址根据SMMU的页表或者数据源确定
 - ◆ SMMU中的页表可以由硬件自动同步于CCL中的MMU，也可以独立配置





AArch64状态的地址变换

■ AArch64状态:

- ◆ 虚拟地址最大长度48位，虚拟地址空间大小256T字节

■ 地址变换：虚拟地址VA映射到物理地址PA

◆ 两级连续变换（Two Sequential Stages of Translation）

- 虚拟地址VA转换为40位的**中间物理地址**（Intermediate Physical Address, **IPA**）

- 中间地址IPA的长度：可配置，最大长度与最大物理地址长度相等

- IPA变换为物理地址PA

◆ 单级变换（Single Stage of Translation）

- IPA与物理地址相同





AArch64状态的地址变换

■ 两级变换支持虚拟化操作

◆ 第一级变换：EL1上运行客户操作系统

- 虚拟地址转换为中间物理地址IPA
- IPA：客户操作系统的“物理地址”
- 客户操作系统不知晓第二级变换的存在

◆ 第二级变换：EL2上运行虚拟机管理器

- 将客户操作系统的“物理地址”变换为真正的物理地址
- 通过虚拟机管理器管理的变换表实现

■ 不支持虚拟化操作：可以使用单级变换





Cache综合题 (1)

某计算机存储器按字节编址，虚拟（逻辑）地址空间大小为 16 MB，主存（物理）地址空间大小为 1 MB，页面大小为 4 KB；Cache 采用直接映射方式，共 8 行；主存与 Cache 之间交换的块大小为 32 B。系统运行到某一时刻时，页表的部分内容和 Cache 的部分内容分别如题 44-a 图、题 44-b 图所示，图中页框号及标记字段的内容为十六进制形式。

虚页号	有效位	页框号	...
0	1	06	...
1	1	04	...
2	1	15	...
3	1	02	...
4	0	-	...
5	1	2B	...
6	0	-	...
7	1	32	...

题 44-a 图 页表的部分内容

行号	有效位	标记	...
0	1	020	...
1	0	-	...
2	1	01D	...
3	1	105	...
4	1	064	...
5	1	14D	...
6	0	-	...
7	1	27A	...

题 44-b 图 Cache 的部分内容





Cache综合题 (2)

请回答下列问题。

- (1) 虚拟地址共有几位，哪几位表示虚页号？物理地址共有几位，哪几位表示页框号（物理页号）？
- (2) 使用物理地址访问 Cache 时，物理地址应划分成哪几个字段？要求说明每个字段的位数及在物理地址中的位置。
- (3) 虚拟地址 001C60H 所在的页面是否在主存中？若在主存中，则该虚拟地址对应的物理地址是什么？访问该地址时是否 Cache 命中？要求说明理由。
- (4) 假定为该机配置一个 4 路组相联的 TLB，该 TLB 共可存放 8 个页表项，若其当前内容（十六进制）如题 44-c 图所示，则此时虚拟地址 024BACH 所在的页面是否在主存中？要求说明理由。

组号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号	有效位	标记	页框号
0	0	-	-	1	001	15	0	-	-	1	012	1F
1	1	013	2D	0	-	-	1	008	7E	0	-	-

题 44-c 图 TLB 的部分内容



Cache综合题 (3)

解答:

- (1) 虚地址 24位 页大小4KB, 故虚页号占 $24-12=12$ 位
物理地址 20位, 页框号 (物理页号) 占 $20-12=8$ 位
- (2) 直接映射方式, 物理地址访问cache时分为3部分:
标签 12位 + 行号3位 + 块内地址5位
- (3) 虚地址001C60H, 虚页号001, 查页表有效位为1,
对应的物理地址为04C60H- $\rightarrow$ 000001001100 011 00000
011查cache目录得105H $\neq$ 04CH 故cache未命中
- (4) 虚地址024BACH, 024H- $\rightarrow$ 00000010010 0, 查TLB的0组,
00000010010B=012H, 标记相等且有效位为1, 故页面
在主存

